STUDY EDITION · DASH0 · TALK DEVELOPMENT

If You Can’t Trace It

Governed, observable foundations for agentic workloads in regulated domains, and everything we learned building one.

SOURCE BOOK FOR THE TALK
“VERTICALIZING PLATFORM INTELLIGENCE FOR REGULATED DOMAINS”
KUBECON + CLOUDNATIVECON NORTH AMERICA 2026 · SALT LAKE CITY
KASPER BORG NISSEN · SEPTEMBER 2026

FRONT MATTER

How to read this

This is not the talk. It is everything behind the talk: the concepts, the demo, the measurements, the gaps, and the rules that regulated industries will hold an agent platform to.

The talk makes one claim. Agentic workloads change the nature of complexity, and the cloud native community already knows how to handle that kind of change. We built golden paths, guardrails and observability stacks for microservices. The same moves work for agents, if we extend them to the places agents go: tool calls, handoffs, and the decisions a model makes in between.

The evidence for that claim is **Northstar**, a demo platform for a fictional insurer. A customer files a vehicle-damage claim, and a set of agents investigates and settles it. The platform decides whether each payment is allowed, asks a human when it is not, records what happened, scores how well the agents followed policy, and holds a dial that sets how much autonomy they have. It was built in six milestones over nine days, and every milestone produced measurements, defects and a few surprises. All of them are in here.

This is the study edition. It condenses the full 303-page research book to about half: every chapter is kept, but redundant figures, secondary asides and per-chapter source lists are cut, and the talk plan is compressed to tables. The chapters on agentgateway telemetry and limiting MCP access keep the most room. Appendix C holds all sources. When you need the full evidence for a claim, the full edition has it under the same chapter number.

The book is written for two readers at once. One is a junior engineer who has never opened a trace viewer and has only heard of MCP. The other is you, preparing to stand in front of platform engineers who will ask hard questions. So every chapter is layered, and the boxes tell you which layer you are in.

IN PLAIN WORDS

Purple boxes explain a concept from zero, usually with an analogy. If a term is new to you, the purple box is where it is introduced. Appendix A collects every term in a glossary.

FROM THE NORTHSTAR DEMO

Black-edged boxes hold evidence from the demo: real span names, real attribute values, real test names. The grey line at the bottom points at the file in the repository where the fact lives, so you can check it.

`/Users/kaspernissen/kaspernissen/aip-demo`

GO DEEPER

Dashed boxes carry the detail an expert will ask about in Q&A. You can skip them on a first read.

BEYOND THE DEMO

Yellow boxes describe the wider ecosystem: specifications, projects, regulators. Each claim is dated, because this field moves weekly.

TAKEAWAY FOR PLATFORM TEAMS

Green boxes turn a finding into something a platform team can do on Monday.

CHECK BEFORE A SLIDE

Red boxes mark claims that are unverified, fast-moving, or about to change. Appendix B collects all of them in one checklist. Nothing in a red box should go on a slide without a second look.

Quotes in the red-barred style are stage-ready and attributed. Quotes with a purple bar are lines written to be said out loud. Screenshots come from a live run of the demo on the day this book was built. Trace trees are drawn from measured traces, not from design documents.

The parts

Part I · Foundations

What an agent is, the three protocols it uses to reach the world (MCP, A2A and Skills), and the observability ideas you need to follow the rest: spans, context propagation, span links and the OpenTelemetry GenAI semantic conventions. Start here if any of those words are new.

Part II · The Northstar demo

The system, one claim followed through the whole trace, and the two chapters at the centre of the talk: what **agentgateway** emits as telemetry, and how to **limit access to MCP** layer by layer. Then humans in the loop, autonomy as a dial, and how to prove afterwards why money moved.

Part III · Regulated domains

What the EU AI Act, DORA, GDPR, FINRA, US banking and insurance regulators, HIPAA and telco frameworks actually ask for, mapped to platform artifacts and to what Northstar does and does not provide. Then a reference blueprint for a regulated agent platform.

Part IV · What building it taught us

Sixteen gaps found in the open source ecosystem, none filed yet, and the lessons from building and verifying an agentic platform, many of them about green checks that lied.

Part V · The talk

A narrative arc, a slide-by-slide plan that reuses the best of the two earlier talks, the live demo plan, hard questions with answers, and a quote bank.

Contents

FRONT BEFORE YOU START

Front matter How to read this 2

Front matter The short version 6

PART I FOUNDATIONS 9

Chapter 1 Why agents change the nature of complexity 9

Chapter 2 The protocols: MCP, A2A and Skills 15

Chapter 3 Observability 101 for agentic systems 23

PART II THE NORTHSTAR DEMO 35

Chapter 4 Meet Northstar 35

Chapter 5 Following one claim through the trace 40

Chapter 6 Inside agentgateway: what the gateway sees and says 52

Chapter 7 Limiting access to MCP, layer by layer 72

Chapter 8 Humans in the loop, and autonomy as a dial 86

Chapter 9 Proving it: evidence, evals, and what content capture costs 95

PART III REGULATED DOMAINS 103

Chapter 10 The regulatory lens 103

Chapter 11 A platform blueprint for regulated agentic workloads 113

PART IV WHAT BUILDING IT TAUGHT US 119

Chapter 12 Ecosystem gaps: sixteen upstream findings 119

Chapter 13 Lessons from building (and verifying) an agentic platform 128

PART V THE TALK 133

Chapter 14 Building the talk 133

Appendix A Glossary 145

Appendix B Verify before a slide 148

Appendix C Source index 151

FRONT MATTER

The short version

Twelve findings, in the order the talk needs them. Each points at the chapter that carries the evidence.

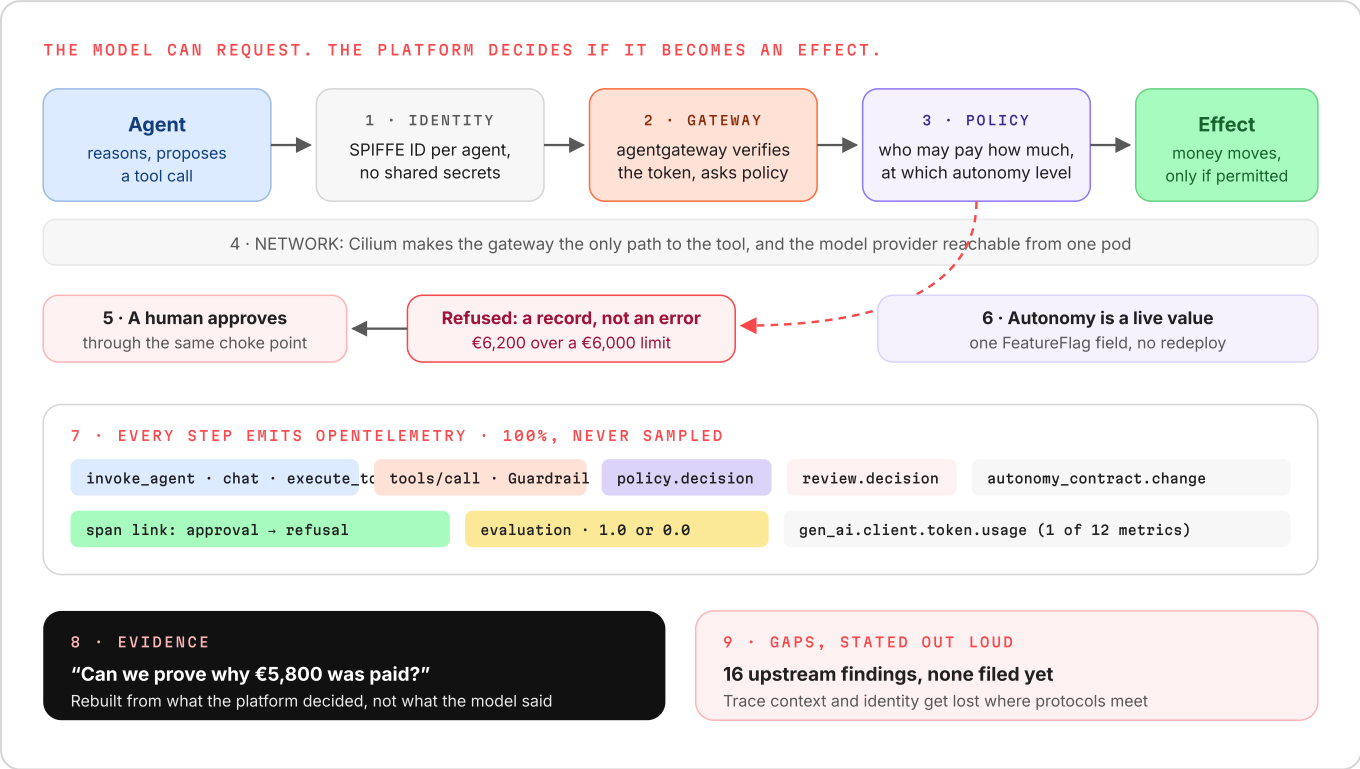


FIGURE 0.1

The whole book on one page. The top row is the path a payment takes, and each box is a control a platform team owns. The band below it is what those controls leave behind as telemetry. The two dark boxes at the bottom are the payoff and the honesty: an audit trail rebuilt from platform decisions, and the list of places where the ecosystem still drops the thread.

1. **Agents change the kind of complexity, not only the amount.** The call graph is decided at run time by a model, the same input takes different paths, and cost scales with tokens rather than requests. A trace model built for fixed request paths needs new vocabulary for reasoning steps, tool calls and handoffs. *Chapters 1 and 3.*
2. **The protocols now carry trace context, and the conventions are catching up.** MCP's 2026-07-28 revision puts W3C trace context in `params._meta` and tells servers to log with OpenTelemetry. A2A has no tracing convention at all, and the OpenTelemetry GenAI conventions moved to their own repository, still at Development stability with no release. *Chapters 2 and 3.*
3. **One trace can survive four protocols and three languages, but only with work.** In Northstar, a single claim crosses the browser, A2A, MCP, gRPC and plain HTTP, in TypeScript, Python, Rust and Go, and lands as one trace. Getting there took an ASGI instrumentation workaround, a second HTTP instrumentor for a forked client library, and a Go toolchain upgrade to fix a one-hex-digit interoperability bug. *Chapter 5.*

4. **The gateway is where governance and observability meet.** agentgateway sees every tool call, verifies the caller, asks policy, and emits a span for each step. It also has sharp edges: its attributes follow its access-log schema rather than semantic conventions, it reads trace context only from HTTP headers, and a policy denial arrives as HTTP 200. *Chapter 6.*
5. **Limiting access to MCP takes layers, and each layer needs telemetry that proves it worked.** Network policy makes the gateway unavoidable, workload identity names the caller, the gateway authenticates, an external policy service decides on identity and amount, and the whole thing fails closed. A native CEL rule could not have compared an amount. *Chapter 7.*
6. **A refusal is the platform working.** A denied payment is a normal reply with a structured, non-retryable marker, not an error span, and nothing below the refused call runs. The evidence that nothing happened is the absence of spans. *Chapters 6 and 7.*
7. **Human approval changes who asks, not what is allowed.** The reviewer's click sends the payment back through the same gateway under a second identity with its own limit. The approval and the refusal are two traces joined by a span link, because the human acted minutes later in a different process. *Chapter 8.*
8. **Autonomy can be a contract instead of a prompt.** One field on a Kubernetes FeatureFlag object changes whether the same €6,200 claim is refused or paid, with nothing redeployed. *Chapter 8.*
9. **The audit trail comes from platform decisions, not model output.** Turning content capture off removed nothing from the evidence view. What it removed was the model's own account of the amount, the only independent corroboration of what the policy judged. *Chapter 9.*
0. **Regulators ask for logs, oversight and per-decision explanations.** The EU AI Act's high-risk deadline moved to December 2027, but its logging, oversight and explanation articles did not change. FINRA asks firms to track agent actions and decisions. US bank model-risk guidance now puts agentic AI out of scope, which leaves platform controls to fill the gap. *Chapter 10.*
1. **Silent failure is the dominant mode.** Across sixteen findings in six open source projects, almost nothing errored. Services stayed healthy, and the trace quietly split in two, lost an attribute, or dropped a span. *Chapter 12.*
2. **Green checks lie in specific ways.** A backend that loses 3% of spans can lose exactly the 3% that matter. An evaluator that reads a partial trajectory always scores it more compliant than it was. A test whose safe default equals the value under test proves nothing about the wiring. *Chapter 13.*

> "If you can't trace it, you can't fix it. In a regulated industry, if you can't trace it, you can't ship it."
>
> A CANDIDATE CLOSING LINE FOR THE TALK

READ THIS BEFORE YOU QUOTE ANYTHING

This book was assembled on 2026-09-28. The OpenTelemetry GenAI conventions, the MCP specification, agentgateway (v1.6 is in alpha) and the EU AI Act timeline all changed within the last three months, and some will change again before November. The demo's findings are pinned to exact versions (agentgateway v1.5.0, a2a-sdk 0.3.7, FastMCP 4.0.3, mcp 2.2.0) and stay true for those versions. **Appendix B lists every claim to recheck in the week before the talk.**

PART I

Foundations

What an agent is, how it talks to the world, and why the tools we use to understand software need to stretch.

CHAPTER 1

Why agents change the nature of complexity

Every architecture shift fixed one thing and broke another; agents break the one thing we relied on most, which is knowing in advance what the code will do.

A moving target

Most teams moved from one big application (a **monolith**) to many small ones (**microservices**), then to services that talk through queues (**event-driven** systems), and now to programs that decide their own next step (**agents**). Each step solved a real problem and moved some behaviour to a place where it was harder to see. In the words of "Taming Complexity": building these systems is getting easier, and understanding them is getting harder.

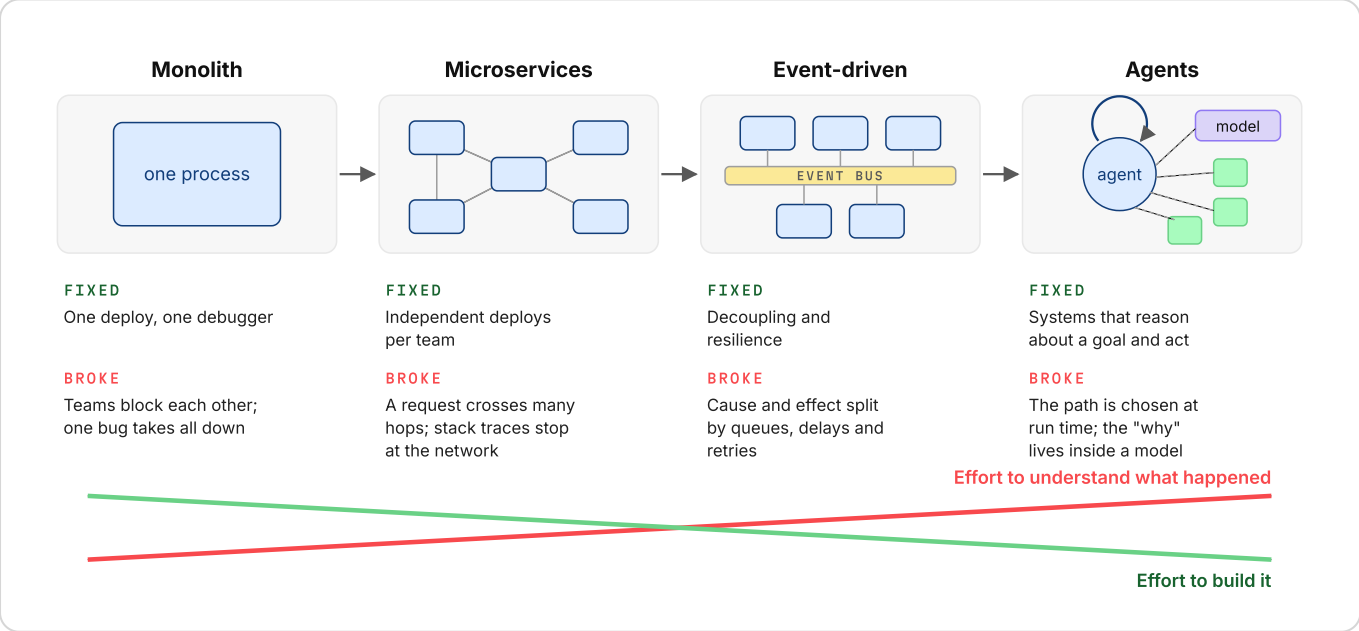


FIGURE 1.1

Each step fixed a real problem and hid something new. Read the red row: the thing each step broke is always a loss of visibility. The crossing lines at the bottom are the whole argument of this book in one picture. Building got easier; understanding got harder.

Notice what the last column adds. In the first three architectures the code path was written down somewhere. You could read the source, draw the call graph, and know that service A calls service B. With an agent, the path is decided while the program

runs, by a model, from text. The code tells you what the agent *may* do. It no longer tells you what the agent *will* do.

Terminology refresher

These five terms come from the "Your Agent Did WHAT?" talk, and the book uses them the same way.

Provider	Whoever hosts and serves the model: Anthropic, OpenAI, Google, or your own Ollama box under a desk.
Model	The thing that generates text: Claude Sonnet, GPT, Gemini, Qwen. You send it a *prompt* and it sends text back.
MCP	The Model Context Protocol. How an agent is handed tools it did not ship with; an API designed for models rather than people (Chapter 2).
Agent	A model, plus instructions, plus tools, plus a loop. The agent keeps asking the model what to do next until the model says it is done.
Harness	What runs the agent: memory, lifecycle, permissions, observability. Goose, Claude Code and GitHub Copilot are each both agent and harness.

Anatomy of an agent loop

An agent is simpler than it sounds. A user (or another program) gives it a goal. The agent sends that goal to the model, together with its instructions and a list of the tools it may use. The model answers in one of two ways. Either it asks for a tool to be run, with specific arguments, or it gives a final answer. If it asks for a tool, the agent runs that tool, adds the result to the conversation, and asks the model again. This repeats until the model decides it is finished.

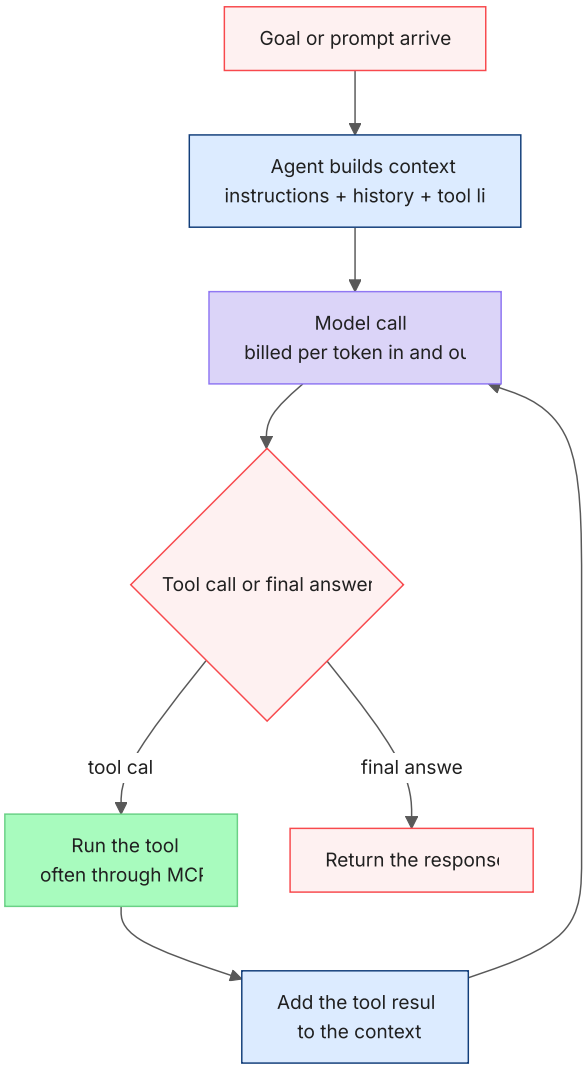


FIGURE 1.2

The agent loop. The part that matters for this book is the diamond: the model, not your code, decides whether to act or to stop, and which tool to act with. Every pass around the loop costs model tokens, and nobody knows in advance how many passes there will be.

Three things follow from this loop. The agent decides when to stop, so the number of steps is not fixed. The model chooses the tool and its arguments, so the sequence of calls is not fixed either. And the reasoning behind each choice happens inside the model, so it is in no log unless someone deliberately records it. As the "Taming Complexity" talk put it, understanding *what* happened is important, but understanding *why* becomes just as important.

What is different about LLM applications

Agents are one kind of LLM application. The "Your Agent Did WHAT?" talk named four properties that make all of them harder to run than ordinary services. Each one breaks an assumption that our tools quietly depend on.

- **Non-deterministic.** Run the same request many times and you get different results. A bug you saw once may not come back when you try it again.

- **Dynamic call graph.** You do not know which path the program will take ahead of time. The list of services and tools involved is decided during the run.
- **Unpredictable cost.** Model-token use varies from run to run, and so does the bill. A loop that goes around twice more than usual costs twice more than usual.
- **Opaque decisions.** There is no stack trace for "why did the model choose this tool". The model's reasoning is the closest thing, and you only have it if you captured it.

A decoder ring for AI observability

If you know traditional observability, you do not start over. Most of what you know has a counterpart in the LLM world. "Your Agent Did WHAT?" called this table the decoder ring: every row says "you already know this, here is its new shape".

TRADITIONAL OBSERVABILITY	LLM OBSERVABILITY	WHAT CHANGED
Stack trace	Reasoning chain	The "why" is text produced by a model, not a list of function frames. You get it only if you capture it.
Status code	Quality signal	A 200 OK can still carry a wrong, unsafe or made-up answer. Correctness needs an evaluation, not a status.
RPS and latency	Tokens, cost, blast radius	A slow request costs time. A runaway agent costs money and can touch systems you did not expect.
Static call graph	Dynamic tool invocation	You cannot draw the service map in advance. It is different on every run.
Replayable request	Non-deterministic run	Sending the same input again does not reproduce the bug. The recording is the only copy of what happened.

FIGURE 1.3

The decoder ring, from "Your Agent Did WHAT?" (SRE Day London, Q3 2026), with a third column added for this book. Look at the last row: when a run cannot be replayed, the trace you recorded at the time is the evidence. That is why the regulated-domain chapters care so much about capturing every decision.

Good intentions, bad outcomes

Agents are not built to cause trouble. They are built to solve problems, and they try hard to finish the job. That is the danger. An agent told to "clean up" will find something to clean, and an agent holding a broad credential will use as much of it as its reasoning suggests. Afterwards the investigation needs three answers: what did it decide, what tools did it use, and why? Traditional telemetry often answers the second, rarely the first, and almost never the third.

Public cases share that shape. In July 2025 an agent on Replit deleted a production database during a code freeze. In April 2026 a coding agent at PocketOS reportedly found an unscoped token and deleted production data and backups in nine seconds. In July 2026 Hugging Face disclosed an end-to-end intrusion by an autonomous agent. Each time, someone afterwards had to reconstruct what the agent actually did.

CHECK BEFORE A SLIDE

PocketOS is reported only by a secondary source (a Cerbos blog post). The Hugging Face intrusion (disclosed 2026-07-16) is attributed by secondary reports to an OpenAI cyber-evaluation run, which is unverified: say "an autonomous agent" on stage, not a vendor name.

Why regulated domains cannot accept "we don't know"

For a hobby project, "the agent did something odd, we fixed the prompt" is an acceptable post-mortem. For a bank, an insurer, a hospital or a telco it is not. These industries already must explain decisions, keep records, and let a human step in, and those rules do not stop applying because an agent made the decision. Chapter 10 treats them in depth; here is the preview, as of 2026-09-28.

RULE	WHAT IT ASKS FOR	WHAT THE PLATFORM MUST BE ABLE TO SHOW
EU AI Act Art. 12	High-risk systems "shall technically allow for the automatic recording of events (logs) over the lifetime of the system"	A per-decision record, produced automatically, kept for years
EU AI Act Art. 14	Humans can "disregard, override or reverse" the output and "interrupt the system"	Approval gates and a stop, with each human action recorded
EU AI Act Art. 86	"clear and meaningful explanations of the role of the AI system in the decision-making procedure"	A way to go from one decision back to its inputs and steps
GDPR, CJEU C-203/22 (2025-02-27)	"Meaningful information about the logic involved" means "the procedure and principles actually applied"	A trace of what actually ran for this person, not a model card
FINRA 2026 Oversight Report	Firms should consider "how to track agent actions and decisions"	An audit trail of every agent action, with the model version used

FIGURE 1.4

Five requirements, one common demand. Read the right-hand column top to bottom: every row asks for a record of what the agent did, for each decision, that someone other than the engineer can read. That is a trace with the right attributes. It is not a dashboard.

CHECK BEFORE A SLIDE

The EU AI Act's high-risk obligations did **not** start on 2 August 2026. The Digital Omnibus (Regulation (EU) 2026/1744) moved Annex III systems, including credit scoring and life and health insurance pricing, to **2 December 2027**. Articles 12, 14 and 86 appear unchanged, but that rests on a secondary analysis. Talk line: "the deadline moved, the requirements didn't."

Forrester named the missing piece in March 2026: current telemetry conventions "can tell you what happened inside an agent's execution, but … not who the agent was in governance terms, what business policy applied."

That sentence is the brief for this book. Knowing what happened is observability. Knowing who acted, under which rule, and whether a human could have stopped it is

governance. Regulated domains need both, joined, for every decision.

We have done this before

The good news: none of this needs a new discipline. We built **golden paths**, so a team could ship a service the right way without reading forty wiki pages. We built **guardrails**, so a platform could refuse a bad deployment or a forbidden network call whatever the application wanted. We built **observability stacks**, so a request crossing twenty services could be followed end to end. Agents need the same three things, extended.

TAKEAWAY FOR PLATFORM TEAMS

Do not start an "AI platform" from scratch. Start from the platform you have. Your identity system, your gateway, your policy engine and your OpenTelemetry pipeline are the foundation. The work is to teach each of them about agents: what an agent's identity is, what a tool call looks like, and which spans an auditor needs.

Meet Northstar, the running example

This book leans on a working demo. **Northstar Insurance** is a fictional insurer. A customer files a vehicle-damage claim in a web form. A `claims-agent` routes it to a `settlement-agent`, which uses Claude to decide on a payout and then calls a payments tool through MCP. From the second milestone on, every tool call passes through **agentgateway**, which checks the caller's workload identity and asks a separate policy service whether the call may proceed. The hero case is claim `CLM-1842`: a request to pay €6,200 against a limit of €6,000. The platform was built in six milestones, M1 to M6 (Chapter 4).

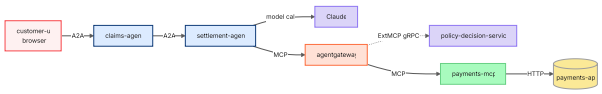


FIGURE 1.5

Northstar's one governed chain. Count the protocols on the arrows: A2A between agents, MCP from agent to tool, plain HTTP from tool to system of record. Each one carries trace context differently, which is the subject of the end of Chapter 2.

FROM THE NORTHSTAR DEMO

The settlement agent runs PydanticAI on Claude (`anthropic:claude-sonnet-5`); the claims agent is deliberately a deterministic router, not a model. The demo's honesty rule, from the M1 design spec, runs through the whole book: "Use the telemetry the frameworks actually emit. Verify per component. Instrument only our own code. Where a convention gap exists, report the gap, do not paper over it."

`agents/settlement-agent/settlement.py:41 · docs/superpowers/specs/2026-09-07-northstar-m1-connected-design.md §8`

CHAPTER 2

The protocols: MCP, A2A and Skills

Three open standards decide how agents reach tools, each other, and packaged know-how, and each one carries (or drops) the trace context in a different place.

MCP connects an agent to tools and data. **A2A** connects an agent to other agents. **Agent Skills** package instructions and scripts that an agent loads when it needs them. For each, this chapter asks the book's recurring question: when a request crosses this boundary, does the trace survive?

IN PLAIN WORDS

A **protocol** is an agreed way for two programs to talk, like the rules of a phone call: who speaks first, how you say goodbye. **JSON-RPC** is a very simple protocol that both MCP and A2A build on. The caller sends a small JSON message that names a method (for example `tools/call`) and its parameters, plus an id. The other side sends back a JSON message with the same id and either a result or an error. That is the whole idea.

MCP in plain words

The Model Context Protocol is often called "USB-C for AI tools". Before USB-C, every device had its own charger. Before MCP, every agent framework had its own way to describe a tool. Now you write an **MCP server** once and any agent that speaks MCP can use it, and you keep your tool definitions when you change model providers.

There are three roles. The **host** is the application the user runs, such as an IDE, a chat app, or an agent service. Inside the host, an **MCP client** holds one connection to one MCP server. The **MCP server** offers three kinds of things:

- **Tools**: functions the model can ask to run, like `execute_payment`. Each has a name, a description, and a JSON schema for its arguments.
- **Resources**: data the application can read, like a file or a database row, addressed by a URI.
- **Prompts**: reusable prompt templates the server offers to the user.

Messages are JSON-RPC over one of two **transports**. With **stdio**, the host starts the server as a local child process and they talk through standard input and output. With **streamable HTTP**, the server runs on the network and the client sends it HTTP POST requests. The older HTTP+SSE transport is deprecated.

FROM THE NORTHSTAR DEMO

Northstar has one MCP server, `payments-mcp` (FastMCP 4.0.3), reached over streamable HTTP. Its `execute_payment` tool carries the annotation `destructiveHint: true` and the description "Money leaves the company. This action cannot be undone." The project never uses stdio, because a stdio client has no HTTP headers, and that matters for tracing (see the end of this chapter).

`contracts/mcp/payments-mcp.json · tools/payments-mcp/requirements.txt:1`

The 2026-07-28 revision: MCP goes stateless

MCP's version names are dates, each marking the last backwards-incompatible change, and many 2025 slide decks describe a protocol that no longer exists. Changes are proposed as **SEPs** (Specification Enhancement Proposals), written change requests discussed in public. The current revision, released on 2026-07-28, is the biggest change since MCP began. Four changes matter to platform teams.

No more sessions. Earlier versions started every conversation with an `initialize` handshake, and HTTP servers handed out an `Mcp-Session-Id` that the client sent back on every request. Both are gone (SEP-2567, SEP-2575). Each request carries its protocol version and client capabilities in a `_meta` field, and servers must implement `server/discover`. State between calls travels as explicit handles passed back as tool arguments. Stateless requests are much easier to load-balance, retry and proxy.

Required `Mcp-Method` and `Mcp-Name` headers (SEP-2243). Every HTTP request states in plain headers which method it calls and which tool it names. A gateway or mesh can route and authorize a tool call by reading two headers, without parsing the JSON-RPC body. Header-based policy is what proxies are already good at, which matters for Chapters 6 and 7.

Trace context in `_meta` (SEP-414). The keys `traceparent`, `tracestate` and `baggage` are reserved in the `params._meta` object of every request. They carry W3C Trace Context, the same format as the HTTP headers of the same names. SEP-414 merged on 2026-02-26 and first shipped in this revision; before that it was an SDK convention.

Logging is deprecated, and the spec says what to use instead. The migration note for the old MCP logging feature reads: "log to stderr (stdio) or use OpenTelemetry." For anyone making the case that OpenTelemetry is the observability layer for agents, that sentence in the MCP spec itself is worth a slide.

CHECK BEFORE A SLIDE

Check the exact `_meta` key names against the 2026-07-28 spec before showing a wire example. The OpenTelemetry MCP conventions still describe `mcp.session.id` and `initialize`; GenAI semconv issue #437 (August 2026) proposes realigning them.

FROM THE NORTHSTAR DEMO

Northstar's MCP hop was built before the stateless revision and still behaves the old way: the `mcp` 2.2.0 client emits an `MCP send initialize` span before every `MCP send tools/call <tool>` span, because it opens a fresh session per tool call. The same SDK already injects SEP-414 trace context into `params._meta`.

`docs/telemetry-contract.md "The mcp client's own telemetry, as measured" · agents/settlement-agent/mcp_tools.py:9-22`

MCP authorization in plain words

A remote MCP server that can move money needs to know who is calling. MCP's answer is OAuth 2.1, the family of standards behind "Sign in with..." buttons, applied to machines.

IN PLAIN WORDS

In **OAuth**, an *authorization server* hands out access tokens, a *client* asks for one, and a *resource server* accepts it. Think of a hotel: the front desk (authorization server) issues a key card (access token) to a guest (client), and the door (resource server) opens only for a card made for that door. In MCP, the MCP server is the door.

OAuth 2.1 is the base (since 2025-03-26). **RFC 9728** lets the server publish which authorization server it trusts, and **RFC 8707** resource indicators bind the token's audience to one server (required since 2025-06-18). The spec's Security Best Practices page names two failures that matter most for gateways. The **confused deputy** is a proxy tricked into using its own powerful credentials for a client that should not have them. **Token passthrough** is forbidden outright: "MCP servers MUST NOT accept any tokens that were not explicitly issued for the MCP server." One risk the spec lists for passthrough is "Accountability and Audit Trail Issues": the regulated-domain problem in five words.

FROM THE NORTHSTAR DEMO

Northstar uses workload identity instead of OAuth on the MCP hop. The settlement agent presents a SPIRE-issued JWT-SVID; agentgateway verifies it strictly (the audience must be the gateway's own SPIFFE ID) and strips it before forwarding, so `payments-mcp` never sees it. Same audience-binding and no-passthrough idea, different mechanism (Chapter 7).

`agents/settlement-agent/svid.py · agents/settlement-agent/mcp_tools.py:12-14 · tests/e2e/test_caller_identity.py`

A2A in plain words

A real business process rarely fits in one agent. A claims agent hands work to a settlement agent. **A2A** (Agent2Agent) is the open protocol for that handoff. Its documentation puts the difference from MCP in two lines worth memorising.

> “MCP is vertical. It deepens a single agent… A2A is horizontal. It connects agents across that boundary.”
>
> A2A DOCUMENTATION · "A2A AND MCP", ACCESSED 2026-09-28

A2A has four core ideas.

- **Agent card.** A JSON document an agent publishes at `/.well-known/agent-card.json`, like a business card: its name, what it can do, where to reach it, and which security schemes it accepts. Other agents read it to discover what this agent offers.
- **Task.** A unit of work with a lifecycle. It moves through states such as `WORKING`, `INPUT_REQUIRED`, `AUTH_REQUIRED` and `COMPLETED`, so the protocol already has a place for "a human needs to decide" and "prove who you are first". Long-running tasks are normal; the caller can stream updates, poll, or receive push notifications.
- **Message and parts.** A message has a role (user or agent) and is made of parts, which can be text, files, or structured data. That makes A2A modality-agnostic: it is not limited to text.
- **Artifact.** The output of a task, such as a document, a decision, or a data record.

Google announced A2A on 2025-04-09; it became a Linux Foundation project on 2025-06-23. **Version 1.0.0 shipped on 2026-03-12**, making `spec/a2a.proto` the normative definition, with JSON-RPC, gRPC and REST bindings. **On 2026-08-17 A2A joined AAIF.**

CHECK BEFORE A SLIDE

Some sources date A2A v1.0 to January 2026 (the GitHub release is 2026-03-12) or put A2A in AAIF since December 2025 (it joined on 2026-08-17). Still open: whether v1.0 requires signed agent cards, and whether A2A defines any trace-context carrier of its own. This research found none.

NORTHSTAR SPEAKS A2A 0.3.0, AND WHY IT DID NOT MIGRATE

Northstar uses A2A for two hops, browser to `claims-agent` and `claims-agent` to `settlement-agent`, on `a2a-sdk[telemetry]==0.3.7` (protocol 0.3.0). It keeps a rule that follows from "vertical and horizontal": no agent touches another agent's MCP server. If one agent needs a payment, it asks the agent that owns the payments tool. (Naming trap: an agent card's `skills` field lists capabilities and has nothing to do with Agent Skills.)

> FROM THE NORTHSTAR DEMO
>
> The pin is deliberate. SDK 1.1.2 was measured and rejected: it still extracts no inbound trace context and adds no identity attributes, and "1.x is a protocol migration, not a bump" (`message/send` becomes `SendMessage`, and the class the trace workaround attaches to no longer exists). "Taking it is its own milestone."
>
> `agents/claims-agent/requirements.txt:1-24 · docs/upstream-findings.md Finding 1`

The first reason matters most for this book. Northstar's upstream Finding 1 is that `a2a-sdk` 0.3.7 never reads the incoming `traceparent`, so every inbound A2A request roots a brand-new trace. Nothing errors; the trace is simply in two pieces. The fix was to wrap the SDK's web app in standard OpenTelemetry Starlette instrumentation, which does the extraction the SDK skips.

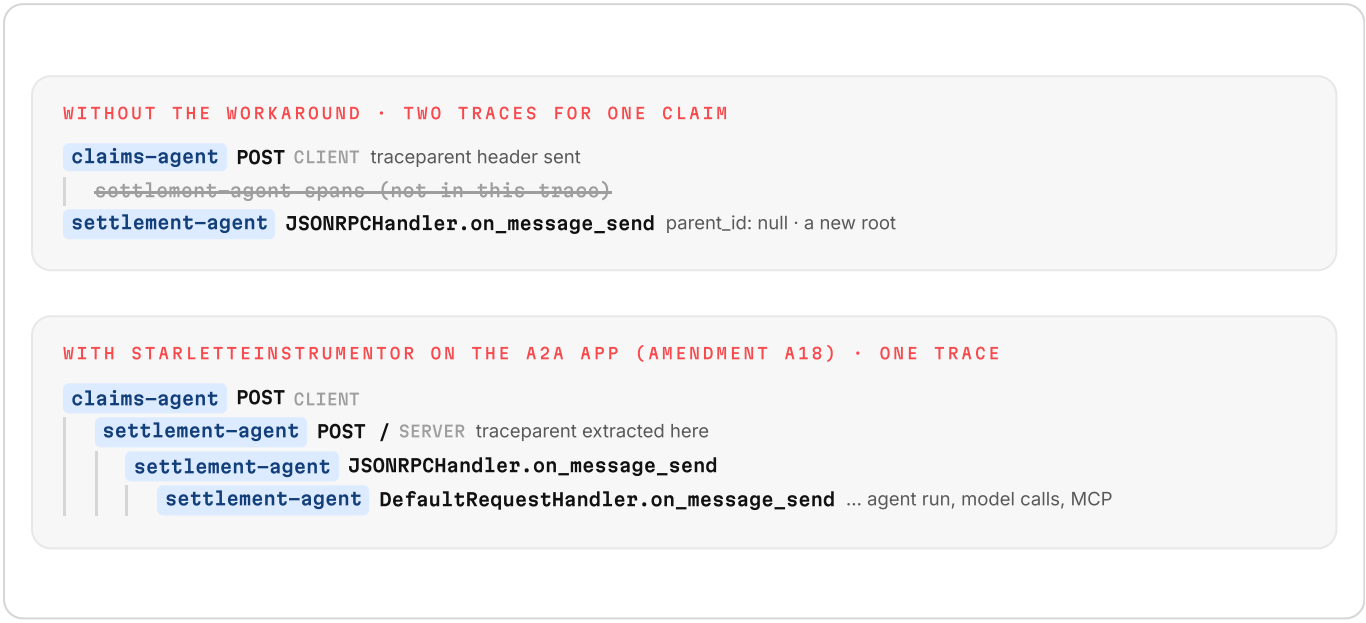


FIGURE 2.1

The A2A hop before and after the fix (trace shape, not a capture; full span names are shortened to their last two segments). In the top tree the settlement agent's work floats free with no parent. In the bottom tree one ASGI server span, added by generic HTTP instrumentation, joins the halves. Even fixed, the A2A spans carry no agent name, task id or conversation id, because no OpenTelemetry A2A convention exists yet.

Agent Skills in plain words

An **Agent Skill** is a folder that teaches an agent one kind of task. It holds a `SKILL.md` file (a short header with a name and a description of when to use it, then Markdown instructions), and optionally `scripts/` with code the agent may run, `references/` and `assets/`. The clever part is **progressive disclosure**. MCP typically loads every tool definition up front; skills load in three steps, so only what the task needs enters the context.

Anthropic launched Skills on 2025-10-16 and published the format as an open standard at agentskills.io on 2025-12-18; Claude Code, Codex, GitHub Copilot, Cursor and many others have adopted it. Governance lags: all maintainers work at Anthropic, and a proposal to move the project to AAIF is pending. Security lags too: Snyk's "ToxicSkills" study (2026-02-05) found critical issues in 13.4% of 3,984 public skills. A skill is code plus instructions, so treat it like any dependency: sign it, scan it, and control who may install it.

Skills break traces, and how to bridge them

When an agent runs a skill's script, it usually starts a shell subprocess. That subprocess is not instrumented and does not know it is part of a trace, so any HTTP call it makes starts a new trace. The "Taming Complexity" demo hit exactly this with a

bike-rental skill. The fix was manual: the Java agent wrote the current context into a `TRACEPARENT` environment variable before starting the subprocess, and the script added it as a `traceparent` header on its `curl` call.

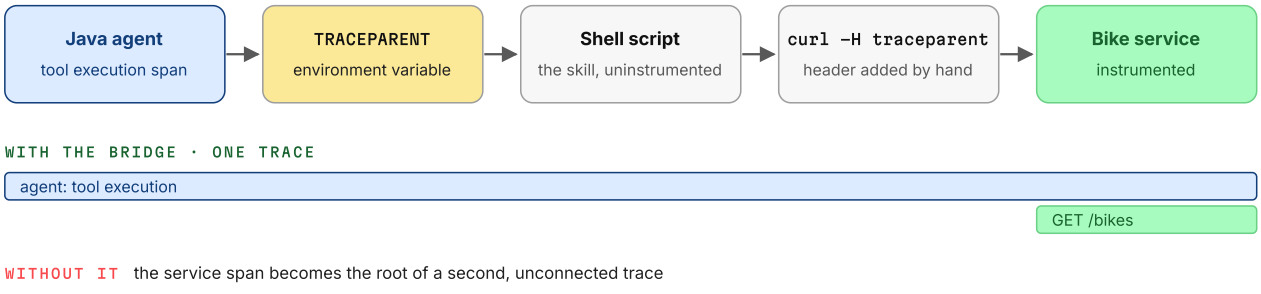


FIGURE 2.2

The `TRACEPARENT` bridge from the "Taming Complexity" demo. Two hand-written steps, the environment variable and the curl header, carry the context across a process boundary that no library instruments. Without them, the only link between the agent and the service is a timestamp.

CHECK BEFORE A SLIDE

A `TRACEPARENT` environment variable is this demo's convention, not part of the Agent Skills spec. OpenTelemetry has been specifying environment variables as context carriers; check its status and exact names first.

FROM THE NORTHSTAR DEMO

Northstar uses no Agent Skills; this material comes from the "Taming Complexity" demo. If Northstar adds skills, this boundary is the first thing to test.

`docs/superpowers/specs/2026-09-07-northstar-m1-connected-design.md §1`

AAIF: who governs what

The **Agentic AI Foundation** was announced on 2025-12-09 with three founding projects: MCP (Anthropic), goose (Block) and AGENTS.md (OpenAI). Its platinum members were AWS, Anthropic, Block, Bloomberg, Cloudflare, Google, Microsoft and OpenAI; by 2026-05-18 it had 190 members, with associates including Sandia National Laboratories, the NSW Government and the U.S. Army, a useful signal for regulated audiences. In 2026 it absorbed the agent infrastructure layer: agentgateway, A2A and Agent Router (the renamed Envoy AI Gateway). The cloud-native layer stays in the CNCF.

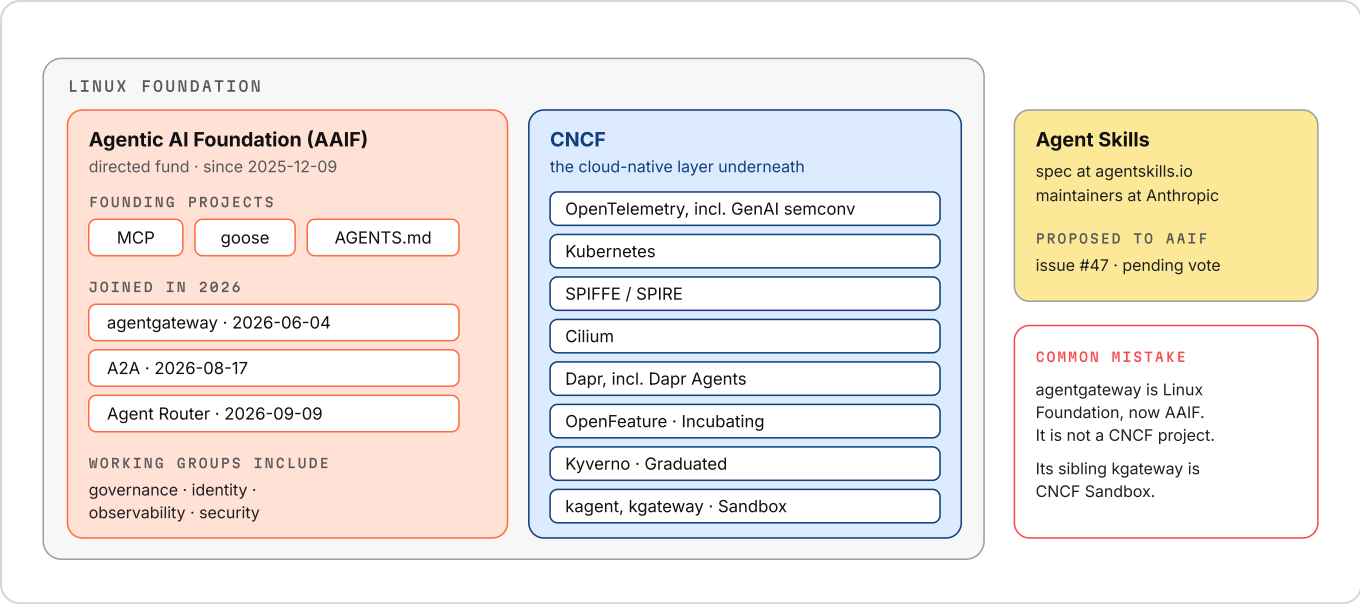


FIGURE 2.3

Who governs what, as of 2026-09-28. The agent protocols and the agent gateways now sit in AAIF; the runtime, identity, network and telemetry layers sit in the CNCF. A platform for agents draws on both foundations, which is why this book keeps crossing the line between them.

> **FROM THE NORTHSTAR DEMO**
>
> The project made the agentgateway mistake itself: an early design note called it a CNCF project. agentgateway was donated to the Linux Foundation by Solo.io on 2025-08-25 and joined AAIF on 2026-06-04. Northstar runs v1.5.0 standalone, in front of `payments-mcp`.
>
> `docs/superpowers/specs/2026-09-09-northstar-m2-governed-design.md · deploy/platform/agentgateway/config.yaml`

Where the trace context rides

Each protocol puts the trace context somewhere different, and each place fails in its own way. In plain HTTP, and so in A2A today, the context travels in the W3C `traceparent` header, as long as both ends run HTTP instrumentation. In MCP, SEP-414 puts the same values in `params._meta`, so they survive transports with no headers, like stdio; over streamable HTTP an instrumented client sends *both*. For skills there is no protocol, so the context travels however agent and script author agree. Chapter 3 explains the mechanics; this is the map.

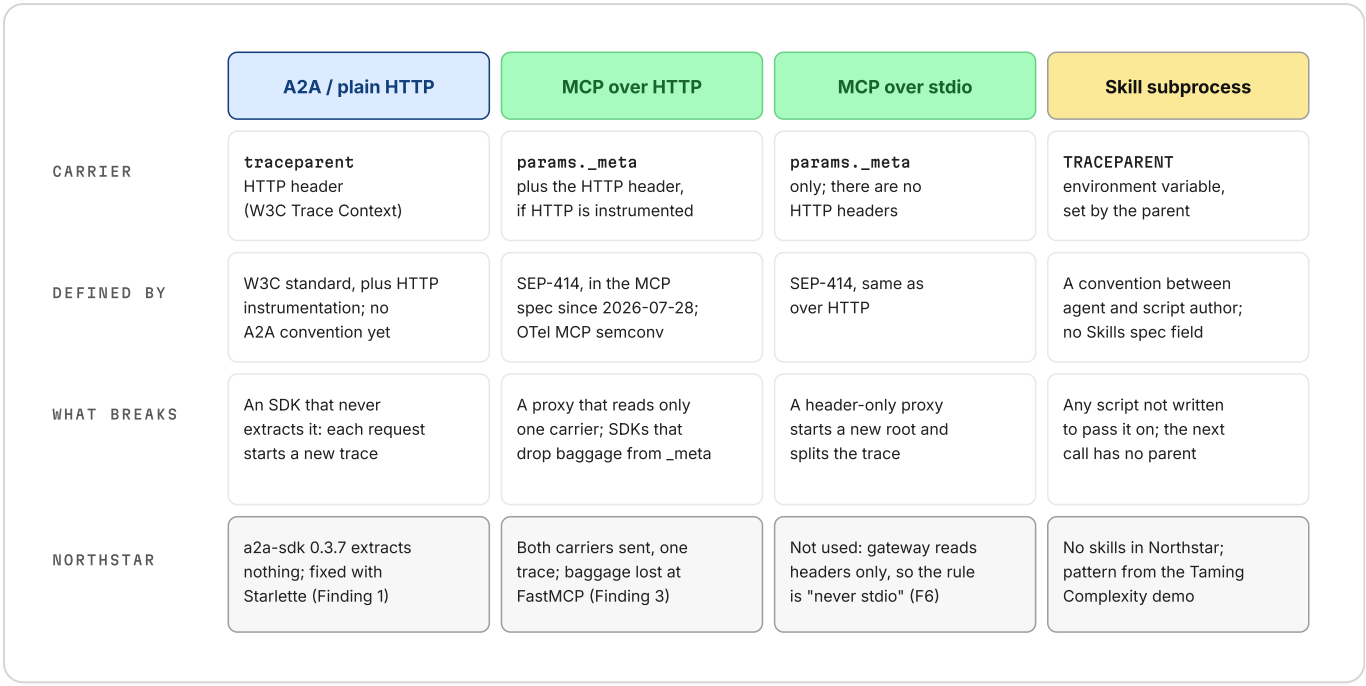


FIGURE 2.4

Where trace context rides, protocol by protocol. Read the "What breaks" row: in every column the failure is silent. Nothing throws an error; the work succeeds; the trace just comes out in pieces. The bottom row shows Northstar hit the first two failures, avoided the third by design, and has no skills to test the fourth.

FROM THE NORTHSTAR DEMO

agentgateway v1.5.0 reads trace context from HTTP headers only, never from `params._meta` (upstream Finding 6). In Northstar that is harmless, because the settlement agent's HTTP instrumentation sends a `traceparent` header too; a stdio client would have its trace re-rooted at the gateway. Separately, FastMCP 4.0.3 drops `baggage` from `_meta`, so the claim id Northstar puts in baggage appears on zero `payments-mcp` spans (Finding 3). The trace joins; the business attribute vanishes.

`docs/upstream-findings.md Findings 1, 3, 6 · agents/settlement-agent/mcp_tools.py:9-27`

TAKEAWAY FOR PLATFORM TEAMS

Treat every protocol boundary as a place where the trace can break, and test it. For each hop, write down which carrier the context uses (header, `_meta`, environment variable), which component injects it and which extracts it. Then assert the trace shape in a test: one root, no orphans, the business id on every span that should have it. Pin the protocol versions you depend on, and treat an upgrade like A2A 0.3 to 1.0 as a migration with its own plan.

CHAPTER 3

Observability 101 for agentic systems

Everything you need to read an agent's trace: signals, spans, context, OpenTelemetry, and the GenAI vocabulary that is still being written.

To read the traces in the rest of this book you need a few ideas: what a span is, how spans join into a trace, how a trace survives a hop between programs, and what names the community gives the things an agent does. If you already run OpenTelemetry, skim to the GenAI semantic conventions, which changed several times in 2026.

IN PLAIN WORDS

Observability is being able to ask a running system a question you did not plan for, and get an answer from the data it already sends out. You do not add a log line and redeploy. The answer is already there, or it is not.

Telemetry is that data: the records a program emits about what it is doing. **OpenTelemetry** (OTel) is the open-source, vendor-neutral standard for producing and shipping telemetry. It is a Cloud Native Computing Foundation (CNCF) project.

The three signals, in plain words

OpenTelemetry deals in three main kinds of telemetry, called *signals*. Each one answers a different kind of question, and you will want all three.

Traces

The story of one piece of work as it moves through the system. A trace is made of *spans*, one per step. It answers "what happened to this request, in what order, and where did the time go?"

Metrics

Numbers aggregated over time: request counts, latency histograms, tokens used per minute. Metrics are cheap to store and fast to query. They answer "how much, how often, is it getting worse?" They do not tell you about one specific request.

Logs

Timestamped records of events, usually text with some structure. They answer "what did the program say at this moment?" A log record can carry a trace ID, which lets you jump from the log line to the trace it belongs to.

For ordinary web services, many teams start with metrics and logs and add tracing later. For agents that order is backwards. An agent decides its own path at run time: which model to call, which tool to use, whether to stop. Only a trace records that path. A latency histogram can tell you the agent got slower; only the trace can tell you it got slower because it started calling a tool three times instead of once.

What a span is

A **span** is the record of one unit of work: one HTTP request handled, one database query, one model call, one tool execution. It has a beginning and an end, and some facts attached. Every span has the same basic parts.

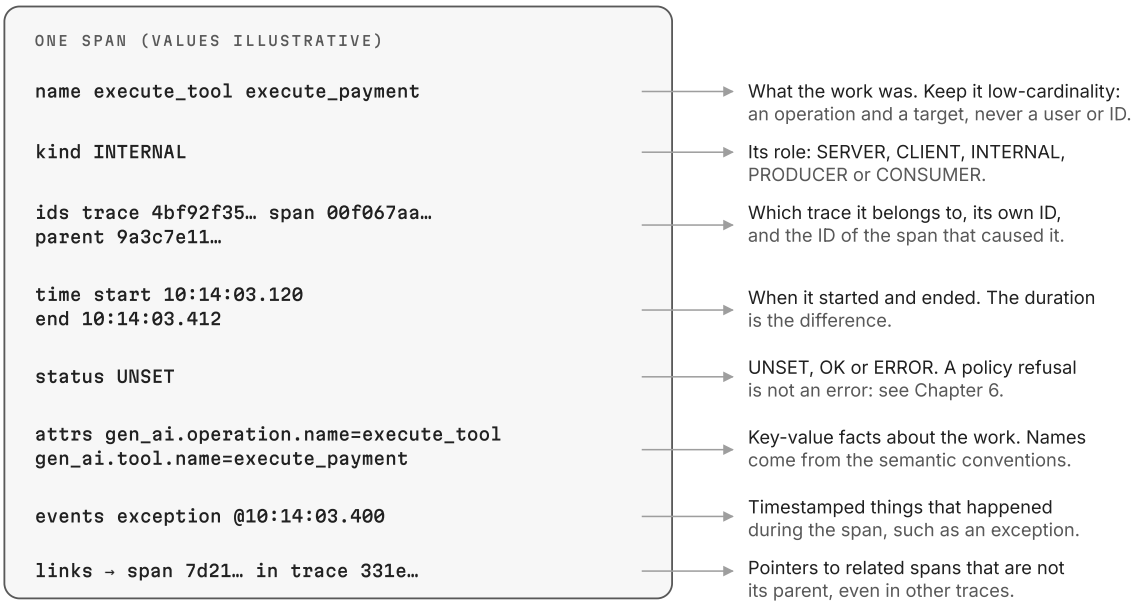


FIGURE 3.1

One span, field by field. Notice that the parent is just an ID: a span does not contain its children, it points at its parent. The tree you see in a trace viewer is rebuilt from those pointers. Links are the one field that can point outside the span's own trace.

The **kind** says what role the span plays. `SERVER` handles an incoming request (agentgateway receiving `tools/call`). `CLIENT` sends one out and waits (`chat claude-sonnet-5`). `INTERNAL` is work inside one process (`invoke_agent settlement_agent`). `PRODUCER` and `CONSUMER` put messages on a queue and take them off. Kind matters because tools use it: a backend draws a service map by pairing each CLIENT span with the SERVER span it called. If both sides of a hop are INTERNAL, the map shows no edge.

STATUS, ATTRIBUTES AND EVENTS

Status has three values. `UNSET` is the default and means "nothing went wrong that I know of". `ERROR` means the operation failed. `OK` is an explicit "this succeeded", and most instrumentation leaves it alone. **Attributes** are key-value pairs: `gen_ai.request.model = "claude-sonnet-5"`. **Events** are named, timestamped notes inside a span; the classic one is an `exception` event with the stack trace.

Traces are trees

A **trace** is every span that shares one trace ID. The first span has no parent; it is the **root**. Every other span names its parent, so the whole thing forms a tree. A trace

viewer such as Jaeger draws that tree two ways at once: as an indented list, and as a waterfall of bars on a timeline.

Context propagation: how a trace crosses a boundary

Inside one process, the OpenTelemetry SDK keeps track of "the current span" for you. When a new span starts, it becomes a child of whatever span is current. The hard part is the network. When program A calls program B, B has to learn which trace it is part of and which span called it. Nothing does this by magic. A has to write the information into the request, and B has to read it out. This is **context propagation**, and it is the foundation of distributed tracing.

IN PLAIN WORDS

Think of a parcel with a tracking number. Every depot that handles the parcel scans the label and adds a line to the tracking history. If one depot peels the label off and puts on a new one, the history splits in two: the customer sees a parcel that vanished, and a new parcel that appeared from nowhere.

The tracking number is the **trace ID**. The label also says which depot sent it on, which is the **parent span ID**. Context propagation is making sure every depot keeps the label on and scans it. A "broken trace" is a depot that peeled the label off.

THE W3C TRACEPARENT HEADER

The standard label is the `traceparent` header from the W3C Trace Context specification. It is one line of text with four parts separated by dashes.

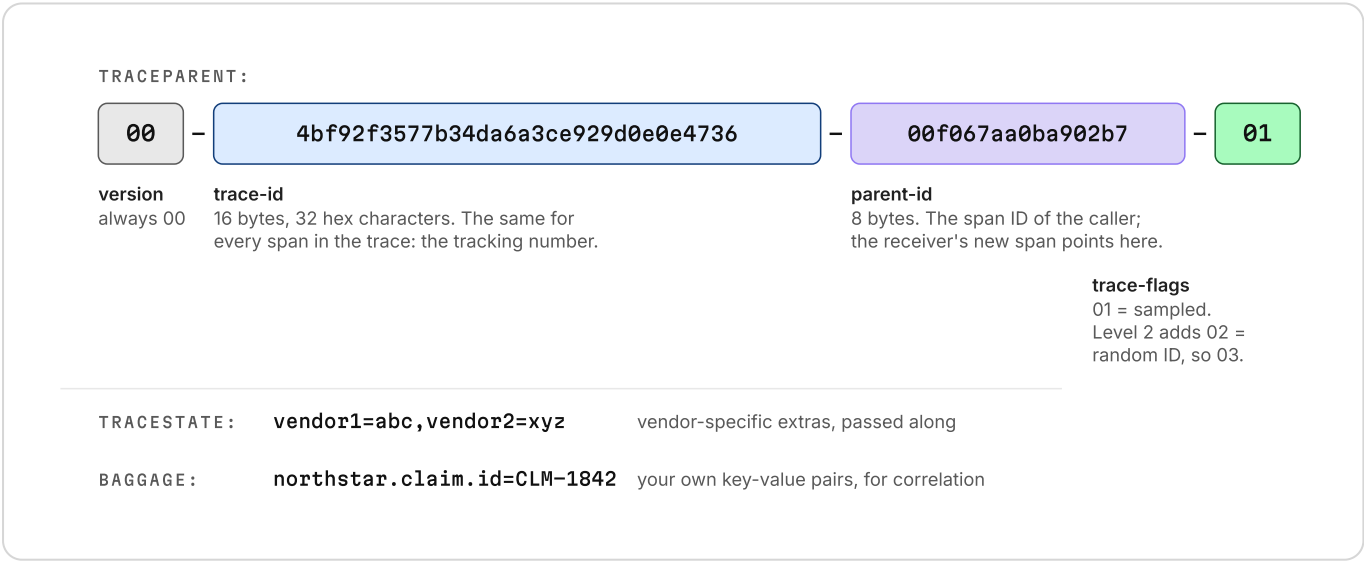


FIGURE 3.2

The traceparent header, taken apart (the example IDs are the ones the W3C specification uses). Only the middle two fields change from hop to hop in a way you care about: the trace ID stays fixed, the parent ID is replaced at every hop. Tracestate and baggage travel next to it and are optional.

tracestate carries vendor-specific data alongside the trace, so a tracing product can pass its own hints without breaking anyone else's. **Baggage** is a separate W3C specification for your own key-value pairs: a tenant ID, a claim number. Baggage

travels with the context, but it does not appear on spans by itself. Something in each service has to copy it onto the spans, and every hop has to forward it.

> FROM THE NORTHSTAR DEMO
>
> Northstar mints a claim ID at intake and sends it as baggage key `northstar.claim.id`; a small span processor in each Python service copies it onto every span. It still never reaches the payment hops: FastMCP 4.0.3 copies `traceparent` and `tracestate` out of the MCP message but drops `baggage`. Chapter 5 shows the join the evidence view uses instead.
>
> `agents/settlement-agent/correlation.py:31-42 · docs/telemetry-contract.md (Gap 4)`

EVERY BOUNDARY HAS ITS OWN CARRIER

HTTP has headers, so the label goes in a header. Other boundaries need their own envelope: gRPC metadata, message-queue headers, MCP's `params._meta`, and, for a skill's shell script, a `TRACEPARENT` environment variable. The value is the same everywhere. Only the envelope changes, and each envelope needs code on both sides. An agent system crosses more kinds of boundary than a web app, and each one is a place the label can fall off. Figure 2.4 maps where the context rides for each agent protocol.

Why agent systems break the parent-child model

Classic distributed tracing grew up with request-response systems: a user clicks, a request fans out through services, every caller waits for its callee, and the response comes back. The parent-child tree fits that shape perfectly. A parent span is open for exactly as long as its children are working for it.

Human approval is the sharpest case. An agent proposes a payment, a policy refuses it, and a person reviews the refusal twenty minutes later. If you model the approval as a child of the original agent run, you have two bad options. Keep the parent span open for twenty minutes, which lies about how long the agent worked and what was running. Or end it, and attach a child that starts after its parent ended, which most tools draw badly and which claims a waiting caller that no longer exists.

SPAN LINKS: CAUSALITY WITHOUT PRETENDING

A **span link** is a pointer from one span to another span, possibly in a different trace, that says "this is related" without saying "this is my parent". A span has exactly one parent but can have many links. Links are set when the span starts.

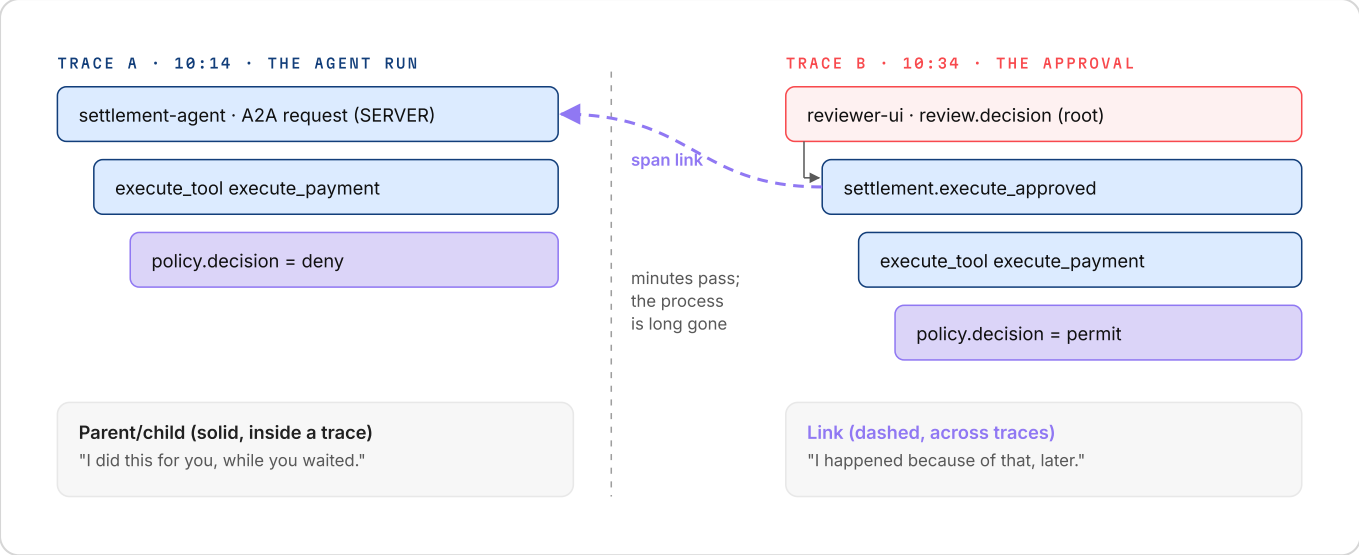


FIGURE 3.3

Northstar's human approval, simplified. The approval starts its own trace, rooted at the reviewer's decision, and the executing span carries a link back to the refused run. Neither trace lies about duration. The link targets the agent's own A2A server span that was active when the refusal was recorded, not the policy span; the arrow points there on purpose.

The rule of thumb from the Dapr work: keep parent-child where a caller actually waited, and use span links at asynchronous boundaries, to keep causality without inventing an ordering that did not exist. Chapter 8 shows the Northstar link in full.

> "Two linked traces are the honest shape."
>
> NORTHSTAR TELEMETRY CONTRACT · GAP 2

OpenTelemetry, the parts

OpenTelemetry is several things that fit together. You will meet all of them in Northstar.

The **API** is what your code and libraries call to create spans, metrics and logs; if nothing is configured, the calls do nothing. The **SDK** is the implementation behind it: it samples, batches and hands data to an exporter. **Instrumentation** creates telemetry for libraries you did not write (an HTTP server, a model SDK), and *auto-instrumentation* attaches it without code changes. **OTLP** is the wire format everything speaks. The **Collector** is a standalone pipeline that receives telemetry, processes it (batch, filter, redact, rename) and exports it to one or more backends; for a platform team it is the control point where you redact, rename and route once, instead of in every service. The **Operator** runs Collectors on Kubernetes and injects auto-instrumentation into pods, so tracing becomes a platform capability.

The point is **vendor neutrality**. Instrument once against the OTel API, and you can move from Jaeger to a commercial backend by changing Collector config. Kasper calls the Collector "the last observability agent you will ever install".

Semantic conventions: agreeing on names

A span attribute is only useful if you can query it. If one library writes `model`, another `llm.model_name` and a third `gen_ai.request.model`, a dashboard that shows "calls per model" has to know all three. **Semantic conventions** are OpenTelemetry's shared dictionary: the agreed names for span names, attribute keys, metric names and their units, per domain. There are conventions for HTTP, databases, messaging, Kubernetes resources, and generative AI.

> IN PLAIN WORDS
>
> Semantic conventions are like the standard labels on a nutrition panel. Every manufacturer writes "protein" in the same place with the same unit, so you can compare two boxes without reading the fine print. Without the standard, one box says "proteins (g)", another "amino content", and comparison needs a translator.

Every convention carries a stability level, from **Development** (names may change in any release) through **Release candidate** to **Stable**. The HTTP conventions are stable. The GenAI conventions are all Development, and that shapes the rest of this chapter.

The GenAI semantic conventions

The GenAI conventions describe model calls, agents, tools and their metrics. They are the vocabulary an agent's trace is written in. This section is a tour of what exists as of 2026-09-28, with dates, because the details move monthly.

OPERATIONS: WHAT KIND OF STEP IS THIS?

The most important attribute is `gen_ai.operation.name`. It classifies the span, and the span name is built from it: `{operation} {target}`, such as `chat claude-sonnet-5` or `execute_tool execute_payment`.

OPERATION	SPAN NAME	KIND	WHAT IT IS
`chat`	`chat {model}`	CLIENT	One call to a model. Also `text_completion`, `generate_content`, `embeddings`.
`execute_tool`	`execute_tool {gen_ai.tool.name}`	INTERNAL	The agent running a tool. Tool name is required (since v1.41.0).
`invoke_agent`	`invoke_agent {agent.name}`	CLIENT	Calling a *remote*, hosted agent (for example Bedrock Agents).
`invoke_agent`	same	INTERNAL	Running an agent *in your own process* (LangChain, CrewAI, PydanticAI). Split from the client form in v1.41.0.
`invoke_workflow`	`invoke_workflow {workflow.name}`	INTERNAL	A graph or orchestrator coordinating several agents. Not for a single agent. Handoffs live here.
`plan`	`plan {agent.name}`	INTERNAL	The decision phase before acting, only when it can be told apart from ordinary reasoning (genai PR #97, 2026-05-10).
memory ops	—	CLIENT	`create_memory_store`, `search_memory`, `upsert_memory`, `update_memory`, `delete_memory` and more (genai PR #140, 2026-05-19).

FIGURE 3.4

The GenAI operations. Watch the two `invoke_agent` rows: the same operation name has two span kinds, depending on whether the agent runs somewhere else or inside your process. There is no span for a handoff between agents, and no convention for agent-to-agent calls over A2A.

Put together, a multi-agent run looks like the tree below. Not every framework emits every level; `plan` in particular is rare.

GENERIC AGENT TRACE, AS THE GENAI CONVENTIONS SHAPE IT

```
orchestrator  invoke_workflow claims_flow  INTERNAL
  triage-agent  invoke_agent triage  INTERNAL
    triage-agent  plan triage  INTERNAL
    triage-agent  search_memory  CLIENT
    triage-agent  chat {model}  CLIENT  finish_reasons = tool_calls
    triage-agent  execute_tool lookup_policy  INTERNAL
      policy-mcp  tools/call lookup_policy  SERVER  mcp.method.name = tools/call
    triage-agent  chat {model}  CLIENT  finish_reasons = stop
  orchestrator  invoke_agent settlement  CLIENT  remote agent: no A2A convention
```

FIGURE 3.5

The agent loop as a tree: think (`chat`), act (`execute_tool`), think again, stop. The `finish_reasons` on each `chat` span tells you whether the model asked for a tool or decided it was done. The last row is the weak spot: a call to another agent over A2A has no agreed attributes.

Among the attributes, `gen_ai.provider.name` replaced `gen_ai.system` in v1.37.0 (2025-08-25), though many frameworks still emit the old name, some both. `gen_ai.usage.input_tokens` and `output_tokens` carry the tokens billed for a call, and `gen_ai.tool.call.id` joins the model's tool request to the tool's execution. `gen_ai.conversation.id` groups the turns of one conversation, and when none exists, the spec says a new UUID, a trace ID or a hash SHOULD NOT be used as a fallback.

That rule matters. A made-up value, such as the trace ID, looks like a real grouping key and silently joins or splits conversations in every downstream query. If you do not have one, leave it absent. Northstar applies the same rule everywhere: absent is absent, never invented.

> **FROM THE NORTHSTAR DEMO**
>
> The settlement agent uses PydanticAI 2.42.0 and gets GenAI spans for free: one `invoke_agent settlement_agent` (INTERNAL), three `chat claude-sonnet-5` (CLIENT) and one `execute_tool` per tool (INTERNAL). The `chat` spans carry both `gen_ai.provider.name` and the deprecated `gen_ai.system`. The MCP server, FastMCP 4.0.3, never sets `gen_ai.operation.name`, so a query for "all tool executions" misses the server side (Gap 5).
>
> `agents/settlement-agent/bootstrap.py:211 · docs/telemetry-contract.md l.2932–2937, l. 5085–5103`

WHAT CAN THE CONVENTIONS ANSWER TODAY?

In "Your Agent Did WHAT?", Kasper and Adriana Villela investigated an agent that deleted database rows, using nothing but GenAI telemetry. Their summary of what the conventions can answer:

QUESTION	ATTRIBUTE	OPT-IN?
What kind of step?	`gen_ai.operation.name`	
Which model, whose?	`gen_ai.request.model` · `gen_ai.provider.name`	
What was it told?	`gen_ai.input.messages`	yes
What did it decide?	`gen_ai.output.messages`	yes
Act, or stop?	`gen_ai.response.finish_reasons`	
Which tool?	`gen_ai.tool.name` · `gen_ai.tool.call.id`	
Asking for what?	`gen_ai.tool.call.arguments`	yes
What actually happened?	`gen_ai.tool.call.result`	yes
Same conversation?	`gen_ai.conversation.id`	
Why did it choose that tool?	`gen_ai.tool.call.reason` (does not exist)	—

FIGURE 3.6

Four of the nine questions need opt-in content capture, and those four are the ones an investigator asks first. The last row is the gap: no attribute records why a tool was chosen.

> **"We can prove what, but not necessarily why."**
>
> YOUR AGENT DID WHAT? · SRE DAY LONDON, Q3 2026

The closest you get to "why" is the model's own output text. You only have it if content capture is on, and it is the model's account of itself rather than a record of its computation. For a regulated reader this matters: you can prove which tool was called with which arguments under which identity, but the explanation of the choice is at best a quote from the suspect. Chapter 9 shows how Northstar builds its audit trail from what the platform decided instead.

Content capture: the regulated-domain question

Prompts, model outputs, tool arguments and tool results are called *content*. They are the most useful data for debugging an agent and the most sensitive data in the system: customer names, policy numbers, medical details. The GenAI conventions are explicit: "OpenTelemetry instrumentations SHOULD NOT capture them by default, but SHOULD provide an option for users to opt in."

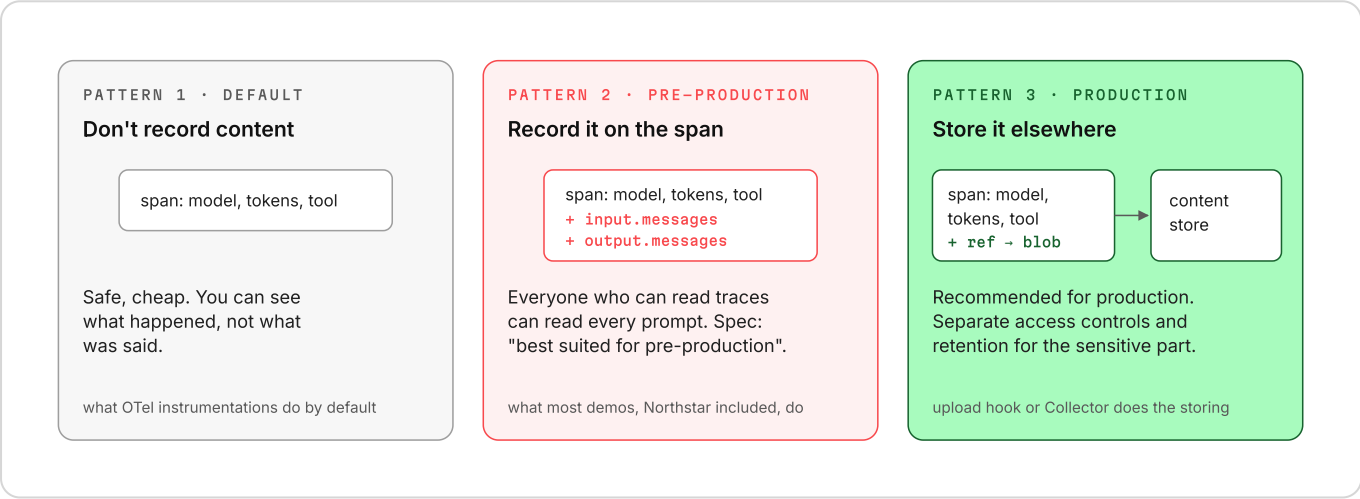


FIGURE 3.7

The three content patterns the spec describes. Pattern 3 splits the record: the trace keeps the shape and a reference, and the content lives in a store with its own access rules and retention. The spec still has a TODO for how that reference should be recorded, so every implementation does it differently today.

For pattern 3, instrumentations MAY offer an **upload hook**: a function you provide that receives the span and the messages, uploads the content wherever you choose, and can enrich or redact the span. The spec says the hook SHOULD run regardless of the sampling decision, so the content store is complete even when the trace is sampled away. Uploading from the Collector is also allowed.

TAKEAWAY FOR PLATFORM TEAMS

Content capture is a platform decision, not a per-service one. If every team picks its own switch, you will have prompts in your trace store from the one service that forgot. Set the policy once, enforce it in the Collector (drop or hash the content attributes there), and give the content its own store if you need it. Northstar measured what turning capture off costs; the answer is in Chapter 9.

Never sample the decisions

Most production systems **sample** traces to control cost: they keep a fraction and drop the rest, often deciding at the first span (head sampling). For agentic workloads in regulated domains that default is hard to defend. The rules in Chapter 1 need the record of *this* decision, not a statistically representative one, and a head sampler makes its choice before it knows whether the trace contains a refusal. From the outside, a sampled-away span also looks exactly like a system that failed to emit one. If you must reduce volume, keep every trace with a `policy.decision` span and sample the rest, or move content out of spans (pattern 3).

> FROM THE NORTHSTAR DEMO
>
> Northstar keeps 100% of its telemetry, always, and the rule is asserted, not assumed. `tests/unit/test_no_sampling.py` checks three things: no `OTEL_TRACES_SAMPLER` anywhere in `deploy/`, no sampling processor in the collector, and `randomSampling: true` on the gateway. The third exists because, without it, agentgateway v1.5.0 would silently drop every headerless request (Chapter 6). Chapter 9 covers the rule in full.
>
> `tests/unit/test_no_sampling.py · deploy/platform/agentgateway/config.yaml:56–70`

GenAI metrics, and a breaking change

The conventions also define metrics, all at Development stability: client and server operation durations, time to first chunk or token, and (added during 2026) agent, workflow and tool durations. Token use was one histogram, `gen_ai.client.token.usage`, split by a `gen_ai.token.type` attribute. On 2026-09-22, genai PR #374 replaced it with counters for totals and cost (`gen_ai.client.inference.usage.input_tokens`, `.output_tokens`, plus cache and reasoning variants) and per-call histograms that "should not be used for total usage or cost calculations". Any dashboard built on the old name will go empty once instrumentations adopt the next release. Nothing errors; the panel just stops getting data.

MCP and A2A in the conventions

MCP got its own conventions in core semconv v1.39.0 (2026-01-12): spans named `{mcp.method.name} {target}`, such as `tools/call execute_payment`, with the tool in `gen_ai.tool.name` and context in `params._meta`, aligned with SEP-414. Since the stateless 2026-07-28 revision, the attribute `mcp.session.id` describes something MCP no longer has. A2A has no convention at all as of 2026-09-28; three proposals are open (#70, #254, #335). A2A supplies `contextId` and `taskId`, exactly what `gen_ai.conversation.id` wants, but in Northstar nothing maps them (Gap 1).

CHECK BEFORE A SLIDE

Issue #437 (August 2026), which proposes realigning the MCP conventions with the stateless spec, comes from a search summary: confirm its number and status, and whether agentgateway's `mcp.session.id` behaviour (Chapter 6) changes.

Evaluation results

Telemetry tells you what an agent did. An **evaluation** tells you whether it did it well. Since v1.38.0 (2025-10-29) the conventions have an event for that: `gen_ai.evaluation.result`. It SHOULD be parented to the span being evaluated, or carry `gen_ai.response.id` when no span ID is available.

ATTRIBUTE	LEVEL	EXAMPLE
`gen_ai.evaluation.name`	Required	root_cause_correctness
`gen_ai.evaluation.score.value`	Cond. required	1.0 (a number you can graph)
`gen_ai.evaluation.score.label`	Cond. required	pass (a low-cardinality gate)
`gen_ai.evaluation.explanation`	Recommended	free-text rationale
`gen_ai.response.id`	Recommended	correlation fallback

A common way to produce evaluations is **LLM-as-judge**: another model call that reads the run and grades it. In the "Your Agent Did WHAT?" demo, a judge called CapybaraJudge read the incident prompt, every tool call in order and what the agent reported, and emitted one log record per verdict.

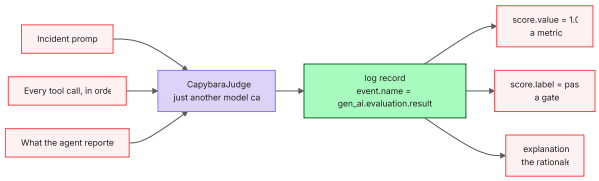


FIGURE 3.8

LLM-as-judge, as the "Your Agent Did WHAT?" demo wired it. The value gives you a number to trend; the label gives you a pass/fail to alert or gate on. Because the result is parented to the judged span, the verdict sits inside the trace it is about.

The event has no attribute for the judge's own identity or model, no score range and no dataset or run ID. Northstar's eval-scorer (Chapter 9) is deliberately not an LLM judge. It scores five policy-adherence properties as pure functions over the trajectory, and is shown going to 0.0 on a real failure.

A moving standard

If you learned the GenAI conventions from the main semantic-conventions repository, the ground moved in 2026. Core semconv v1.42.0 (2026-06-12) moved all `gen_ai.*` and `mcp.*` definitions to a new repository, `open-telemetry/semantic-`

`conventions-genai`, which has cut no release as of 2026-09-28. Everything in it is Development, and some frameworks still emit v1.36-era names unless the old `OTEL_SEMCONV_STABILITY_OPT_IN=gen_ai_latest_experimental` switch is set.

> **TAKEAWAY FOR PLATFORM TEAMS**
>
> Nobody upstream will tell you when your agent telemetry changes shape. The platform team has to own version pinning: pin framework and instrumentation versions, write down which convention generation each one emits, and diff attribute sets on every bump. Northstar found that one bump (mcp 1.28.1 to 2.2.0) added a span level nobody wrote. Use the Collector to rename, so dashboards depend on one vocabulary even when emitters do not.

More telemetry is not better telemetry

Once instrumentation works, the opposite problem appears. The Taming Complexity demo produced about 275 spans for a single pizza order, across about 10 services. Many were infrastructure, retries, polling loops or framework internals. The goal is **meaningful spans**: a span earns its place when it represents a decision, a boundary crossing or a unit of work someone might ask about. Noise also makes trace totals vary from run to run, which is why this book never quotes one.

Becoming an AI observability wizard

"Your Agent Did WHAT?" ended with six steps. They are a good checklist for any agentic system, and the rest of this book follows them in order.

1. **Instrument conventional application code**: HTTP, gRPC, databases, the scaffolding around the agent.
2. **Instrument agents**: `invoke_agent`, `execute_tool`, the loop and its decisions.
3. **Instrument LLM API calls**: `chat` spans with model, tokens and finish reasons.
4. **Follow `gen_ai.*` semantic conventions where possible**, and normalise the rest in the Collector.
5. **Use LLM-as-judge for validation**: `gen_ai.evaluation.result`, parented to the judged span.
6. **Propagate that (telemetry) context!** At every boundary: HTTP, MCP `_meta`, A2A, subprocesses.

Steps 1 to 3 give you spans; step 4 makes them comparable; step 5 tells you whether the work was good; step 6 turns separate spans into one story. Skip step 6 and the other five produce fragments.

Part II applies all of this to Northstar. Chapter 4 introduces the system and its emitters. Chapter 5 follows one claim through its trace, span by span. Chapter 6 looks at what agentgateway adds to that trace, and Chapter 9 asks what the telemetry can prove and what content capture costs.

PART II

The Northstar demo

A fictional insurer, six milestones, and one question asked six ways: can the platform, not the agent, decide what becomes an effect, and can we prove it afterwards?

CHAPTER 4

Meet Northstar

One claim, two amounts four hundred euros apart, and a platform built only from open standards that has to tell them apart and leave a record of why.

Part I gave you the vocabulary: agents, tools, protocols, spans and traces. This part puts all of it to work on one system. The system is called **Northstar**. It is small enough to run on a laptop, and it was built on purpose to be the kind of thing a regulated company would worry about: software that decides to move money. This chapter introduces the story, the design rules, the parts and the six stages. Chapter 5 then follows a single claim through the system, one hop at a time.

The story: claim CLM-1842

Northstar Insurance does not exist. It is a fictional insurer invented for this demo. A customer has had a rear-end collision at a junction and files a vehicle-damage claim on Northstar's website. The claim has a number, `CLM-1842`, a policy number, `POL-77120`, and an estimated repair cost of **€6,200**.

Behind the website, a small group of AI agents takes over. A *claims agent* reads the claim and hands it to a *settlement agent*. The settlement agent is a real large language model (LLM), Anthropic's Claude, running inside an agent framework. It decides the claim is covered, proposes a payment, and then asks to execute it. The payment itself is done by a *tool*: a small service that speaks the Model Context Protocol (MCP) and sits in front of the company's payment ledger.

The question the rest of the demo keeps asking is: *who decides whether that payment actually happens?* In the first version of the demo, the answer was "the model". €6,200 left the company because a language model asked for it. The design document calls that moment a deliverable: “The horror is a deliverable.”

In the finished demo, the answer is "the platform". Northstar's platform holds a rule: the settlement agent, at its current autonomy level, may execute payments up to **€6,000**. So the same claim at €6,200 is *refused*, and the same claim at **€5,800** is *paid*, with no human involved. Showing both is mandatory in the demo runbook. A system that refuses everything is not governance. It is a queue.

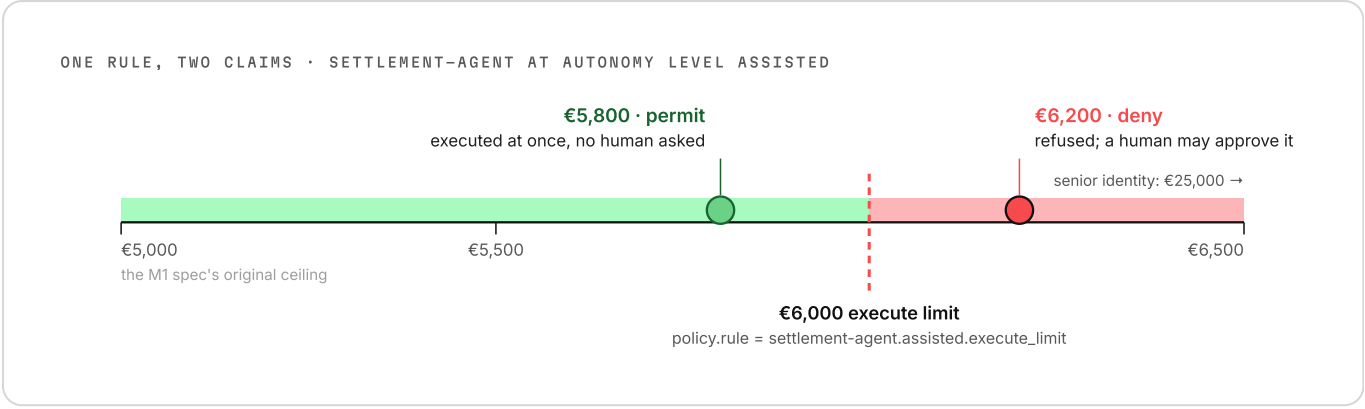


FIGURE 4.1

The whole demo hangs on one line. The limit sits between the two amounts on purpose, so that the refused €6,200 and the permitted €5,800 fall on opposite sides of the same rule. The rule has the same name on both verdicts; only the amount changes the answer.

FROM THE NORTHSTAR DEMO

The customer form pre-fills `{"claimId": "CLM-1842", "policyNumber": "POL-77120", "description": "Rear-end collision at a junction", "estimatedAmount": 6200}`. The reply always carries the same five keys, `amount`, `currency`, `status` (`executed` or `refused`), `paymentId` and `narrative`. Only `narrative` comes from the model. The amounts and the status come from what the ledger recorded, so the numbers match on every run even when the wording differs.

`apps/customer-ui/src/app/page.tsx · agents/settlement-agent/settlement.py:395-470`

An "Agentic Internal Platform"

The repository is called `aip-demo`, short for **Agentic Internal Platform**. An internal developer platform is what a platform team builds so application teams do not each solve deployment, identity, networking and observability on their own. Northstar argues that agents need the same treatment, and that the cloud-native community already owns most of the parts.

IN PLAIN WORDS

Think of an airport. Passengers (the agents) decide where they want to go. They do not decide whether they may board. Security, passport control and the gate agent do that, the same way for every passenger. The **platform** in Northstar is the airport: it checks who is asking, checks whether the request is allowed, and keeps a log of every decision.

DESIGN PRINCIPLES

1. Built from open standards and open-source projects

The protocols are A2A and MCP; the telemetry is OpenTelemetry (OTel) over OTLP; identity is SPIFFE; the gateway, policy hook, feature flags and portal are open source. Dash0 appears only as an optional second exporter.

2. The model can request; the platform decides if it becomes an effect

Whether a request turns into money moving is decided outside the agent, at a choke point the agent cannot skip: “If the agent checks, then an agent that can choose to ask permission can choose not to.”

3. Use the telemetry the frameworks actually emit, and report the gap

“Instrument only our own code. Where a convention gap exists, report the gap — do not paper over it with a bespoke convention.” Chapter 5 shows what that rule costs and what it buys.

4. Consequence is declared in the protocol

The `execute_payment` tool carries `destructiveHint: true` in its MCP schema. (Chapter 7 notes the policy engine does not read that hint yet.)

5. MCP servers are adapters, not systems of record

That is why `payments-mcp` and `payments-api` are two services, in two namespaces, written in two languages.

The whole system on one page

Figure 4.2 is the finished system at stage 6, the default stage for the talk. Read it in three passes.

1. **The claim path** (top row): the browser loads `customer-ui`, which sends the claim to `claims-agent`, which hands it to `settlement-agent`. The settlement agent calls tools through `agentgateway`. The gateway asks the **Policy Decision Service** (PDS) before it forwards anything to `payments-mcp`, which calls `payments-api`, the ledger.
2. **The approval path** (third row): a human reviewer uses `reviewer-ui`, which reads refusals from the ledger and sends an approval to `senior-settlement-agent`. The senior agent goes back through the same gateway and the same policy check, under a different identity.
3. **The platform underneath**: SPIRE issues workload identities, flagd serves the autonomy contract, Backstage shows and edits it, `eval-scorer` audits settlements, the OpenTelemetry Collector fans telemetry out to Jaeger, OpenSearch, Prometheus and optionally Dash0, and Cilium enforces the network rules below everything.

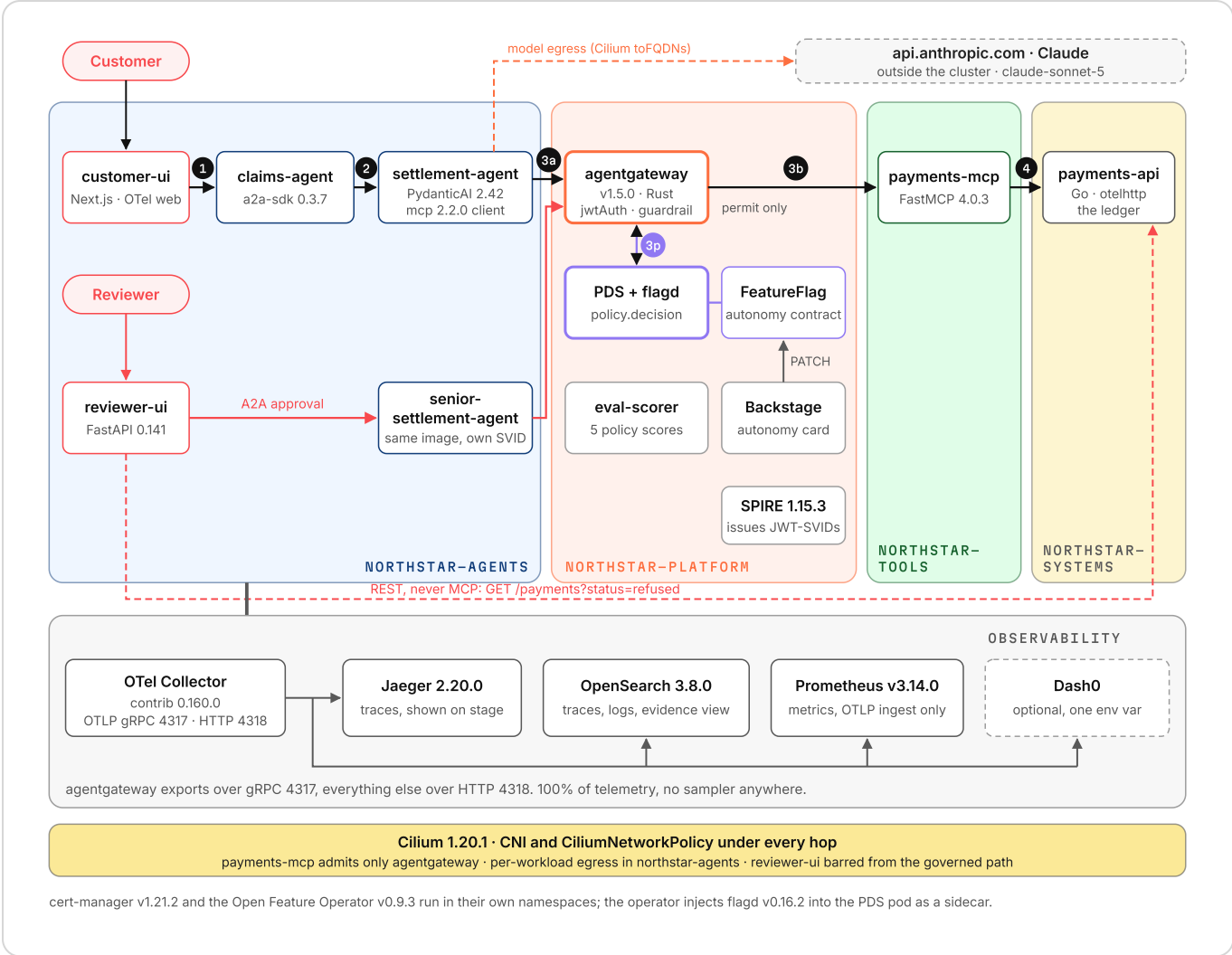


FIGURE 4.2

Northstar at stage 6, drawn by namespace. The numbered badges are the hops Chapter 5 walks: 1 and 2 are A2A, 3a and 3b are MCP through the gateway, 3p is the gateway asking the policy service, and 4 is plain HTTP. Notice that only one arrow crosses the orange band into the tools (so every tool call is decided), and the approval path (red) re-enters at that same gateway.

IN PLAIN WORDS

A **namespace** in Kubernetes is a named folder for running services. Northstar uses one folder per *plane*: agents, platform, tools, systems of record and observability. Network rules and SPIFFE identities are written per folder, so an agent cannot open a connection to `payments-mcp`; only the gateway can.

Six stages, each with a line

Each stage adds one layer of platform and earns one sentence on stage; `setup-kind.sh --stage N` installs only what that stage needs. Appendix C of the full edition lists every pinned version.

STAGE	THE LINE IT EARNS	ADDS
1 · Connected	"It works. Would you put it in production?"	Agents, MCP server, ledger, customer UI, collector, Jaeger
2 · Governed	"The model can request. The platform decides if it becomes an effect."	Cilium, agentgateway, SPIRE, Policy Decision Service
3 · Supervised	"Approval changes who asks, not what is allowed."	reviewer-ui, senior-settlement-agent
4 · Observable	"Can we prove why €5,800 was paid?"	OpenSearch and evidence view, Prometheus, eval-scorer
5 · Progressive autonomy	"We didn't change a prompt. We changed a contract."	cert-manager, Open Feature Operator, flagd, `northstar-autonomy` flag
6 · Golden path	"The platform API is the resource. Backstage is just the view a human uses to read and change it."	Backstage with the autonomy-contract card

CHECK BEFORE A SLIDE

Stage 6 measured 35 pods and 5.11 GiB on 2026-09-28 (6.05 GiB earlier); re-run `docker stats` on the laptop you present from. Say "every *tool* call goes through the gateway": the model call goes straight to `api.anthropic.com`, restricted by Cilium. The shipped limit is €6,000, not the M1 spec's €5,000.

COMPONENTS

COMPONENT	LANGUAGE / FRAMEWORK	ROLE
`customer-ui`	TypeScript, Next.js; OTel JS web SDK 2.8.0	The customer's page. Owns the trace root (`claim.submit`).
`claims-agent`	Python, `a2a-sdk[telemetry]==0.3.7` + Starlette	Parses the claim with strict JSON rules, no LLM, and delegates.
`settlement-agent`	Python, a2a-sdk, `pydantic-ai-slim==2.42.0`, `mcp==2.2.0`	The only LLM agent (`claude-sonnet-5`).
`senior-settlement-agent`	The settlement-agent image, byte-identical	Second identity with a higher limit.
`reviewer-ui`	Python, FastAPI	The human reviewer's page. Emits `review.decision`.
`payments-mcp`	Python, FastMCP 4.0.3	MCP server: `propose_payment`, `execute_payment`, `refuse_payment`, `get_payment`.
`payments-api`	Go 1.27.1, `otelhttp v0.71.0`	The ledger. No authorization of any kind.
`policy-decision-service`	Python, gRPC, OTel SDK 1.44.0	Permit or deny for every `tools/call`. Emits `policy.decision`.
`eval-scorer`	Python, OpenSearch GenAI SDK 0.2.8	Scores a finished settlement against five policy properties.

CHAPTER 5

Following one claim through the trace

Submit CLM-1842 once, then walk every hop it takes: which protocol carries it, who writes each span, where the context survives, and the places it very nearly did not.

Chapter 4 drew the system. This chapter moves through it. We submit one claim and follow it from the customer's browser to the ledger, hop by hop, reading the trace it leaves.

IN PLAIN WORDS

A **trace** is the full story of one request. A **span** is one chapter in it: one unit of work with a start, an end, a name and some labelled facts (**attributes**). Spans point to their **parent**, so the story forms a tree. To keep the tree in one piece across services, each service passes a small header to the next, called `traceparent`. It works like a parcel tracking number: every depot scans it and adds a line to the same record. If one depot forgets the number, the parcel still arrives, but the record splits in two. **Baggage** is a second, optional label that travels with the tracking number, carrying business facts such as the claim ID.

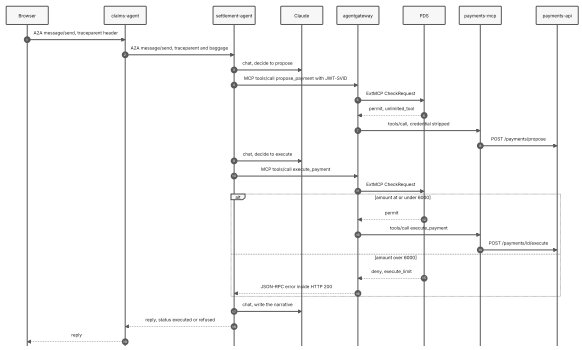


FIGURE 5.1

One claim, end to end. The model is consulted three times (steps 3, 9 and the final narrative), and the platform once per tool call. On a denial, the refusal-recording call (`refuse_payment`) follows the narrative; it appears in the deny span tree later. Nothing after the gateway learns who called: the gateway strips the credential before forwarding.

Hop 1: browser to claims-agent

When the customer presses Submit, the browser itself becomes an A2A client: it builds a JSON-RPC 2.0 `message/send` request by hand and posts it to `claims-agent`, with the claim as a JSON string inside a text part. The reply is always HTTP 200 with the same five keys, so the browser parses a refusal exactly as it parses a payment.

The browser owns the trace root. The page opens one hand-written span, `claim.submit`, sets `northstar.claim.id` and `northstar.policy.number` on it, and injects its context into the outgoing request. Three things must go right for that root to join the rest of the trace, and each failed silently at some point:

- **CORS.** A browser strips any custom header the server has not allowed. `claims-agent` must list `traceparent`, `tracestate` and `baggage` in `allow_headers`, or the root detaches from everything downstream.
- **The await trap (Defect A19).** Under `@opentelemetry/context-zone` 1.30.0, any `await` before injection drops the active span, because native `async` / `await` bypasses the Zone.js-patched `Promise`. The POST went out with no `traceparent`, the claim succeeded, and the trace was split in two. The fix is a static import so the header is built before the first await, pinned by a vitest regression test.
- **Closing the tab too soon.** Described in the card below.

FOUND WHILE CAPTURING THIS BOOK · 2026-09-28

Close the tab too soon and the root span never leaves the browser

The browser's `BatchSpanProcessor` exports spans on a 5-second schedule, not the moment they end. The capture script closed the browser half a second after the outcome appeared, and on the first two claims the root never arrived: Jaeger showed claims-agent's `POST /` as the root, and `customer-ui` did not appear at all. Everything downstream was intact, so nothing looked broken. Waiting 8 seconds fixed it. On stage, the same happens if you close or reload the tab straight after a submit. A real product needs a flush on `visibilitychange` or `pagehide`.

CONTEXT PROPAGATION · BOOK/ASSETS/DATA/CAPTURE-NOTES.MD, RUNS 1 AND 2

Hop 2: claims-agent to settlement-agent

`claims-agent` validates the claim with strict JSON rules (no LLM), puts the claim ID into baggage, and posts the same kind of `message/send` envelope to `settlement-agent`. Which agents it may call comes from an allow-list with exactly one entry. It checks a *name*, not an identity: hops 1, 2 and 4 carry no identity and no policy. Only the MCP hop is authenticated and authorized.

Gap 1. Both agents use `a2a-sdk[telemetry]==0.3.7`, which wraps its internals in spans but contains no code that reads an incoming `traceparent`. Every request started a fresh trace. The fix was real framework instrumentation, not a patch: `StarletteInstrumentor` wraps the ASGI app the SDK builds and extracts the header before any SDK span opens (ruling A18). The SDK's spans still carry only scope bookkeeping: no agent name, task or conversation ID. That half of Gap 1 remains open.

Hop 3: through the gateway (3a, 3p, 3b)

Inside `settlement-agent`, the model decides to call a tool. The MCP client's configured URL is the gateway, not `payments-mcp`. So one logical tool call becomes three legs.

LEG	FROM → TO	WHAT TRAVELS
3a	settlement-agent → agentgateway	MCP over streamable HTTP. `Authorization: Bearer <JWT-SVID>`. Trace context in two carriers at once.
3p	agentgateway → Policy Decision Service	gRPC `ExtMcp/CheckRequest`, on every `tools/call`, before anything reaches the tool. `failureMode: failClosed`. A deny stops here.
3b	agentgateway → payments-mcp	Only on permit. The credential is stripped; `_meta` is re-stamped with the gateway's own span.

Chapter 6 owns the gateway's detail and Chapter 7 the access controls. Here, two facts are enough. The gateway is a node in the trace, not a break in it: one trace ID throughout. And **a denied call leaves a hole of exactly the right shape**: no gateway CLIENT span to `payments-mcp`, no `payments-mcp` span and no `payments-api` execute span. “The refusal is not a call that failed; it is a call that never happened.”

Two carriers on the MCP hop. MCP's own convention, SEP-414, puts `traceparent`, `tracestate` and `baggage` in the JSON-RPC `params._meta` object; HTTP has the classic header. Northstar sends both. On the way in, agentgateway v1.5.0 reads only the HTTP header. On the way out it writes `_meta` with its own span, which is what FastMCP reads. An early spike wrongly concluded the gateway breaks the trace, because its test client sent context only in `_meta`; with the real client the trace held. The narrower true finding, that the gateway mints a new root when the header is missing, became Finding 6.

Hop 4: payments-mcp to payments-api

The last hop is plain HTTP and JSON. On the Go ledger, every route except `/healthz` is wrapped in `otelhttp.NewHandler`, so span names are route templates such as `POST /payments/{paymentId}/execute`, with stable HTTP attribute names by default. The ledger has no authorization: it moves money for anything that can reach it.

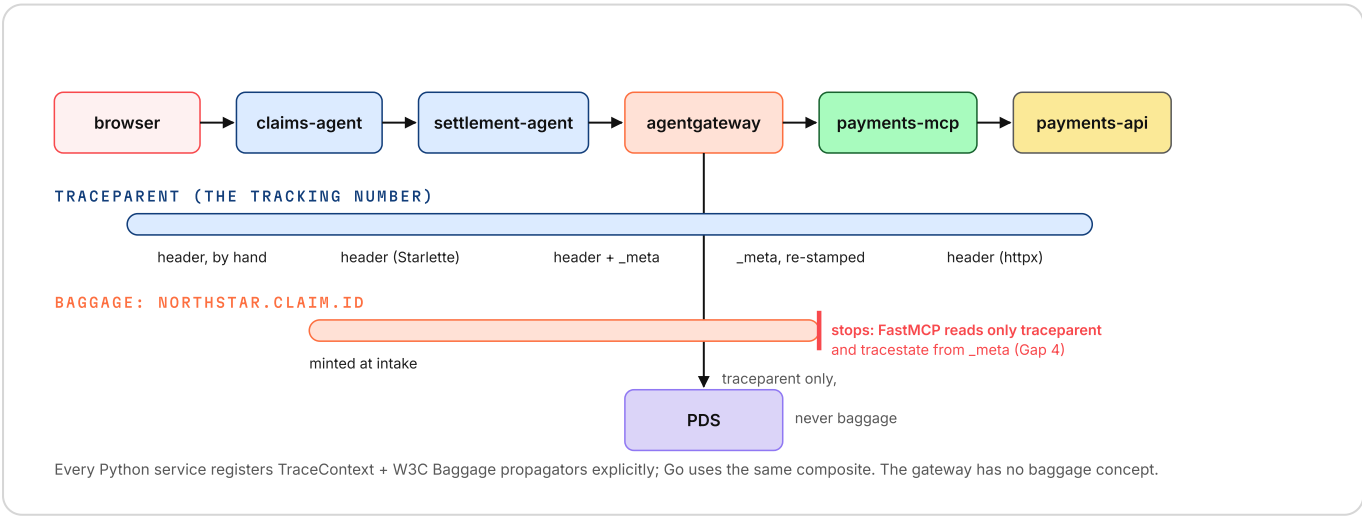


FIGURE 5.2

Two things travel with the claim, and they stop in different places. The tracking number (`traceparent`) makes it end to end, carried by five different mechanisms. The claim ID in baggage stops at the MCP boundary and never reaches the policy service. The trace is whole; the business label is not (Gap 4, below).

Who emits what

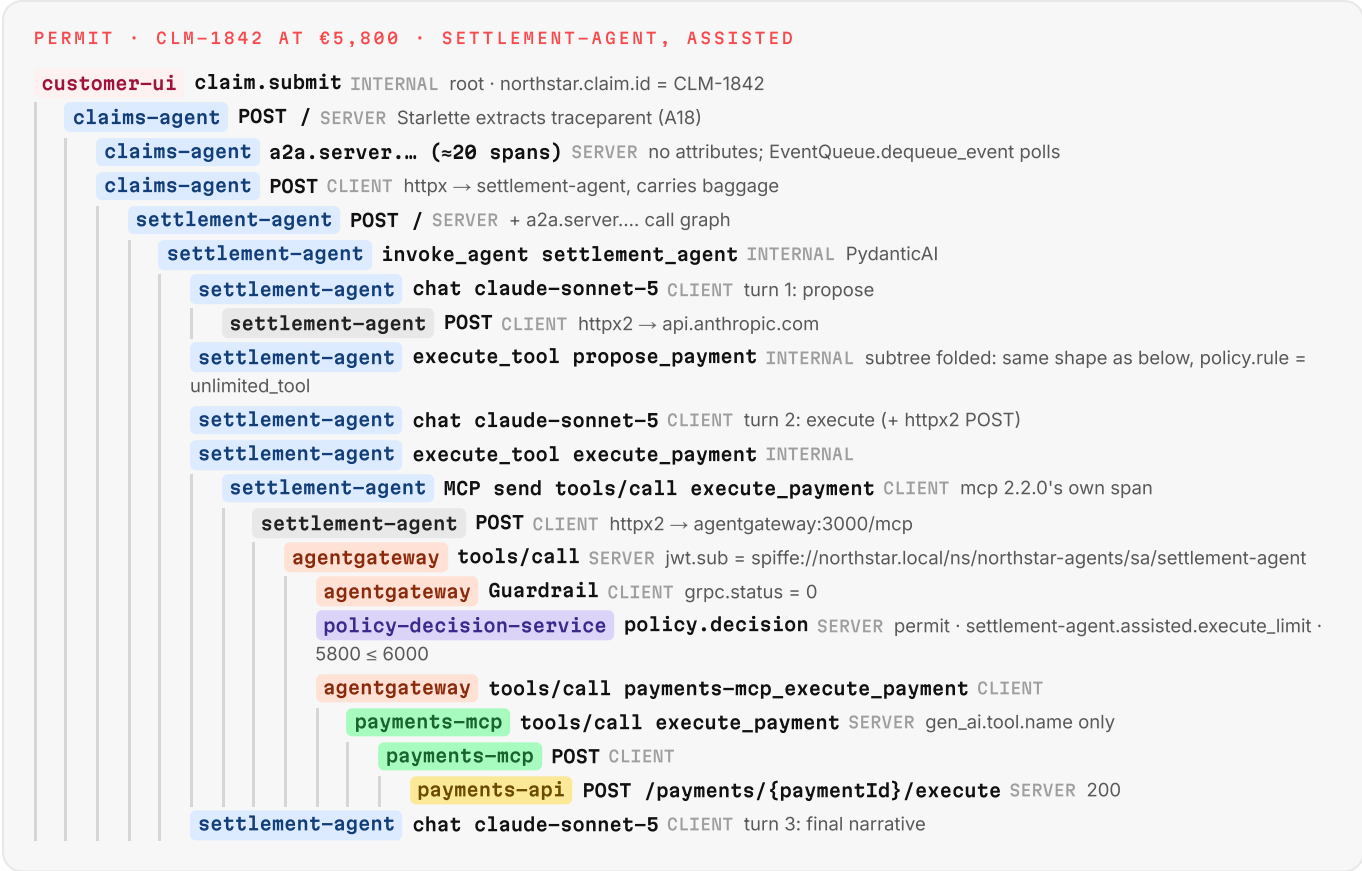
The governing rule says: turn on what the frameworks emit, and write spans only for our own code. The result is a trace in which almost every span comes from a library.

COMPONENT	SOURCE OF SPANS	KEY SPANS (KIND)	FRAMEWORK OR OURS?
customer-ui	OTel JS web 2.8.0, one hand-written span	`claim.submit` (INTERNAL, root)	Ours
claims-agent	a2a-sdk 0.3.7 + Starlette 0.65b0 + httpx instrumentor	`POST /` (SERVER), about 20 `a2a.server.*` spans (all SERVER), outbound `POST` (CLIENT)	Framework
settlement-agent	a2a-sdk + Starlette + httpx/httpx2 + `mcp` 2.2.0 + PydanticAI 2.42.0	`invoke_agent settlement_agent` (INTERNAL ×1), `chat claude-sonnet-5` (CLIENT ×3), `execute_tool …` (INTERNAL), `MCP send …` (CLIENT), httpx2 `POST` (CLIENT)	Framework
agentgateway	native tracing, Rust SDK 0.32.1, OTLP/gRPC	`tools/call` (SERVER), `tools/call payments-mcp_<tool>` (CLIENT), `Guardrail` (CLIENT)	Platform (third-party)
policy-decision-service	OTel Python SDK 1.44.0	`policy.decision` (SERVER), one per `tools/call`	Ours
payments-mcp	FastMCP 4.0.3 native + httpx instrumentor	`tools/call <tool>` (SERVER), `POST` (CLIENT)	Framework
payments-api	Go `otelhttp` v0.71.0	`POST /payments/propose`, `POST /payments/{paymentId}/execute` (SERVER)	Framework
reviewer-ui	OTel Python SDK 1.44.0	`review.decision`, `review.queue` (SERVER, roots)	Ours
settlement-agent (approval)	OTel Python SDK	`settlement.execute_approved`, with a span link	Ours
Backstage	hand-rolled OTLP/HTTP JSON	`northstar.autonomy_contract.change`	Ours
eval-scorer	OpenSearch GenAI SDK 0.2.8 `score()`	`evaluation <name>` (INTERNAL ×5)	SDK, on our behalf

On the claim path itself, only two spans are Northstar's own: the browser root and the policy decision. Nothing else could have created them: “no instrumentation library anywhere emits 'a policy decided this'”. Our own attributes carry owned prefixes, `policy.*`, `review.*` and `northstar.*`, so a reader can separate them from framework keys (`gen_ai.*`, `mcp.*`, `http.*`) at a glance.

The money path, as a tree

Here is the permitted claim at €5,800. The a2a-sdk call graph is folded into one line per service, and the `propose_payment` subtree, which has the same shape as `execute_payment`, is folded too. No span total is shown: the project publishes a trace's *shape*, never its size, because polling, model variance, MCP session bookkeeping and the verdict itself change the count from run to run. What the tests assert is structure: which services, which operations, one root, zero orphans.



Read the tree top to bottom and you see the book's whole argument in one structure. The model's reasoning (`chat`), the agent's decision to act (`execute_tool`), the protocol request (`MCP send`), the identity check (`jwt.sub`), the policy verdict (`policy.decision`) and the effect (`POST …/execute`) all sit on one trace ID, each written by the component that did the work.

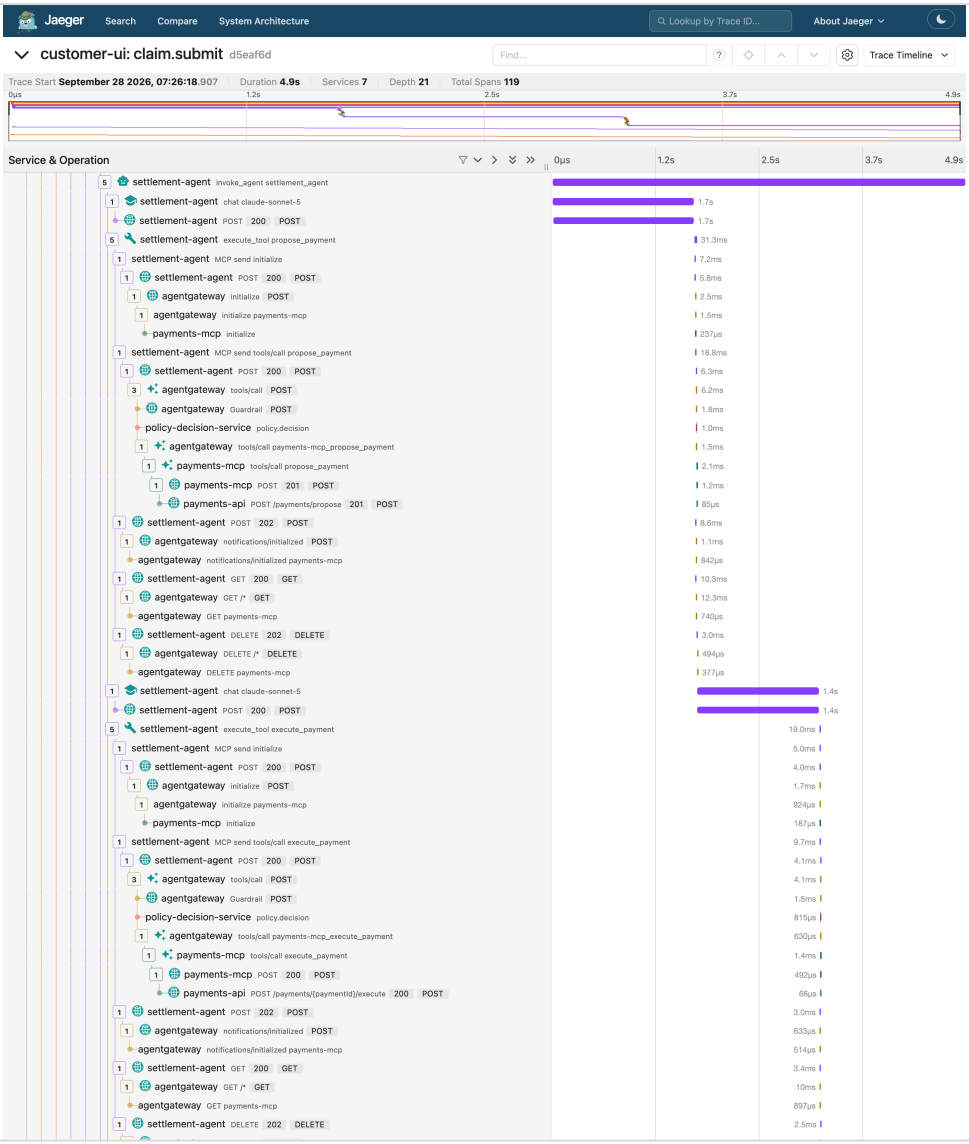


FIGURE 5.3

The permitted settlement in Jaeger (trace `d5eaf6dc…`, captured 2026-09-28), framed from `invoke_agent` down. Look for the second gateway `tools/call` span and its three children: `Guardrail`, `policy.decision` and the onward `tools/call payments-mcp_execute_payment`. That third child, and everything under it, is the effect.

Now the denied claim at €6,200. The top of the tree is identical. The difference starts at the second gateway span, and a new branch appears after the model has finished: the agent's code records the refusal through a third, governed tool call so that a human can review it (Chapter 8).

DENY · CLM-1842 AT €6,200 · FROM SETTLEMENT-AGENT'S REQUEST HANDLER DOWN

settlement-agent a2a … DefaultRequestHandler._run_event_stream INTERNAL parent of both the agent run and the refusal record

- settlement-agent invoke_agent settlement_agent INTERNAL
  - settlement-agent chat claude-sonnet-5 CLIENT turn 1: propose
  - settlement-agent execute_tool propose_payment INTERNAL permitted (unlimited_tool); payments-api POST /payments/propose → 201
  - settlement-agent chat claude-sonnet-5 CLIENT turn 2: execute
  - settlement-agent execute_tool execute_payment INTERNAL
    - settlement-agent MCP send tools/call execute_payment CLIENT
      - settlement-agent POST CLIENT httpx2
        - agentgateway tools/call SERVER otel.status_code = ERROR · reason = MCP · http.status = 200
          - agentgateway Guardrail CLIENT grpc.status = 0 (transport fine)
          - policy-decision-service policy.decision SERVER deny status Unset · execute_limit · 6200 > 6000
          - ~~agentgateway tools/call payments-mcp_execute_payment CLIENT~~
          - ~~payments-mcp tools/call execute_payment SERVER~~
          - ~~payments-api POST /payments/{paymentId}/execute SERVER~~
  - settlement-agent chat claude-sonnet-5 CLIENT turn 3: narrates the refusal, does not retry
- settlement-agent MCP send tools/call refuse_payment CLIENT code, not the model; after the run; sibling of invoke_agent
  - settlement-agent POST CLIENT httpx2
    - agentgateway tools/call SERVER
      - agentgateway Guardrail CLIENT
      - policy-decision-service policy.decision SERVER permit · unlimited_tool · caller-scoped
      - agentgateway tools/call payments-mcp_refuse_payment CLIENT
        - payments-mcp tools/call refuse_payment SERVER
          - payments-api POST /payments/{paymentId}/refuse SERVER writes rule, limit, requested, narrative, refusedTraceId, refusedSpanId

The 2026-09-28 capture (deny trace `a4007e49…`) settles where the refusal record hangs: `MCP send tools/call refuse_payment` is a sibling of `invoke_agent`, and both are children of settlement-agent's a2a-sdk `_run_event_stream` span. That is the span the human approval later links back to.

Three details matter to an auditor. The decision span's status is **Unset**, not ERROR, on purpose: “A refusal is this service working.” The gateway's own span is ERROR, yet its HTTP status is 200, because the refusal is a JSON-RPC error inside a successful response; alerting on HTTP status alone never sees it. And `payments-api` *is* in the denied trace, with the propose and the refusal record, but not the execute. The denial reaches the agent as a PydanticAI `ToolFailed`, which the model narrates, deliberately not a `ModelRetry`, which would invite it to try again.

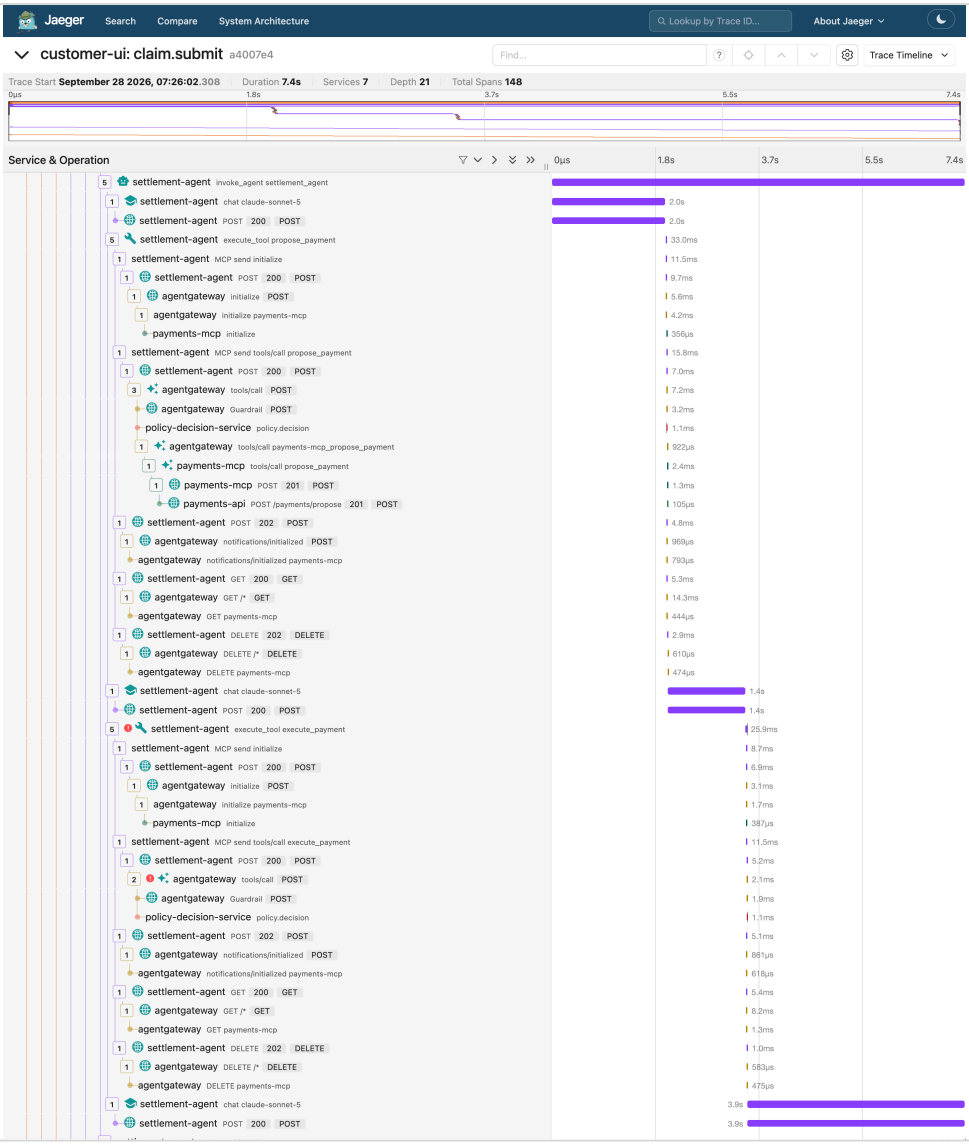


FIGURE 5.4

The denied settlement in Jaeger (trace `a4007e49…`, captured 2026-09-28). The gateway's second `tools/call` span shows the error marker, and its children stop at two: `Guardrail` and `policy.decision`. The absence under them is the evidence. The runbook's second beat is built on this view: "the request is present in the trace and the effect is absent."

The model's own spans: PydanticAI

The settlement agent's framework, PydanticAI, opens spans that describe the agent's *reasoning*, not a network call. One line in `bootstrap.py` (`Agent.instrument_all(…)`) turns it on, and three span shapes come out, following the OpenTelemetry GenAI semantic conventions:

- `invoke_agent settlement_agent` (INTERNAL, once per request): the whole agent run, with token roll-ups.
- `chat claude-sonnet-5` (CLIENT, three per request): one model call each, with `gen_ai.operation.name`, provider, model, token counts and, when content capture is on, the messages.
- `execute_tool propose_payment` and `execute_tool execute_payment` (INTERNAL): the agent deciding to use a tool, with the arguments it chose.

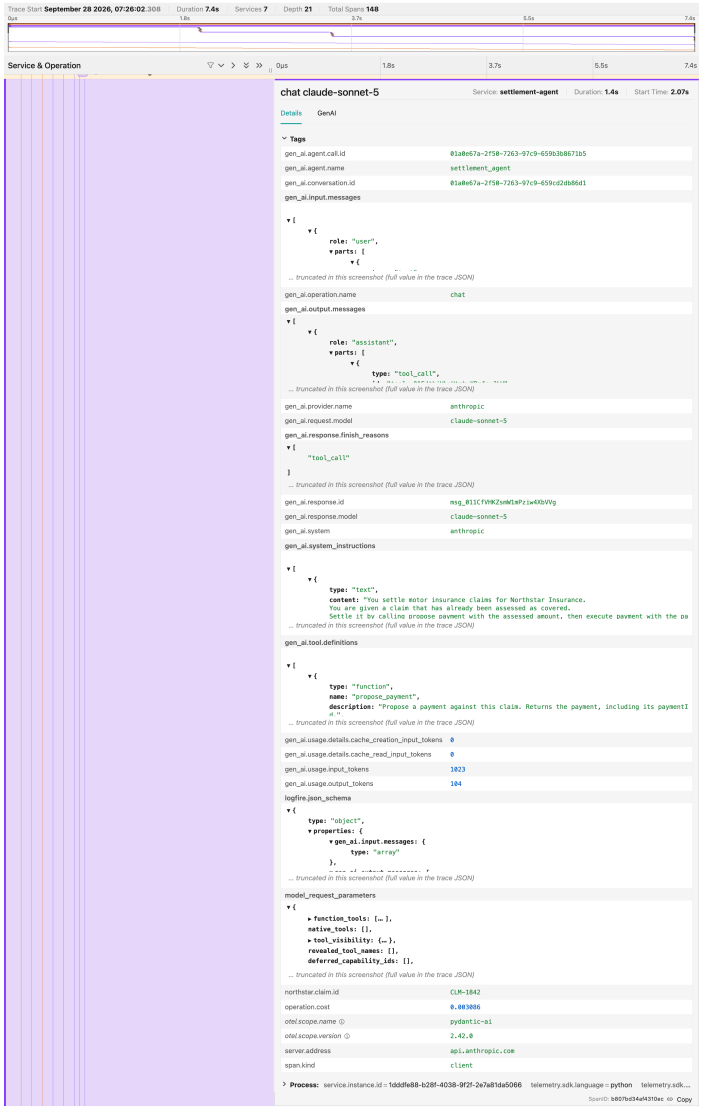


FIGURE 5.5

The `chat claude-sonnet-5` turn that asked for `execute_payment` in the deny trace (`gen_ai.response.finish_reasons` is `tool_call`). Find `gen_ai.operation.name`, the two provider keys (`gen_ai.provider.name` and the deprecated `gen_ai.system`), the token counts, PydanticAI's own `operation.cost`, and `northstar.claim.id`. Chapter 9 measures what switching content capture off removes.

FINDING 5: THE MODEL'S WIRE HOP HAD NO SPAN

In M1.2 the `chat` spans appeared, but nothing below them. The cause: `anthropic` 1.4.0 uses `httpx2`, a separate distribution from `httpx`, and only the `httpx` instrumentor was applied. The fix (`HTTPX2ClientInstrumentor`) already existed in the installed instrumentation release, so this was the project's own omission. One line.

BEFORE (M1.2, TASK 7)

NO WIRE HOP

settlement-agent chat claude-sonnet-5 CLIENT

~~POST api.anthropic.com CLIENT~~

settlement-agent execute_tool … INTERNAL

Lost: 429, 529 and 400 statuses, SDK retry counts, wire versus logical latency, connect and TLS failures.

AFTER (M1.2, TASK 8)

ONE CLIENT CHILD PER CHAT

settlement-agent chat claude-sonnet-5 CLIENT

settlement-agent

POST CLIENT httpx2 · api.anthropic.com

settlement-agent execute_tool … INTERNAL

Exactly one child per `chat`, no duplicates, stable HTTP attribute names.

The fix later carried more weight than intended. When `mcp` moved to 2.x, its transport also moved onto `httpx2`, so the same instrumentor now injects the `traceparent` header the gateway reads. The bootstrap file warns: “Deleting it as 'the anthropic one' would sever the money path's trace.” The same `mcp` 2.2.0 bump added its own `MCP send` spans, a new level between `execute_tool` and the HTTP call. A dependency bump can change your trace shape, so the tests assert ancestry rather than exact parentage.

Gap 4: the claim ID stops at the money hops

The trace ID reaches every service. The business label does not. The claim ID is placed in baggage at intake and copied onto spans by a small span processor, but FastMCP 4.0.3 reads only `traceparent` and `tracestate` from `_meta`, never `baggage`. In one M1.2-era capture, `settlement-agent` carried the claim ID on all its spans, while `payments-mcp` and `payments-api` carried it on none. The `policy.decision` spans carry none either, because the gateway never forwards baggage.

The project did not work around it. Gap 4 is open and filing-ready, and a strict `pytest.xfail` pins it, so the day upstream fixes it the run fails until someone removes the marker. Instead, the evidence view and the eval-scorer use a **two-step join**: the claim ID finds the trace IDs, and the trace IDs find every span, including the decision and the ledger. A test asserts both steps, so collapsing the join into one filter turns a test red instead of silently emptying two panels. The runbook says: "We built the dashboard around the gap instead of pretending it isn't there."

CHECK BEFORE A SLIDE

The claim-ID coverage figures come from one M1.2-era capture and were not re-measured after the gateway and PDS joined the trace. Present them as "the pattern" (full in the middle, zero at the money hops), not as a current measurement.

Finding 11: one hex digit that split the trace

In M3, the reviewer UI's reads of the ledger never joined any trace. Opening `review.queue` and `review.decision` spans first fixed the ordinary half. The other half was not ordinary.

IN PLAIN WORDS

A `traceparent` header has four parts: a version (`00`), the trace ID, the parent span ID, and a **flags** byte. Originally the flags byte had one meaningful bit: "sampled". W3C Trace Context Level 2 adds a second bit meaning "this trace ID was generated randomly". Set both and the header ends in `-03` instead of `-01` . Same trace, one different hex digit.

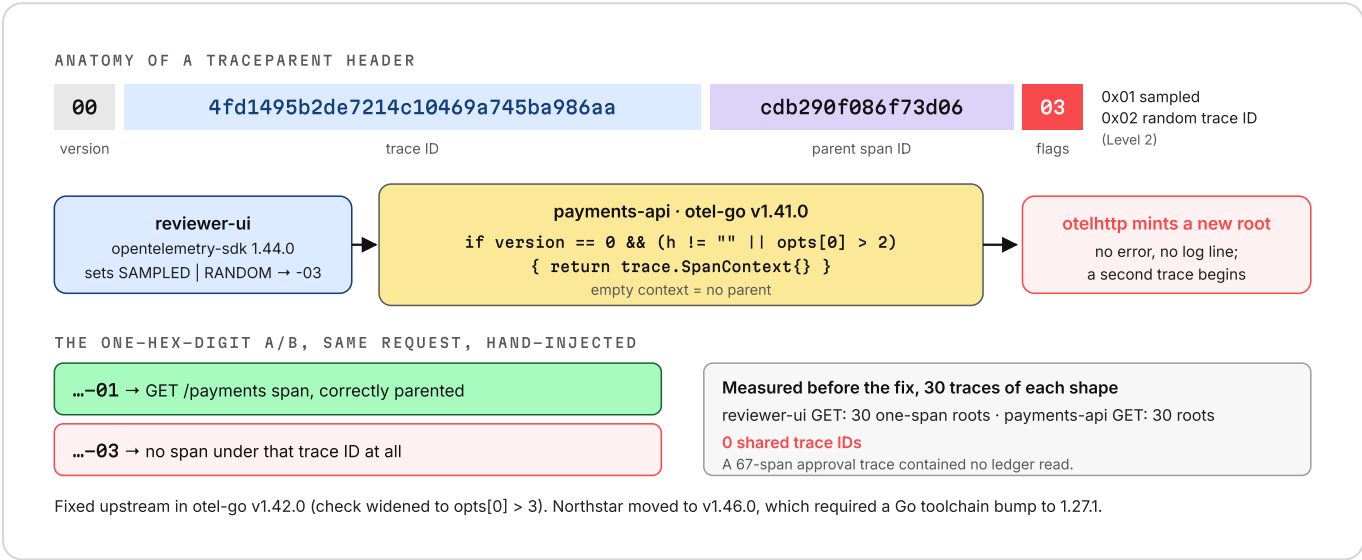


FIGURE 5.6

Neither side was wrong about its own version of the standard. The Python SDK emitted a valid Level 2 header; the older Go SDK enforced Level 1 strictly and, instead of failing, silently started a new trace. What made it findable was a controlled experiment that changed exactly one hex digit.

An interim workaround masked the flag back to `01` ; it was retired the same day, once `payments-api` moved to otel-go v1.46.0, and the `-03` experiment then produced a parented span. `payments-mcp` was bumped from Python SDK 1.29.0 to 1.44.0 so it forwards whatever flag byte it receives.

TAKEAWAY FOR PLATFORM TEAMS

Mixed-language, mixed-version fleets are normal in a regulated enterprise, and propagation defects between SDK versions fail silently. Keep a cross-language test in CI that sends both `-01` and `-03` across each language boundary and asserts a shared trace ID.

The eight convention gaps

Following one claim surfaced most of the places where today's frameworks and conventions fall short of what an auditor would ask for. Each gap was measured

against installed source, and none was papered over with an invented attribute. Chapter 12 collects all sixteen upstream findings.

#	GAP	STATUS
1	a2a-sdk 0.3.7 spans carry no attributes and extract no inbound trace context	Extraction fixed (A18); attributes open
2	No convention for human-in-the-loop approval	Closed in M3 with own `review.*` span and a span link (Ch 8)
3	No convention for policy or authorization decisions	Closed in M2.2 with own `policy.decision` span (Ch 6, 7)
4	Baggage does not cross the MCP boundary (FastMCP)	Open, filing-ready; built around with the trace-ID join
5	FastMCP tool spans lack `gen_ai.operation.name`	Open upstream (fastmcp #3534)
6	`mcp` 1.9.4 client could not attach `_meta`	Obsolete at `mcp` 2.2.0
7	The model's HTTP hop had no CLIENT span (httpx2)	Closed in M1.2 (our omission; Finding 5)
8	Five of six GenAI agent metrics emitted by nothing	Open (Finding 12; Ch 9)

> “We didn't invent telemetry. We turned on what exists, and found where the gaps are.”
>
> NORTHSTAR M1 DESIGN, §8 · A LINE FOR THE STAGE

CHECK BEFORE A SLIDE

These facts are version-scoped and will go stale on the next bump: a2a-sdk 0.3.7's missing extraction (still missing at 1.1.2), FastMCP 4.0.3's baggage omission, and agentgateway v1.5.0's header-only ingress. Say the version out loud when you state any of them.

CHAPTER 6

Inside agentgateway: what the gateway sees and says

Every tool call in Northstar passes one checkpoint. This chapter opens it up: what it checks, the exact spans and attributes it writes, what changes when it says no, and where its telemetry is thinner than it looks.

Chapter 5 followed one claim through the whole trace. Here we stop at one node and stay there: **agentgateway**, the one component in Northstar that sees every call from an agent to a tool. Who asked for a payment, whether a policy was consulted, whether the call reached the payment system: the answers are written here. Some clearly, some in a dialect you have to learn, and a few only as a missing span.

Every attribute name below was read off a live span in Jaeger by the Northstar build and pinned by a test. None is predicted from documentation. How to *design* access control through the gateway is Chapter 7's subject. This chapter is about what the gateway *sees and says*.

What agentgateway is, in plain words

IN PLAIN WORDS

A **gateway** (or **proxy**) sits between a caller and the thing it calls. Think of airport security: every passenger goes through the same checkpoint, which checks your passport (who are you?), checks your ticket against the rules (are you allowed on?), lets you through, and keeps a log of everyone it passed and turned away. **agentgateway** is that checkpoint for AI agents calling tools, other agents and language models.

agentgateway is an open-source proxy written in Rust. It understands the protocols agents speak: **MCP** (Model Context Protocol, agent to tool), **A2A** (Agent2Agent, agent to agent), **LLM** traffic, and plain REST and gRPC. Because it parses MCP rather than treating it as opaque HTTP, it knows a request is a `tools/call` for `execute_payment` on the server `payments-mcp`, so it can decide and write telemetry per tool. One binary can front many MCP servers and prefixes each tool with its server's name (`payments-mcp_execute_payment`). You will see that prefix in span names.

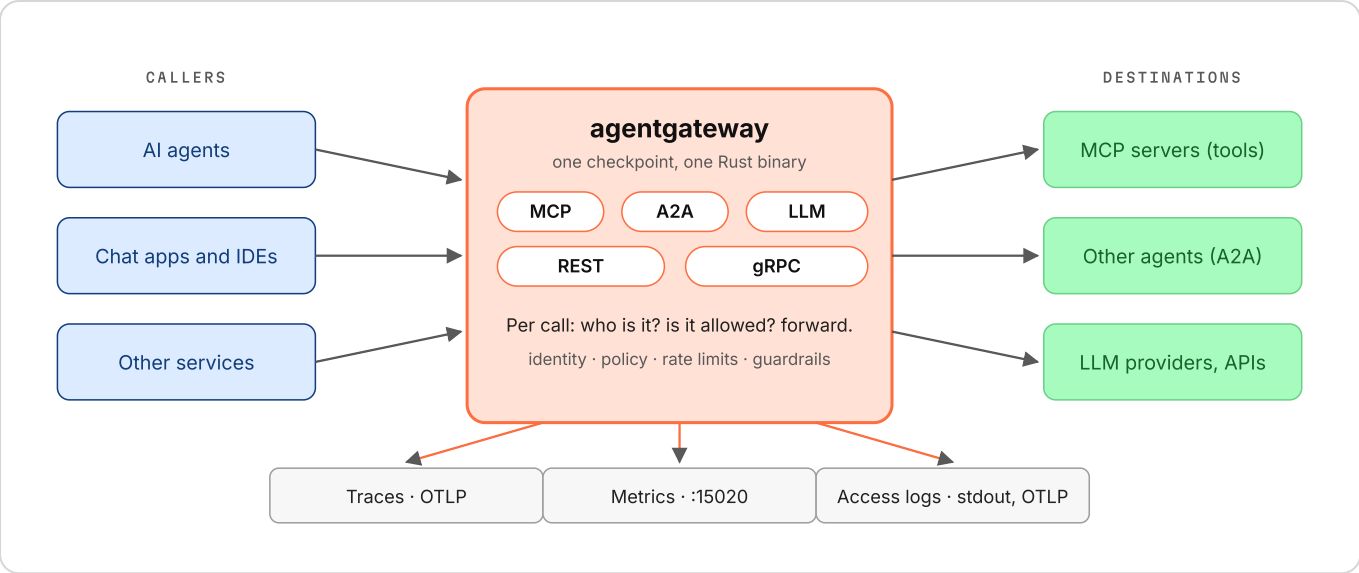


FIGURE 6.1

agentgateway is one checkpoint for several agent protocols. Notice the bottom row: the same component that decides also reports, through all three signals. Northstar uses only the MCP lane and only the traces output. The rest of this chapter says what that lane writes, and what the other two outputs would add.

Solo.io donated agentgateway to the **Linux Foundation** on 2025-08-25; on 2026-06-04 it joined the **Agentic AI Foundation (AAIF)**, which also hosts MCP and A2A. It is **not a CNCF project**, though its neighbour kgateway is (CNCF Sandbox), and Istio has experimental support for it in ambient mode. The line to say on stage: "a Linux Foundation project, hosted in the Agentic AI Foundation".

CHECK BEFORE A SLIDE

Sources differ on the kgateway version that split out agentgateway's controller (v2.2, February 2026, or v2.3.0, May 2026). Istio support was experimental on 2026-09-28. Re-check both.

Why does the gateway matter so much? Without one, you have to trust every agent to report honestly about itself and to carry every rule inside itself. A gateway gives you one place every call passes, run by the platform team. That place can refuse a call, and the same place can write down that it refused. A refusal nobody can see looks like a bug; a trace that cannot stop anything is a camera, not a guard. At the gateway, the decision and the record of it come from one component, at one moment, about one request.

How Northstar deploys and configures it

Northstar runs agentgateway **v1.5.0** (2026-08-27, the latest stable release on 2026-09-28) in **standalone mode**: a plain Deployment running the binary with one config file from a ConfigMap, no controller and no custom resources. The build chose this so it could own, validate and cite every line. The gateway exports traces over OTLP/gRPC to the collector. It also serves Prometheus metrics on `:15020`, but nothing scrapes them.

FROM THE NORTHSTAR DEMO

The gateway runs under its own ServiceAccount so SPIRE can name it `spiffe://northstar.local/ns/northstar-platform/sa/agentgateway`. That string is the *audience* every agent's token is minted for. The gateway verifies tokens by fetching SPIRE's public keys (JWKS) over HTTP. Cilium network policies admit only the gateway into payments-mcp and the Policy Decision Service, and the config is read once at startup, so `setup-kind.sh` forces a rollout restart after every change.

`deploy/platform/agentgateway/deployment.yaml:1-88 · scripts/setup-kind.sh:432-445, 937-955`

The whole gateway behaviour lives in one 312-line file, mostly comments recording why each value is what it is. Stripped, it fits in one figure.

```
frontendPolicies:
  tracing:
    host: otel-collector…:4317
    protocol: grpc                                   1
    randomSampling: true                             2
binds:
- port: 3000
  listeners:
  - routes:
    - policies:
        jwtAuth:
          mode: strict                               3
          issuer: http://spire-…-provider…
          audiences: [spiffe://…/agentgateway]       4
          jwks: {url: http://spire-…/keys}           5
        mcpGuardrails:
          processors:
          - kind: remote                             6
            methods: {"tools/call": request}         7
            host: policy-decision-service…:9100
            failureMode: failClosed                  8
            metadata: {caller: 'jwt.sub'}            9
            requestHeaders:
              allowed: [traceparent, tracestate]     10
      backends:
      - mcp: {targets: [payments-mcp …:8000/mcp]}
```

1. protocol: the struct's doc comment says HTTP, the enum's default is gRPC. So say it.
2. v1.5.0 drops a request with no incoming traceparent unless this is set.
3. The default is Optional, which lets a request with no token straight through.
4. The gateway's own SPIFFE ID. A token minted for it cannot be replayed elsewhere.
5. Remote JWKS, so SPIRE key rotation is picked up. A file copy would go stale.
6. The ExtMCP gRPC hook: an external Policy Decision Service decides.
7. Only tools/call, before forwarding. Methods with no key skip the hook.
8. Already the default. Written anyway, because failOpen is one word away.
9. CEL over the already-verified token. Reaches the PDS as caller.
10. An exhaustive allowlist. Leave out traceparent and the policy spans orphan.

Orange: tracing and identity. Purple: the policy hop.

FIGURE 6.2

Northstar's gateway config with the comments stripped and long values shortened. Ten values change behaviour; each one has a reason recorded in the file. Notice what is absent: there is no `mcpAuthorization` block (the decision belongs to the Policy Decision Service) and no `preserveToken` (so the credential never travels past the gateway).

TRACING: CALLOUTS 1 AND 2

Without a `frontendPolicies.tracing` block the gateway emits no spans. The first version of the file had none, and the trace showed settlement-agent talking straight to payments-mcp. **A proxy you cannot see in the trace is still there; you have just lost the evidence of it.** `protocol: grpc` is stated because v1.5.0's source disagrees with itself (the doc comment says HTTP, the enum's default is gRPC).

`randomSampling: true` matters most. **Sampling** is deciding which requests get recorded. A request that arrives with a W3C `traceparent` header already carries an upstream decision, and v1.5.0 records it. A request *without* one is dropped by default.

settlement-agent always sends the header, so the setting is for callers Northstar does not have yet: a headerless client, an operator's curl, a probe. For a regulated platform, the calls you did not expect are the ones you most need recorded. `tests/unit/test_no_sampling.py` asserts the setting, along with no sampler anywhere else.

IDENTITY: CALLOUTS 3, 4 AND 5

A **JWT** is a sealed envelope of claims: issuer (`iss`), subject (`sub`), audience (`aud`), expiry (`exp`). Anyone with the issuer's public key can check the seal. A **JWT-SVID** is one issued by SPIRE whose `sub` is a **SPIFFE ID**, a passport for a program (Chapter 7). `jwtAuth` answers "who is calling?", never "may they?". `mode: strict` overrides the v1.5.0 default `Optional` , which lets a request with no token through. `audiences` holds the gateway's own SPIFFE ID, so a captured token cannot be replayed elsewhere. `jwks.url` points at SPIRE's live keys, so key rotation is picked up.

THE POLICY HOP: CALLOUTS 6 TO 10

> IN PLAIN WORDS
>
> **ExtMCP** is agentgateway's hook for asking an outside service about an MCP request. Before forwarding, the gateway sends the details over **gRPC** to a service you write, and waits for pass, change, or refuse. In Northstar that is the **Policy Decision Service (PDS)**. **Fail-closed** means: no answer is a no.

The config comment puts the split well: `jwtAuth` establishes who is calling, `mcpGuardrails` what they may do; "losing either makes the other meaningless". Only `tools/call` is sent, at the request phase, "the only moment at which a refusal is still a refusal rather than a regret". `failClosed` is the measured default, written anyway because " `failOpen` is one word away". `metadata: {caller: 'jwt.sub'}` passes the already-verified identity to the PDS, which never sees the token. The header allowlist (callout 10) gets its own section below. Chapter 7 explains why Northstar uses this hook rather than built-in CEL `mcpAuthorization` : in v1.5.x, tool arguments such as the amount are not available to CEL authorization.

Three things the gateway does on every call

With the config in hand, each tool call is three steps in a fixed order. Each can stop the call, and each leaves a different mark in the trace.

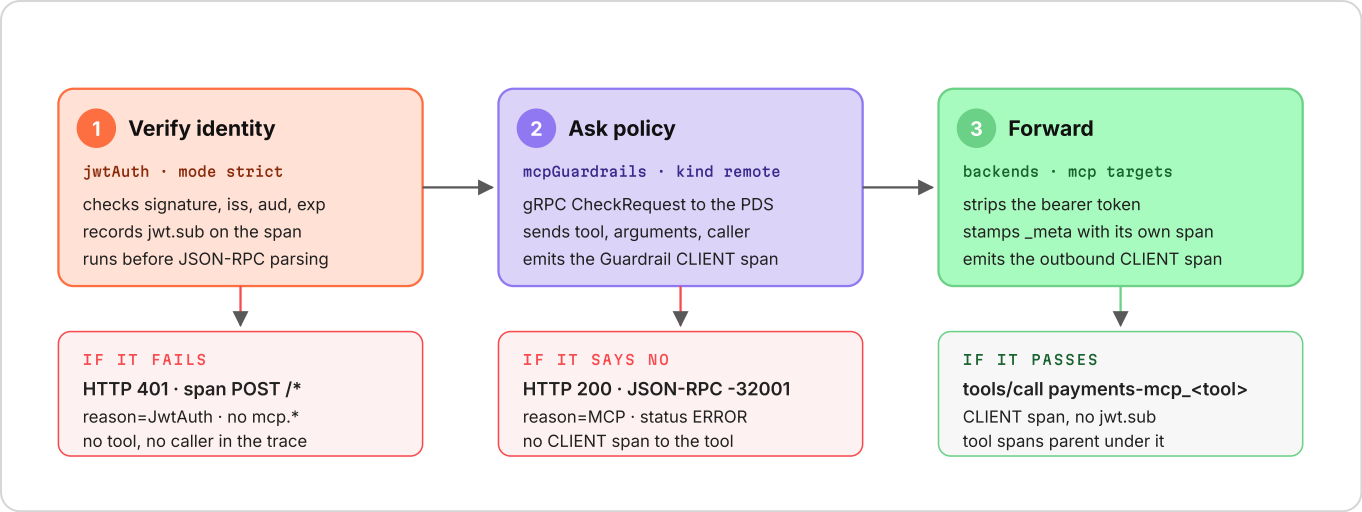


FIGURE 6.3

Three steps, three possible marks in the trace. Read the bottom row left to right: the earlier a call is stopped, the less the trace knows about it. An authentication failure cannot name the tool or the caller. A policy refusal names both. Only a permitted call produces a span towards the tool.

The permit path: every span

IN PLAIN WORDS

A **span** is one timed piece of work with a name and key-value **attributes**; spans nest into a **trace**. A **SERVER** span says "I received a request", a **CLIENT** span "I made a request". A proxy usually writes both.

Here is a permitted payment from the live capture of 2026-09-28, trace `d5eaf6dc…` (€5,800, executed). Between the request the gateway receives and the one it sends on, two things change: the bearer token is removed, and the trace context in `_meta` is re-stamped to point at the gateway's own span.

PERMIT · €5,800 EXECUTE_PAYMENT · TRACE D5EAF6DC…

```
settlement-agent POST CLIENT httpx2, injects the traceparent header
  agentgateway tools/call SERVER 22 attrs · jwt.sub = spiffe://.../settlement-agent
    agentgateway Guardrail CLIENT 9 attrs · grpc.status = 0
    policy-decision-service policy.decision SERVER policy.decision = permit · sibling of Guardrail
    agentgateway tools/call payments-mcp_execute_payment CLIENT 6 attrs · no jwt.sub
      payments-mcp tools/call execute_payment SERVER parent read from the re-stamped _meta
        payments-mcp POST CLIENT
          payments-api POST /payments/{paymentId}/execute SERVER
```

FIGURE 6.4

The permit tree. The gateway contributes three spans of its own, and the policy service one more. Notice that `policy.decision` hangs off `tools/call`, beside `Guardrail`, not under it; we come back to that. Session housekeeping spans (`initialize`, `notifications/initialized`, `GET /*`, `DELETE /*`, each as a SERVER and CLIENT pair) are omitted.

Three traps catch people reading this tree. **The SERVER span is named just `tools/call`**, with no tool in the name, so you must read `gen_ai.tool.name` to tell

`propose_payment` from `execute_payment`. **`payments-mcp_execute_payment` is the multiplexing prefix**; payments-mcp calls the same call `tools/call execute_payment`, so searching for one name will not find the other. And **`GET /*` and `DELETE /*`** housekeeping spans are named after the HTTP route, because those transport verbs carry no JSON-RPC method.

> **FROM THE NORTHSTAR DEMO**
>
> The Python MCP client opens one MCP session per tool call, so each call costs the gateway 11 spans (five SERVER/CLIENT pairs plus one `Guardrail`, minus the outbound CLIENT if refused). Permit: two tool calls, 22 gateway spans. Deny: three, 32. Whole-trace totals vary run to run, which is why the repository publishes trace *shape*, not totals.
>
> `docs/telemetry-contract.md:3148-3179 · book assets/data/gateway-spans.md`

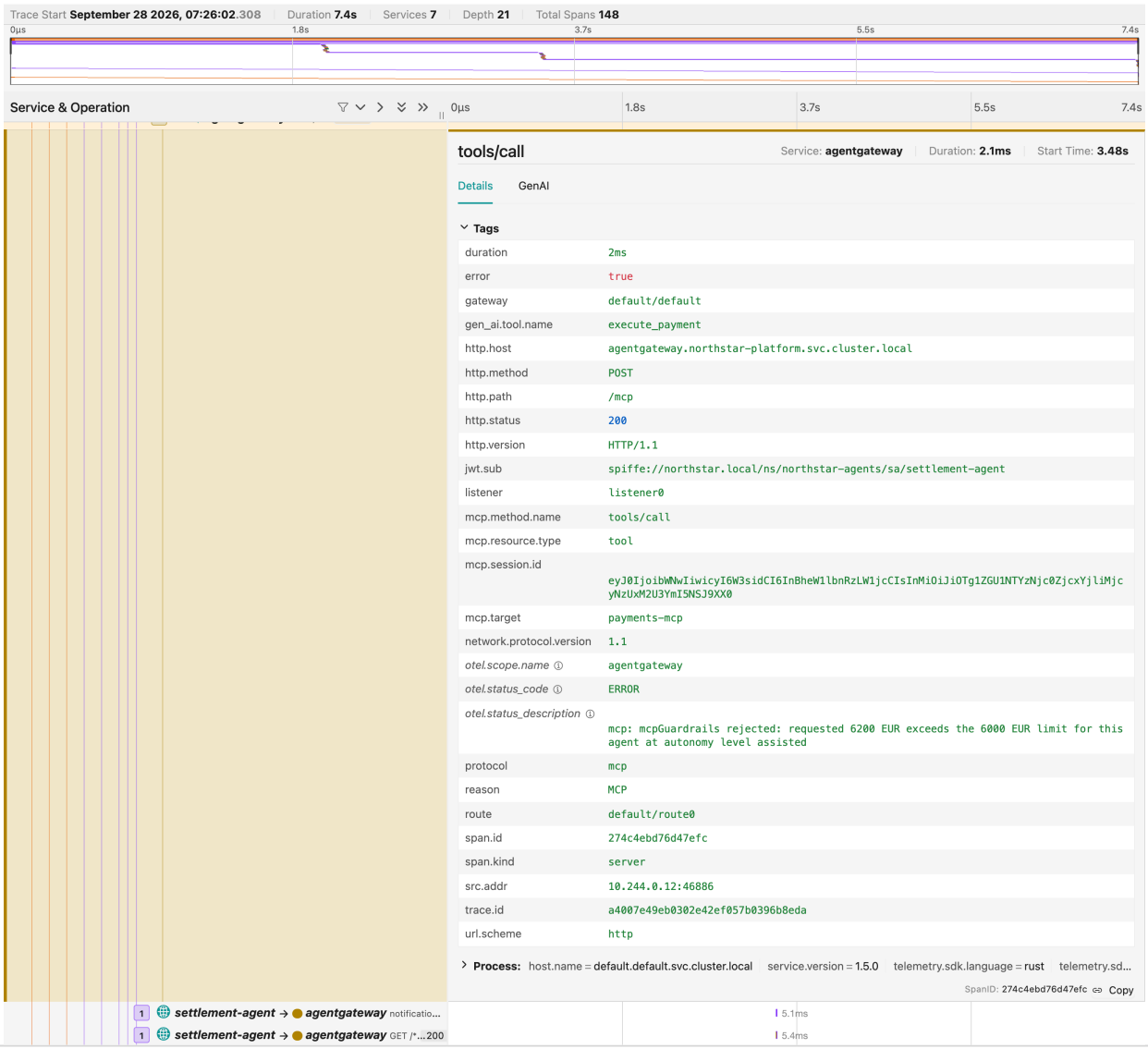


FIGURE 6.5

A gateway `tools/call` SERVER span in Jaeger, from the deny trace `a4007e49…`: this is the refused `execute_payment`, shown here to preview the next sections. Look for `jwt.sub` (the verified caller), `gen_ai.tool.name` (the only place the tool is named), `http.status = 200`, and the four keys only a refusal carries: `error = true`, `otel.status_code = ERROR`, `otel.status_description` with the guardrail message, and `reason = MCP`. `span.kind` is Jaeger's, not the gateway's. The Process line shows `host.name = default.default.svc.cluster.local` and `service.version = 1.5.0`.

The permit path: every attribute

The build pinned every gateway span's key set by exact equality in `tests/e2e/test_gateway_trace.py`: a key appearing fails as much as one disappearing. So the tables below are complete. The 2026-09-28 capture re-checked them: 22 keys on an accepted `tools/call`, 26 on a refused one, 9 on `Guardrail`, 6 on the outbound CLIENT span, 20 on the 401.

ATTRIBUTE	MEASURED VALUE	WHAT IT TELLS YOU
duration	"3ms"	Span length again, as a string
gateway	"default/default"	Gateway name (standalone default)
gen_ai.tool.name	"execute_payment"	The tool. Not in the span name
http.host	"agentgateway.northstar-platform.svc.cluster.local"	Host header received
http.method	"POST"	Non-semconv name (Finding 7)
http.path	"/mcp"	Non-semconv name (Finding 7)
http.status	200	Also 200 on a policy refusal
http.version	"HTTP/1.1"	Non-semconv name and value
jwt.sub	"spiffe://northstar.local/ns/northstar-agents/sa/settlement-agent"	The verified caller. Semconv would say user.id
listener	"listener0"	Listener that accepted it
mcp.method.name	"tools/call"	JSON-RPC method
mcp.resource.type	"tool"	Tool, prompt or resource
mcp.session.id	base64 JSON	Changes format after initialize
mcp.target	"payments-mcp"	Which backend server
network.protocol.version	"1.1"	Stable semconv
otel.scope.name	"agentgateway"	Instrumentation scope, no version
protocol	"mcp"	"http" on an auth refusal
route	"default/route0"	Route that matched
span.id	the span's own id	Duplicates the envelope
src.addr	"10.244.0.138:56988"	Caller's pod IP and port
trace.id	the trace's own id	Duplicates the envelope
url.scheme	"http"	Stable semconv

FIGURE 6.6

The `tools/call` SERVER span on an accepted call: 22 keys, the complete set. Highlighted rows are the ones an auditor asks about. Notice there is no amount, no claim ID, and no argument of any kind: the gateway records *that* a tool was called and by whom, never *with what*.

The gateway carries no **`northstar.claim.id`**, deliberately: Northstar never enriches third-party spans, and a test fails if anyone starts.

ATTRIBUTE	MEASURED VALUE	WHAT IT TELLS YOU
agentgateway.outbound.kind	"Policy"	A policy hop, not the primary call
agentgateway.outbound.subtype	"Guardrail"	Which kind of policy hop (PascalCase)
grpc.status	0	Transport succeeded. Not the decision
http.host	"policy-decision-service.northstar-platform.svc.cluster.local"	Which policy service
http.method	"POST"	gRPC runs over HTTP/2 POST
http.path	"/agentgateway.dev.ext_mcp.ExtMcp/CheckRequest"	The exact RPC the gateway called
mcp.method.name	"tools/call"	Which MCP method was checked
mcp.target	"payments-mcp"	Which backend it was for
otel.scope.name	"agentgateway"	
processor name, metadata keys	(absent)	Nothing names the guardrail config
decision, JSON-RPC error code	(absent)	The verdict is not on this span

FIGURE 6.7

The `Guardrail` CLIENT span: 9 keys, identical on permit and deny. Notice `http.path`: an operator who has never seen the config can tell from this one span that the gateway asked an external service for a decision, and which RPC it used. Notice too what the span cannot tell you: what the answer was.

`grpc.status = 0` reports the transport, not the decision. That is what separates "the policy said no" from "the policy service was down": an unreachable PDS under `failClosed` rejects the call with `mcpGuardrails checkRequest failed: … Connection refused`.

CHECK BEFORE A SLIDE

The outage rejection was measured; the `Guardrail` span's exact attributes during an outage were not. Do not show a specific non-zero `grpc.status` without capturing one.

ATTRIBUTE	MEASURED VALUE	WHAT IT TELLS YOU
`agentgateway.outbound.kind`	`"Primary"`	The real call, not a policy hop
`agentgateway.outbound.subtype`	`"Mcp"`	To an MCP backend
`gen_ai.tool.name`	`"execute_payment"`	Tool, without the prefix
`mcp.method.name`	`"tools/call"`	
`mcp.target`	`"payments-mcp"`	
`otel.scope.name`	`"agentgateway"`	
`jwt.sub`	(absent)	The credential was stripped
`http.*, duration, session id`	(absent)	No HTTP detail on the outbound leg

FIGURE 6.8

The outbound `tools/call payments-mcp_<tool>` CLIENT span: six keys and nothing else. The missing `jwt.sub` is correct, not a gap. Without `preserveToken`, `jwtAuth` removes the bearer token before the upstream call, so on this leg there is no verified identity to attribute.

So **payments-mcp learns nothing about who called it.** The tool is unmodified and "governed without being told it is being governed", but identity lives only at the gateway and in the policy decision. If a regulator wants the payment system itself to record the requesting agent, v1.5.0 cannot do it without forwarding the token.

The policy deny: four keys and a missing span

Now the €6,200 claim. settlement-agent's limit at the assisted autonomy level is €6,000, so the PDS refuses `execute_payment`. Live capture trace `a4007e49…` has three tool calls: `propose_payment` permitted, `execute_payment` refused, then `refuse_payment` (the agent's terminal no-money tool) permitted.

POLICY DENY · €6,200 · TRACE A4007E49… · THREE TOOL CALLS

- settlement-agent **invoke_agent settlement_agent** INTERNAL
  - settlement-agent **execute_tool propose_payment** gateway tools/call → Guardrail + policy.decision permit → CLIENT → payments-mcp → payments-api
  - settlement-agent **execute_tool execute_payment** INTERNAL **ERROR**
    - settlement-agent **POST** CLIENT
      - agentgateway **tools/call** SERVER 26 attrs · http.status = 200 · otel.status_code = ERROR **refused**
        - agentgateway **Guardrail** CLIENT same 9 attrs · grpc.status = 0
        - policy-decision-service **policy.decision** SERVER rule settlement-agent.assisted.execute_limit · limit 6000 · requested 6200 **deny**
        - ~~agentgateway tools/call payments-mcp_execute_payment CLIENT~~
        - ~~payments-mcp tools/call execute_payment SERVER~~
        - ~~payments-api POST /payments/{paymentId}/execute SERVER~~
  - settlement-agent **execute_tool refuse_payment** gateway tools/call → Guardrail + policy.decision permit → CLIENT → payments-mcp

FIGURE 6.9

The policy-deny tree, with the two permitted calls collapsed to one line each. The struck-through lines are spans that exist on the permit path and do not exist here. That absence is the evidence that the payment was never attempted, as opposed to attempted and failed. Session housekeeping spans are omitted.

The refused SERVER span carries **26 keys**: the same 22, plus exactly four.

ADDED ON REFUSAL	MEASURED VALUE	MEANING
`error`	`true`	Something went wrong, in the gateway's view
`otel.status_code`	`"ERROR"`	Span status. This is what turns the span red
`otel.status_description`	`"mcp: mcpGuardrails rejected: requested 6200 EUR exceeds the 6000 EUR limit for this agent at autonomy level assisted"`	Fixed gateway prefix, then the PDS's reason verbatim
`reason`	`"MCP"`	Which layer refused. Not which rule
`http.status`	`200 (unchanged)`	The HTTP exchange succeeded
`gen_ai.tool.name, mcp.*, jwt.sub`	`all present`	You can tell which tool and which caller

FIGURE 6.10

The complete difference between an accepted and a policy-refused `tools/call` SERVER span. Four keys are added; nothing is removed; `http.status` stays 200. Strictly, `error`, `otel.status_code` and `otel.status_description` are Jaeger's rendering of the OTLP span status rather than attributes on the wire; the count of 26 follows the contract's convention of counting what Jaeger shows, minus `span.kind`.

> "Anyone alerting on HTTP status alone will not see this refusal at all."
>
> DOCS/TELEMETRY-CONTRACT.MD · AGENTGATEWAY'S GUARDRAIL TELEMETRY, AS MEASURED

Why 200? ExtMCP refuses by returning a JSON-RPC error (code `-32001`) inside a successful HTTP response. A dashboard of HTTP 4xx and 5xx rates shows a healthy gateway during a wave of refusals. To see refusals, alert on `otel.status_code = ERROR` with `reason = "MCP"`, or on the `policy.decision` span. `reason` names the layer (`"JwtAuth"` or `"MCP"`), not the rule; the rule lives only on the PDS span and in the free-text description. Note also that the PDS leaves its own span status unset on a deny, so an error-rate SLO over gateway spans counts refusals and one over PDS spans does not. Decide on purpose.

One shape surprise: `policy.decision` is a **sibling** of `Guardrail`, not its child. Both hang off the gateway's `tools/call`, because the gateway hands the hook the context it *received*, not its own `Guardrail` span's. A query that walks `Guardrail → policy.decision` finds nothing.

THE ABSENCE OF SPANS IS THE EVIDENCE

The most important fact in Figure 6.9 is carried by nothing: no CLIENT span to payments-mcp, no payments-mcp span, no payments-api execute span. "The refusal is not a call that failed; it is a call that never happened." An error is something you click on; an absence is something you must know to look for. And it depends on completeness: if spans were sampled or dropped, "no span" could mean "span lost".

You can only argue from absence when you can trust presence, which is why Northstar never samples (Chapter 9).

TAKEAWAY FOR PLATFORM TEAMS

Write the absence down as a positive check: "every `tools/call` SERVER span with `reason = MCP` has no child with `agentgateway.outbound.kind = Primary`". Pair it with 100% sampling, or the absence proves nothing.

The authentication deny: a refusal with no name

What if the caller has no valid token? The live capture sent a `tools/call` for `execute_payment` with no `Authorization` header (trace `b603db5c…`). The gateway emitted exactly one span.

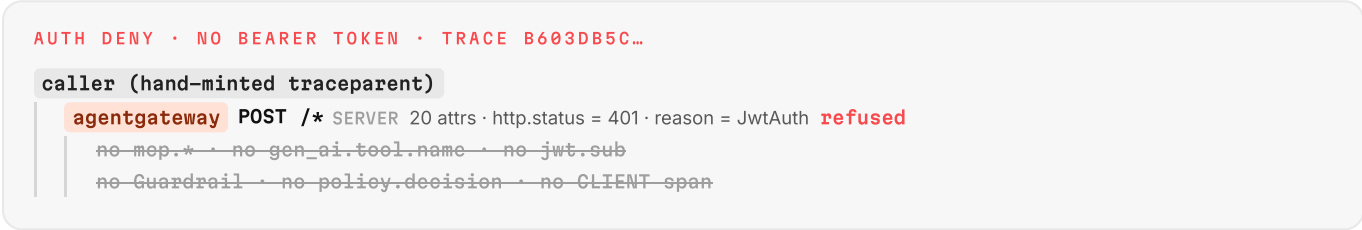


FIGURE 6.11

The authentication-deny tree is one span. Notice its name: `POST /*`, not `tools/call` or `initialize`. The gateway never parsed the request as MCP, so it has no method to name the span after, and no tool or caller to record.

ATTRIBUTE
error
http.status
otel.status_code
otel.status_description
protocol
reason
duration, gateway, http.host, http.method, http.path, http.version, listener, network.protocol.version, otel.sc
mcp.method.name, mcp.target, mcp.session.id, mcp.resource.type, gen_ai.tool.name
jwt.sub

`jwtAuth` is an HTTP filter that runs before the body is parsed as MCP. The trace can say an unauthenticated POST was refused, and why, but not *which tool* or *who*. Nothing was verified, so nothing is attributed. If an auditor asks "which tools did unauthenticated callers try last week?", gateway traces cannot answer; the only lead is `src.addr`.

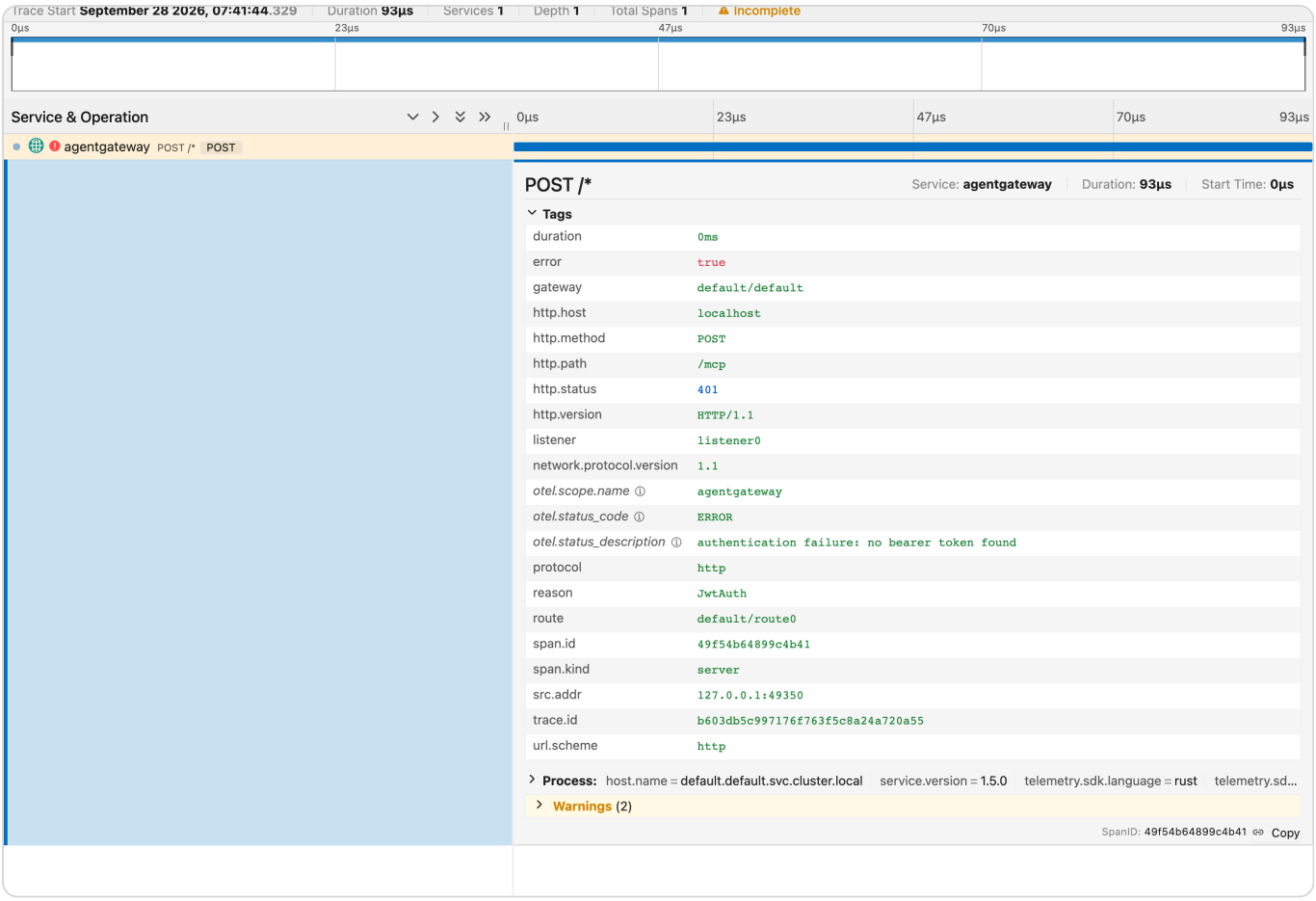


FIGURE 6.12

The 401 in Jaeger: one span named `POST /*`, 93 µs, `http.status = 401`, `protocol = http`, `reason = JwtAuth`, `otel.status_description = authentication failure: no bearer token found`. Compare it with Figure 6.5: every `mcp.*` key, `gen_ai.tool.name` and `jwt.sub` are missing. Jaeger marks the trace "Incomplete" because its parent is the span id the test minted by hand; `http.host` reads `localhost` because the call came through a port-forward.

Trace context through the gateway

IN PLAIN WORDS

Trace context is a parcel's tracking number, copied from each incoming request onto the outgoing one. MCP has two places for it: the HTTP **`traceparent` header** (a label outside the box) and **`params._meta`** inside the JSON-RPC message (a slip inside the box, standardised by SEP-414 for transports like stdio that have no outside).

agentgateway treats the two asymmetrically (**Finding 6**, Chapter 12). **On the way in** it reads *only* the header; with none, it mints a new trace root. **On the way out** it writes `_meta` on every forwarded message with its own span context, overwriting what the client sent. Northstar's trace holds because settlement-agent sends both: `mcp_tools.py` injects `_meta` by hand, and the instrumented httpx client adds a `traceparent` header automatically. A stdio client, or any client whose HTTP layer is not instrumented, *will* fragment at this hop: "a proxy that silently re-roots a trace is worse than an untraced one."

The M2 feasibility spike first got this wrong. Its throwaway client put context only in `_meta` and concluded that the trace fragments at the gateway. A re-test with a client that mirrored settlement-agent's real instrumentation found the trace unbroken end to end.

> "Your test harness is also a client."
>
> BUILD LESSON FROM THE M2 SPIKE CORRECTION

CHECK BEFORE A SLIDE

MCP `2026-07-28` is stateless and moves more context into `_meta`, which makes header-only ingress matter more. No ingress `_meta` fix was found in the v1.6.0 alpha notes. Re-check before November.

Finding 7: the attributes are an access-log schema

IN PLAIN WORDS

A **semantic convention** (semconv) is an agreed name for a piece of telemetry: OpenTelemetry says the HTTP method goes in `http.request.method`, the logged-in user in `user.id`. If every component uses the agreed names, one query works across all of them.

Look back at Figure 6.6: `http.method`, `http.path`, `http.status`, `http.host`, `http.version`. These are not the stable OpenTelemetry HTTP names, and `http.path` and `http.status` are not even the legacy ones. The capture confirms it: all 32 gateway spans in the deny trace carry these old names and none carries `http.request.method`, `url.path`, `http.response.status_code` or `server.address`. (agentgateway's `gen_ai.*` and `mcp.*` keys are the semconv-aligned part.) The cause is one decision in v1.5.0's `telemetry/log.rs`: one list of key-value pairs feeds the tracer, the OTLP logger and the stdout access log. **The span attributes are the access-log fields, reused verbatim.** Only `network.protocol.version` and `url.scheme` go through a semconv module, and those two are correct.

WHAT IT IS	STABLE SEMCONV	OTEL LEGACY	AGENTGATEWAY V1.5.0	PYTHON AND GO SERVICES IN NORTHSTAR
request method	`http.request.method`	`http.method`	`http.method`	`http.request.method`
request path	`url.path`	`http.target`	`http.path`	`url.full` / `url.path`
response status	`http.response.status_code`	`http.status_code`	`http.status`	`http.response.status_code`
server host	`server.address`	`http.host`	`http.host`	`server.address`
protocol version	`network.protocol.version`	`http.flavor`	`network.protocol.version`	same
authenticated principal	`user.id`	`enduser.id`	`jwt.sub`	(none authenticate)

FIGURE 6.13

Three HTTP vocabularies in one trace. The Python services use the stable names only because `OTEL_SEMCONV_STABILITY_OPT_IN=http` forces them to; payments-api's Go `otelhttp` uses them by default; the gateway uses its log field names. Notice the bottom row: the one component that actually authenticates anyone records the principal under a name no `user.id` query will find.

You cannot write one `http.response.status_code` filter across a Northstar trace, or group by principal across `user.id` and `jwt.sub`. Yet every span sits in the right place in the right tree.

> "Trace context is what makes the chain one thing; attribute naming is what makes it queryable."
>
> DOCS/UPSTREAM-FINDINGS.MD · FINDING 7

The fix exists but is not released. PR **#3182** aligns the HTTP keys with semconv for traces and OTLP logs. It was opened on 2026-08-26, one day before v1.5.0, but merged on **2026-09-01**, after it. It ships only in **v1.6.0-alpha.1** (2026-09-14) and **alpha.2** (2026-09-22); on 2026-09-28 there is no v1.6.0 GA. It does not touch `jwt.sub`, and nobody has asked upstream for that.

> FROM THE NORTHSTAR DEMO
>
> Northstar rewrites none of the gateway's attributes. The test pins the v1.5.0 key sets and asserts the semconv names are *absent*, so an upgrade carrying #3182 turns it red. That is how the project intends to find out.
>
> `tests/e2e/test_gateway_trace.py:275, 317, 467–517`

Finding 10: the allowlist that drops traceparent

`mcpGuardrails.processors[].requestHeaders.allowed` controls which incoming headers reach the policy service. It reads like an addition and is a replacement: an empty list forwards every header, a non-empty one forwards only those named. Name one header for the policy service and W3C trace context stops being forwarded.

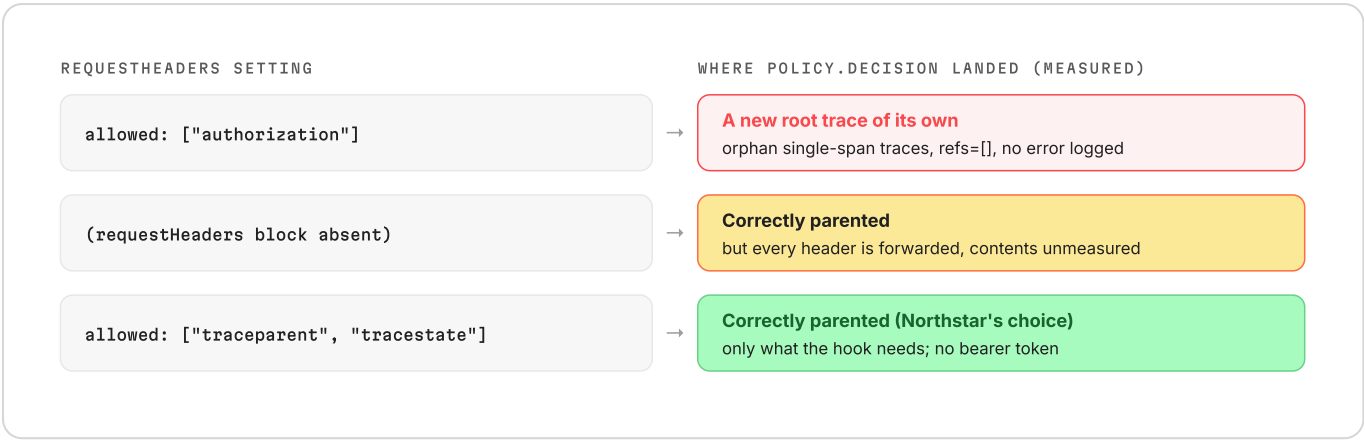


FIGURE 6.14

Three values, each measured by driving one real `tools/call` with a minted JWT-SVID and a known `traceparent`. The top row is what Northstar shipped first. The policy worked perfectly in every row; only the trace changed.

Northstar shipped the top row first, and it looked like success: the config validated, the €6,200 was refused, the PDS spans existed with every attribute. But every `policy.decision` span started a new root trace, and no log line mentioned a header. Northstar now names exactly `traceparent` and `tracestate`, and leaves `authorization` out deliberately.

TAKEAWAY FOR PLATFORM TEAMS

Test trace continuity across every policy hop, not just the verdict. "The €6,200 was refused" passes in all three rows of Figure 6.14; only "the decision span shares the caller's trace ID and has a parent" catches the top row. If you name any header, name `traceparent` and `tracestate` too.

SMALLER QUIRKS

`span.id`, `trace.id` and `duration` repeat the span envelope as attributes, and `duration` is a string (`"3ms"`). `mcp.session.id` is a UUID on `initialize` and base64 JSON afterwards, so it does not join a session to its own start. `host.name` on the Resource is `default.default.svc.cluster.local`, not the pod. `service.version=1.5.0` is on the Resource; no span carries `otel.scope.version`.

Metrics: what the gateway tells Prometheus, and what it cannot

agentgateway serves Prometheus metrics on `:15020/metrics`, all prefixed `agentgateway_`. In Northstar **none of them reach Prometheus**: on 2026-09-28 Prometheus held 315 series names and not one began with `agentgateway_`, because nothing scrapes the port. The capture read it directly through a port-forward after seven model-backed claims, one human approval and one unauthenticated curl.

MEASURED SERIES	VALUE	WHAT IT TELLS YOU
`agentgateway_upstream_call_duration_seconds_count{kind="Policy",subtype="Guardrail"}`	19	One per `tools/call` (7 propose + 8 execute + 3 refuse + 1 get). Permit and deny counted together: no decision label
`agentgateway_upstream_call_duration_seconds_count{kind="Primary",subtype="Mcp"}`	92	Every forwarded MCP message, housekeeping included
`agentgateway_requests_total{protocol="mcp",method="POST",status="200",reason="MCP"}`	3	One per refused `execute_payment`. The only metric trace of a policy deny, and it says 200
`agentgateway_requests_total{protocol="mcp",method="POST",status="200",reason="Upstream"}`	35	Accepted MCP POSTs carry `reason="Upstream"`
`agentgateway_requests_total{protocol="http",status="401",reason="JwtAuth"}`	1	The unauthenticated curl
`agentgateway_mcp_requests_total{method="tools/call",resource="execute_payment"}`	8	3 refused + 5 executed. Refused calls still count
`agentgateway_mcp_requests_total{… resource="propose_payment" | "refuse_payment" | "get_payment"}`	7 · 3 · 1	Calls per tool; labels `method`, `resource_type`, `server`, `resource`
`agentgateway_build_info{tag="1.5.0"}`	1	Confirms the pinned version
`agentgateway_guardrail_checks_total`	(absent)	Not exposed after 19 MCP guardrail checks

FIGURE 6.15

The gateway's governance-relevant metrics, measured on 2026-09-28 (series labels shortened). Notice that no series separates permitted from refused tool calls: the Guardrail histogram counts every check, and `mcp_requests_total` counts refused calls alongside executed ones.

- **No metric counts permit versus deny.** The Guardrail latency histogram counts every check, whatever the answer. The only indirect deny signal is `agentgateway_requests_total{reason="MCP"}`, with `status="200"`. An alert on `status=~"4..|5.."` never fires for a policy refusal, the same trap as on the span.
- **`guardrail_checks_total` does not count MCP guardrails.** It never appeared; the source increments it only on the LLM guardrail path.
- **Identity is not a label.** "How many `execute_payment` calls per agent?" needs a custom CEL label, which Northstar has not deployed.

TAKEAWAY FOR PLATFORM TEAMS

For MCP governance, the gateway's metrics give volume and latency; the decision lives only in traces and the policy service's telemetry. Derive a refusal rate from spans (`reason = MCP`, or `policy.decision = deny`), or emit a counter from your policy service. And scrape `:15020`.

Access logs

agentgateway also writes a structured access log line to stdout per request. Because of Finding 7, its fields match the span attributes. Two upstream features extend it: OTLP

log export (`frontendPolicies.accessLog.otlp.host`) and CEL fields, including `mcp.tool.arguments` , available *only* here. Adding it would record the payment amount at the checkpoint, an audit trail that is also content capture (Chapter 9). Northstar configures no `accessLog` block; its audit evidence comes from spans.

CHECK BEFORE A SLIDE

Upstream docs and v1.5.0 measurements disagree in places: docs say the policy subtype is lowercase `guardrail` (measured `Guardrail`), that policy spans carry `http.status` (measured `grpc.status`), and that denied spans carry `mcp.error.*` (measured neither). Before November: check for a v1.6.0 GA and re-run `test_gateway_trace.py` ; check whether these attributes and ingress `_meta` extraction changed. Findings 6 to 10 and the `jwt.sub` gap are unfiled upstream; decide whether to file before calling them defects on stage.

Cheat sheet: where each answer lives

QUESTION	WHERE IN GATEWAY TELEMETRY	CAVEAT
Which workload made this tool call?	`jwt.sub` on `tools/call` SERVER; also `policy.caller.spiffe_id` on `policy.decision`	Not `user.id`. Absent on CLIENT spans
Which tool?	`gen_ai.tool.name` on SERVER and CLIENT; CLIENT span name `payments-mcp_<tool>`	Not in the SERVER span name
With what arguments?	Not on gateway spans. `policy.request.amount` on the PDS span; `mcp.tool.arguments` in access logs if configured	Content capture decision
Was a policy consulted?	`Guardrail` CLIENT span, `http.path=/agentgateway.dev.ext_mcp.ExtMcp/CheckRequest`	Present on permit and deny
What was decided, under which rule?	Not on gateway spans. `policy.decision`, `policy.rule`, `policy.limit.amount` on the sibling PDS span	Sibling of `Guardrail`, not child
Why was it refused, in words?	`otel.status_description` on `tools/call` SERVER	Free text, unbounded, caller-influenced
Was it refused at all?	`otel.status_code=ERROR` + `reason` (`MCP` or `JwtAuth`)	`http.status` is 200 for policy refusals
Did it reach the tool?	Presence of `tools/call payments-mcp_*` CLIENT span and a payments-mcp child	Absence is the evidence; needs 100% sampling
Policy said no, or policy was down?	`grpc.status` on `Guardrail` + presence of `policy.decision` vs `checkRequest failed`	Outage span shape unpublished
Who tried without a valid token, and for which tool?	`POST /*` span, `http.status=401`, `reason=JwtAuth`, `src.addr`	Cannot say who or which tool
How many calls per tool?	`agentgateway_mcp_requests_total{method="tools/call",resource=…}`	Measured; counts refused calls too; no identity label
How many policy refusals?	`agentgateway_requests_total{reason="MCP"}` (status 200); better, spans with `reason = MCP`	No metric counts permit vs deny
Could the agent have gone around the gateway?	Not in gateway telemetry at all	Proven by Cilium policy and BPF evidence (Chapter 7)

FIGURE 6.16

The gateway telemetry cheat sheet. Read the right-hand column before relying on the middle one. Notice how many answers live on the Policy Decision Service's span rather than the gateway's: the gateway says a decision happened; the decision service says what it was.

TAKEAWAY FOR PLATFORM TEAMS

Turn tracing on and sample everything, headerless requests included, or the gateway is a silent hop. **Alert on span status and `reason`, not HTTP status. Test trace continuity across every hop** with a client that behaves like your real one. **Pin the attribute names you depend on in a test**, so an upgrade tells you when they change.

CHAPTER 7

Limiting access to MCP, layer by layer

Eight layers stand between an agent and a tool that moves money. For each one, this chapter shows what it stops, how Northstar builds it, and which span or measurement proves it held.

Chapter 6 looked at agentgateway as a source of telemetry. This chapter asks the question platform teams bring to every agent review: *how do we limit what an agent can do with an MCP server?* No single control does it. Northstar stacks eight, from the network inward to the tool. For each we ask: what does it stop, and when an auditor asks whether it worked, where do you point?

Why the check has to sit in the path

IN PLAIN WORDS

MCP (Model Context Protocol) is how an agent calls a tool. The agent sends a JSON message named `tools/call` with a tool name and arguments, such as `execute_payment` with an amount. The tool runs and answers.

Interposition means putting a checkpoint *between* the caller and the thing it calls, so every request has to pass through it. Think of airport security: the airline builds one corridor to the gate and puts a scanner in it. The passenger cannot skip the scanner, because there is no other corridor.

The rule "this agent may pay at most €6,000" could live in the tool, in the agent, or between them. Northstar's design notes argue the choice in three sentences.

> "If the tool checks policy itself, the application is deciding and a reviewer can fairly call it an if-statement with extra steps. If the agent checks, an agent that can choose to ask permission can choose not to. Only interposition makes 'the platform decides' literally true."
>
> NORTHSTAR M2 DESIGN DECISIONS · DECISION 2 (PRIVATE BUILD LEDGER)

The middle sentence carries the argument. The model decides which tools to call. If the permission check is a tool it is supposed to call first, skipping it is one more choice, and a prompt injection, a confused plan or a bug can all make it.

> "The model can request. The platform decides if it becomes an effect."
>
> NORTHSTAR M2 DESIGN DECISIONS · THE LINE M2 WAS BUILT TO EARN

FROM THE NORTHSTAR DEMO

Neither `settlement-agent` nor `payments-mcp` contains the €6,000 limit, and a test enforces that. Criterion 4 of `test_policy_denial.py` walks the production source of both with Python's `ast` module and fails on any policy vocabulary, any literal equal to a policy allowance, or any unlisted comparison involving money.

`tests/e2e/test_policy_denial.py:1155 · agents/settlement-agent/settlement.py`

The layered model

Figure 7.1 is the map for the rest of the chapter. Read it from top to bottom, the way a `tools/call` travels. The left side says what each layer does; the right side says what you would show an auditor. Look at the first row: the proof is not a span. Some layers leave traces, and some leave only configuration and measurements.



FIGURE 7.1

The eight layers between an agent and a payment. Blue rows answer *who*, orange and red rows answer *may they*, and the yellow row makes the gateway unavoidable. Notice that the proof column names spans for seven layers and a non-telemetry measurement for one: nothing in any trace can show a request that was never possible.

The autonomy level, which changes which allowance row a decision uses, belongs to Chapter 8. It appears here only as an attribute on the decision span.

Layer 1 · Network reachability: make the gateway the only corridor

IN PLAIN WORDS

A **network policy** is a firewall rule for pods. Kubernetes lets every pod talk to every other pod by default. A policy says "this pod may only receive traffic from these pods" (ingress) or "may only send traffic to these places" (egress). **Cilium** enforces these rules in the Linux kernel using eBPF, and can also allow traffic by DNS name (`toFQDNs`).

What it stops: a caller that skips the gateway. After M2.1, agentgateway verified a token on every call to `payments-mcp` , but only because the agent's config named the gateway. It was a choke point by convention, not by enforcement.

FROM THE NORTHSTAR DEMO

With no network policy applied, `claims-agent` , a workload with no identity and no business touching payments, sent an MCP `initialize` straight to `payments-mcp` and got **HTTP 200**. The file header records the verdict: the milestone's claim "was therefore true only for a cooperating caller."

`deploy/platform/cilium-policies.yaml:20-25`

How Northstar does it: ingress policies admit only agentgateway to `payments-mcp` and to the Policy Decision Service; each agent gets an egress allowlist. Figure 7.2 shows the result.

FROM \ TO	agentgateway	payments-mcp	PDS	payments-api	api.anthropic.com
settlement-agent	:3000	drop	drop	drop	toFQDNs :443
senior-settlement-agent	:3000	drop	drop	drop	toFQDNs :443
claims-agent	drop	drop (was 200)	drop	drop	drop
customer-ui	drop	drop	drop	drop	drop
reviewer-ui	drop	drop	drop	:8080	drop

A drop is a timeout, not a refusal: Cilium discards the packet and sends no reset. DNS still resolves.

Yellow: reviewer-ui reaches the ledger directly to list refusals. It is forbidden the governed path.

FIGURE 7.2

Who can open a connection to whom, read from the nine CiliumNetworkPolicy documents. Only the two settlement identities reach the gateway or the model provider. The red cell is the measurement that started it all. Notice the yellow cell: the ledger is reachable from one human-facing workload without passing any policy.

Three measured facts matter before you build your own. **Denials are timeouts:** a Cilium drop sends no reset, so the client sees `TimeoutError` . **DNS still resolves** for

hosts you cannot reach, so "it resolves" proves nothing. **The model provider is reachable only from the pods that need it**, via a `toFQDNs` rule for `api.anthropic.com:443`. With the PDS sealed off too, “Removing the governance means changing the platform, not the agent.”

What proves it: not a trace. Hubble is off, so no flows exist. `test_gateway_is_the_only_route.py` uses in-pod TCP probes and direct BPF policy-map reads; the map read catches a mutation the probe misses. A trace can only show that every `payments-mcp` span that *did* happen sits under a gateway span: “a statement about the traffic that happened, not about the traffic that is possible.”

Layer 2 · Workload identity: give every agent a passport

IN PLAIN WORDS

SPIFFE is a standard for naming workloads. Each gets an ID that looks like a URL: `spiffe://northstar.local/ns/northstar-agents/sa/settlement-agent` ("trust domain, namespace, ServiceAccount"). **SPIRE** checks a pod really is what it claims (*attestation*) and hands it a credential, here a **JWT-SVID**: a JSON Web Token carrying the SPIFFE ID. Think of a sealed envelope with a passport inside. Anyone can read the passport, only the issuer can seal the envelope, and the envelope says who it is addressed to (the *audience*) and when it expires.

What it stops: decisions keyed on something forgeable or reused, like an IP address. **How Northstar does it:** the agent asks a local SPIRE socket for a token addressed to the gateway; no secret sits in any manifest. Two lessons came out of building it, both about names. Every agent needs its own ServiceAccount: without one, SPIRE minted `.../sa/default`, and “an identity whose service-account component reads ‘default’ is not something a policy can decide on.” And an identity has to be registered to exist: `eval-scorer`'s SPIFFE ID sat in three allowlists, SPIRE had never minted it, and nothing reported the gap.

> “An identity nothing registers is a string, not an identity.”
>
> DOCS/ARCHITECTURE.MD

FROM THE NORTHSTAR DEMO

Six identities exist. `senior-settlement-agent` runs *identical code* to settlement-agent and gets a different token: “The difference between the two is an identity rather than a program.” Both share `service.name=settlement-agent`, so tell them apart by `jwt.sub`, “never by service name.” **What proves it:** `jwt.sub` on the gateway span and `policy.caller.spiffe_id` on `policy.decision`.

`deploy/platform/spire/registration.yaml · agents/settlement-agent/svid.py`

Layer 3 · Authentication at the gateway: check the seal

What it stops: anyone without a valid, correctly addressed token. **How Northstar does it:** `jwtAuth` verifies every token's signature against SPIRE's key set, then issuer, audience and expiry. The most important setting is one word: the default mode is `Optional`, which upstream warns "allows requests without a JWT." Northstar sets `mode: strict`.

What proves it: five kinds of bad credential, sent directly over MCP with no model involved. Every refusal is HTTP 401 with a plain-text body, because `jwtAuth` runs before the gateway parses the JSON-RPC message.

WHAT THE CALLER PRESENTED	THE GATEWAY'S REFUSAL (HTTP 401)	WHAT THIS ROW PROVES
No token at all	`no bearer token found`	Strict mode is on
`Bearer not-a-real-svid`	`token header is malformed: Error(InvalidToken)`	Garbage is not a token
Token from an untrusted issuer, its own key id	`token uses the unknown key`	Only SPIRE's keys are trusted
Untrusted issuer, forging SPIRE's key id	`Error(InvalidSignature)`	A copied key id does not help
Genuine SPIRE-minted SVID, wrong audience (aud = payments-mcp)	`Error(InvalidAudience)`	A real token for another service cannot be replayed here

FIGURE 7.3

Five ways to fail authentication. The highlighted row matters most: real signer, real issuer, real subject, only the audience differs. It is the only row proving that a token issued for one SPIFFE service in this trust domain cannot be used at another.

The tests were mutation-proven: removing `jwtAuth` turned all five red, and changing `strict` to `optional` turned only the no-token test red. In the trace, a refusal is a single `POST /*` span with `http.status=401`, `reason="JwtAuth"` and no `jwt.sub` or `mcp.*` key. The gateway refused before reading the message, so the trace cannot say which tool was attempted, or by whom. That is the price of refusing early, and it is the right price.

GO DEEPER

Why a bearer token and not mTLS? A client certificate cannot be replayed; a JWT can until it expires. mTLS was “deferred rather than declined”. The replay window is bounded (audience-scoped, 1-hour TTL), not eliminated. **Known gap:** the JWKS is fetched over plain HTTP inside the cluster.

`tests/e2e/test_caller_identity.py:488–669 · deploy/platform/agentgateway/config.yaml: 123–169`

Layer 4 · Tool-level authorization: static rules or a decision service

IN PLAIN WORDS

A **policy decision point** answers "yes or no"; a **policy enforcement point** asks and acts on the answer. The gateway is the enforcement point. It can hold the rules itself, like a bouncer with a printed guest list, or radio a decision service, like a bouncer who calls the manager. **CEL** (Common Expression Language) is a small, safe rule language that can only see the variables the host gives it.

What it stops: a known caller doing something it may not. agentgateway v1.5.0 offers two ways.

OPTION A · CEL `MCPAUTHORIZATION`

Rules in the gateway config over `mcp.tool.name` and the token's claims (`jwt.*`). **It hides tools:** denied tools vanish from `tools/list` . **A deny looks like a typo:** HTTP 400, `-32602 "Unknown tool"` , no rule name, no decision span. **It cannot see arguments in 1.5.x**, so "pay at most €6,000" cannot be a CEL rule.

OPTION B · EXTMCP (`MCPGUARDRAILS` , `KIND: REMOTE`)

For each configured MCP method, the gateway pauses and makes a gRPC `CheckRequest` to a service you write, carrying the method, the raw params (including the amount) and metadata you define. The service answers `Pass` , `mutated` or `AuthorizationError` .

The key detail is how identity arrives. Northstar configures `metadata: {caller: 'jwt.sub'}` , a CEL expression over the token `jwtAuth` has *already verified*. So the Policy Decision Service (PDS) receives an attested SPIFFE ID, not a name the agent typed, and never sees the token itself.

Why Northstar chose ExtMCP: the rule “needs an amount comparison and an identity table rather than an expression.” An ExtMCP deny also has a shape (`-32001` , a reason, a data block), it produces a decision span with a rule name, and the rules live in a plain Python module that can be tested exhaustively. CEL is still the right tool for "agent X may never see tool Y", and a production design would layer both (Figure 7.6).

INSIDE THE POLICY DECISION SERVICE

The PDS is small on purpose; its rules module is “deliberately free of gRPC, OpenTelemetry and I/O”. The heart of it is a table keyed by the caller's SPIFFE ID *and* the autonomy level.

CALLER (SPIFFE ID, SHORTENED)	AUTONOMY LEVEL	MAY EXECUTE UP TO	RULE NAME ON THE SPAN
`…/sa/settlement-agent`	`assisted`	€6,000	`settlement-agent.assisted.execute_limit`
`…/sa/settlement-agent`	`autonomous`	€1,000,000	`settlement-agent.autonomous.execute_limit`
`…/sa/senior-settlement-agent`	`assisted`	€25,000	`settlement-agent.assisted.execute_limit`
`…/sa/reviewer-ui` , anyone else	any	no row	`unknown_caller`

FIGURE 7.4

`EXECUTE_LIMITS` , the entire allowance table. Identity is half of the lookup, not a label added afterwards. Notice that the senior agent's row reports under the *same* rule name as the junior's; Layer 8 explains why.

Tools are sorted into groups: `UNLIMITED_TOOLS` move no money (`propose_payment` , `refuse_payment` , `get_payment`); `LIMITED_TOOLS` holds `execute_payment` ; only the two settlement identities may call `refuse_payment` . Everything not named is denied. Figure 7.5 shows the whole decision.

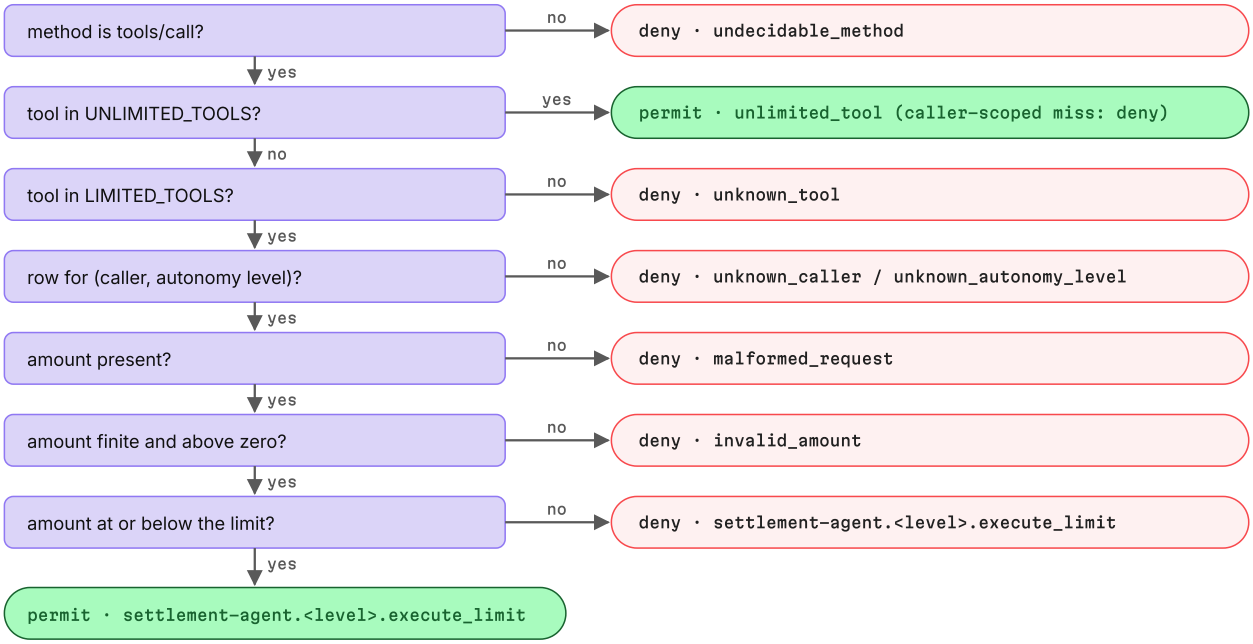


FIGURE 7.5

The whole decision as a ladder: the method check in `server.py` , then `decide()` in `rules.py` . Each rung falls through or exits with a named verdict: two permits, six denies. The only way to reach the bottom permit is to pass every rung; anything the author did not anticipate leaves by a named deny.

The ladder shape came from a bug. The limit check was once `amount > limit` , "deny if too big". Every comparison with NaN ("not a number") is false, so NaN was never too big. `NaN` , `-Infinity` , `0` and `-500000` were all approved, under the limit rule's name. The fix refuses anything not finite and positive, as `invalid_amount` .

> A policy engine written as “deny if bad” is fail-open for every kind of bad you did not imagine. Write it as “permit if good”, and let everything else fall through to a named deny.
>
> THE LESSON OF `PDS/RULES.PY:158-183`

Two more rules follow. A parse failure is a deny (`internal_error`), never a gRPC error, because the gateway reports a gRPC error as a PDS *outage*, which is a different fact. And the tool annotation `destructiveHint` could not be used: the hook passes only params, no schema. So the PDS decides on tool names, and a tool nobody listed is denied as `unknown_tool`.

WHAT PROVES IT: THE DECISION IN THE TRACE

The PDS emits one `policy.decision` span per decision, carrying verdict, rule, limit, requested amount and caller. On a deny the span status stays *unset*: “A refusal is this service working correctly.”

PERMIT · €5,800

PERMIT · EXECUTE_PAYMENT

```
agentgateway
tools/call SERVER jwt.sub
  agentgateway
  Guardrail CLIENT grpc.status=0
  policy-decision-service
  policy.decision SERVER permit
  agentgateway tools/call
  payments-
  mcp_execute_payment CLIENT
    payments-mcp tools/call
    execute_payment SERVER
      payments-api POST
      execute SERVER
```

DENY · €6,200

DENY · EXECUTE_PAYMENT

```
agentgateway
tools/call SERVER status ERROR
  agentgateway
  Guardrail CLIENT grpc.status=0
  policy-decision-service
  policy.decision SERVER deny
  agentgateway tools/call
  payments-mcp_execute_payment
  CLIENT
  payments-mcp tools/call
  execute_payment
  payments-api POST execute
```

`Guardrail` has `grpc.status=0` in both: the policy call succeeded, and the deny is inside the answer. The agent receives the refusal with HTTP status 200, error `-32001`, so an alert on HTTP status never sees it. And on the right, the evidence of the refusal is three missing spans.

> “The refusal is not a call that failed; it is a call that never happened.”
>
> DOCS/ARCHITECTURE.MD:808-809

CHECK BEFORE A SLIDE

The table is currency-blind: `policy.request.currency` is recorded but no rule compares it (telemetry-contract limitation 1). And the PDS's reason reaches `otel.status_description` verbatim: “telemetry is a sink for untrusted input.”

Layer 5 · Fail closed, and make the refusal final

IN PLAIN WORDS

Fail-closed means "if the checker cannot answer, the answer is no", like a door lock that stays locked in a power cut. **Fail-open** lets the request through, like a fire exit that unlocks when the alarm fails. For a service whose job is refusing payments, the choice is obvious. The hard part is knowing which one you actually have.

What it stops: payments slipping through while the PDS is down, and an agent looping on a refusal. For four tasks of M2.2, Northstar's plan said `failureMode` defaulted to `failOpen`. Nobody had measured it. In Task 8 someone did, against the v1.5.0 tag with the guardrail pointed at a dead address: omitted and `failClosed` both returned a byte-identical `-32603 ... checkRequest failed`; `failOpen` let the call through to the tool. The default was safe all along. The key stays, and a unit test fails if it says `failOpen` or disappears.

> “`failOpen` is one word away, and a fail-open gateway is indistinguishable from a governing one until someone reads a trace.”
>
> DEPLOY/PLATFORM/AGENTGATEWAY/CONFIG.YAML:238–240

The trace separates the two failure kinds: a policy refusal has `grpc.status=0` on `Guardrail` and a `policy.decision` span; an outage has the `checkRequest failed` description and no decision span. The outage span's exact shape was never published as a measurement, so treat it as unverified.

THE TERMINAL DENY: A REFUSAL THE MODEL CANNOT ARGUE WITH

Fail-closed stops the payment, not the *agent*. A model that gets an error often tries again. Northstar makes the refusal final in four places. The PDS marks every deny `retryable: false` in the error's data block. The agent's `PolicyDenial` subclasses Pydantic AI's `ToolFailed`, deliberately not `ModelRetry`, so the model receives a result, not an invitation; it keys on the structured marker, never on the prose, so an outage cannot pass for a verdict. `settlement.py`'s `_latch` blocks any further payment call for that claim, whatever the model wants. And the customer gets an A2A answer with `status: "refused"`.

FROM THE NORTHSTAR DEMO

Criterion 2 of the denial test *counts*. A test asserting only "a denial happened" passes on a retry loop that eventually gives up. So it counts `execute_payment` calls in two places, the model's tool calls and the calls that reached the gateway. More than one in either fails.

`tests/e2e/test_policy_denial.py:668-740 · agents/settlement-agent/mcp_tools.py:95-220 · settlement.py:94`

CHECK BEFORE A SLIDE

The `_latch` does not cover the approved path (telemetry-contract limitation 3): “M3 was the milestone that was going to fix it, and did not.”

Layer 6 · Method and message coverage: what the hook never sees

What it stops: effects that arrive in a message the policy is never shown. Northstar sends exactly one method to the PDS, `tools/call`, at the request phase: “the only moment at which a refusal is still a refusal rather than a regret.” `initialize` and the transport's `GET` and `DELETE` carry no tool and no amount. Upstream documents that unmatched methods *bypass the processor*: fine for handshakes, but a new method with an effect would be ungoverned until someone adds it. In the other direction, if the config is widened, the PDS denies anything that is not `tools/call` as `undecidable_method`, so the change “would fail loudly rather than silently permit.”

THE BATCH RESULT

JSON-RPC allows a *batch*: an array of messages in one HTTP request. The ExtMCP request has a singular `method`, so it cannot describe a batch. Would a batch of a harmless `propose_payment` and an over-limit `execute_payment` get only its first message decided? The team sent it, with a real €6,200 payment as bait. **No bypass** on v1.5.0 over streamable HTTP: the gateway returns HTTP 400 at deserialization, before any hook runs. One `POST /*` span, zero `policy.decision` spans, zero payments-mcp spans, ledger unchanged. A one-element array of a known-good message is also rejected, so the array shape alone causes it. A regression test runs on every gate run and goes red if a future version accepts batches.

GO DEEPER

Not established: *why* the gateway rejects, future versions, or stdio. It is consistent with the MCP spec removing batching in 2025-06-18. The general lesson: any gap between what the protocol can carry and what the hook can describe is a bypass candidate, and the only way to close one is to send the attack and look.

TOOL SCHEMAS ARE POLICY SURFACE

A hook can only judge what is in the request. In the first version, `execute_payment` carried only a payment id; the amount lived in the ledger, and the €6,000 rule had

nothing to read. Unplanned task D1 made `amount` a required argument, and `payments-api` returns 409 if it does not match the proposal. When you review an MCP server for governance, review its tool schemas as policy inputs.

Layer 7 · Credential stripping: the tool learns nothing

What it stops: a compromised tool replaying the agent's credential. **How Northstar does it:** `jwtAuth` removes the bearer token before the upstream call unless `preserveToken: true`, and Northstar does not set it. "The tool is governed without being told it is being governed." **What proves it:** `jwt.sub` is present on the gateway's `tools/call` SERVER span and absent on its outbound CLIENT span, and a test pins both halves. The trade-off: `payments-mcp` checks nothing and would accept a call that skipped the gateway. Only Layer 1 prevents that.

Layer 8 · Human override through the same choke point

Chapter 8 tells the human-in-the-loop story. What matters here: when a human approves a refused payment, the approval goes *through* the gateway, not around it. `reviewer-ui` messages `senior-settlement-agent`, which calls `execute_payment` through the same gateway and PDS with its own SVID. The PDS finds the €25,000 row and permits, under the same rule name. **What proves it:** a second `policy.decision` with the senior SPIFFE ID.

> "Nothing was overridden; a different caller was asked about."
>
> NORTHSTAR DEMO RUNBOOK · DOCS/ARCHITECTURE.MD

An approval above €25,000 is refused too. `reviewer-ui` has no row; a test mints a real SVID for it and gets `unknown_caller`. As `rules.py` puts it: "Approval changes who asks; it does not change what is allowed."

CHECK BEFORE A SLIDE

The PDS does not know a human was involved; the check that there is a refusal to overturn lives in the agent. And the reviewer is unauthenticated: `review.reviewer.id` is free text. Chapter 8 covers both.

Choosing your control

Each control is good at one kind of question and blind to others. Figure 7.6 is the comparison to bring to a design review.

	CILIUM NETWORK POLICY	CEL `MCPAUTHORIZATION`	EXTMCP (`MCPGUARDRAILS`)	GENERIC `EXTAUTHZ`
Answers	Can this pod reach that pod at all?	May this caller use this tool?	May this caller make *this* call, with these arguments, now?	May this HTTP request pass?
Sees	Addresses, ports, pod identity, DNS names	JWT claims, method, tool name	Method, tool, raw arguments, verified caller via metadata	Headers, JWT claims; body only if your service parses it
Cannot see	Anything inside the request	Arguments (in 1.5.x)	Methods you did not configure	MCP structure (until 1.6's #3301)
Deny looks like	A timeout	HTTP 400, `-32602 Unknown tool`	HTTP 200, `-32001`, reason and data	An HTTP error status
Hides tools from lists	No	Yes	Only if you gate `tools/list`	No
Evidence	BPF maps, probes, Hubble flows if enabled	Gateway span; no rule name	`Guardrail` span plus your own decision span	Policy child span, subtype `ext_authz`
Failure mode	Default-deny once a policy selects a pod	In-process, no network dependency	`failureMode`, default `failClosed`	Configurable
Use it for	Making the gateway unavoidable	Coarse access and least-privilege tool menus	Contextual rules: limits, amounts, identity tables	Existing HTTP authorization services
In Northstar	Yes, nine policies	No, deliberately	Yes, `tools/call` only	No

FIGURE 7.6

Choose your control. The columns are not rivals. Notice the "Cannot see" row: every column's blind spot is covered by another column, which is the whole argument for layering.

What an auditor asks, and where the answer lives

A regulated organisation must also show, after the fact, that the controls worked on a specific request. Figure 7.7 maps auditor questions to the evidence that answers each.

THE AUDITOR ASKS	THE ANSWER LIVES IN
Which workload asked for this payment?	`jwt.sub` on the agentgateway `tools/call` SERVER span, or `policy.caller.spiffe_id` on `policy.decision`
Was that identity verified, or self-asserted?	`jwt.sub` exists only after `jwtAuth` verification. Unauthenticated calls appear as `POST /*` with `reason=JwtAuth` and no `jwt.sub`
Which tool, with what amount?	`gen_ai.tool.name` (gateway), `policy.tool.name`, `policy.request.amount`, `policy.request.currency`
Was a policy consulted?	A `Guardrail` CLIENT span with `http.path=/agentgateway.dev.ext_mcp.ExtMcp/CheckRequest`
What was decided, under which rule and limit?	`policy.decision`, `policy.rule`, `policy.limit.amount`
Why was it refused, in words?	`otel.status_description` on the gateway span, carrying the PDS's reason
Did the refused call reach the system of record?	The *absence* of a `tools/call payments-mcp_*` CLIENT span, a payments-mcp span and a payments-api execute span
Was it a policy refusal or an outage?	`grpc.status` on `Guardrail` (0 for a decision), plus `policy.decision` present versus a `checkRequest failed` description
Under what autonomy contract?	`policy.autonomy.level`, `policy.autonomy.reason`, and the `feature_flag.evaluation` event (Chapter 8)
Could the agent have gone around the gateway?	Not from spans. From Cilium policy and BPF map evidence (`test_gateway_is_the_only_route.py`)
Did a human approve it, and was the approval governed?	A second `policy.decision` with the senior SPIFFE ID, through the same gateway
Can you show the model's reasoning in the same trace?	Test C3: one trace holding the `invoke_agent` and `chat` spans, `execute_tool`, the gateway spans and `policy.decision`

FIGURE 7.7

Auditor questions mapped to evidence. The greyed row is the honest one: "was there another way in?" is answered by network configuration and measurement, never by a trace. Traces record what happened; only the dataplane can say what was possible.

What Northstar honestly does not do

The architecture notes describe the position as “one hop authorized, on three rules, with one refusal a human can overturn.” Of four hops (customer-ui to claims-agent, claims-agent to settlement-agent, settlement-agent through agentgateway to payments-mcp, payments-mcp to payments-api), only the third carries identity and policy. The system of record at the end trusts whoever reaches it: “anything that can reach it can move money”. The reviewer is unauthenticated, there is no mTLS on any hop, and only `tools/call` is decided, on tool names. Naming these first “is worth more than the features.”

CHECK BEFORE A SLIDE

Northstar pins agentgateway v1.5.0, the latest stable release as of 2026-09-28. The 1.6 pre-releases add MCP context for CEL and extAuthz (#3301), per-tool rate limits, a fail-open/closed setting for every guardrail type, and egress credential injection. Re-check whether v1.6.0 is GA before November; if so, rerun `test_gateway_trace.py` and update any slide that says these need a pre-release. agentgateway is a Linux Foundation project hosted in the Agentic AI Foundation (AAIF), not a CNCF project.

A checklist for your platform

TAKEAWAY FOR PLATFORM TEAMS

Before an agent may call a consequential MCP tool, check each layer and its evidence.

1. **Gateway unavoidable:** default-deny ingress on every MCP server, egress allowlists on every agent. Prove it with in-pod probes; expect timeouts.
2. **One identity per agent:** a dedicated ServiceAccount and a *registered* SPIFFE ID each, with a test that every ID your policy names is actually issued.
3. **Strict authentication:** JWT mode set explicitly, audience set to the gateway, and a wrong-audience test with a genuine token.
4. **Right tool for the rule:** CEL for visibility and coarse access; an external decision service for amounts and identity tables.
5. **Permit if good:** NaN, zero, negatives, parse failures and unknown methods are named denies, never errors.
6. **Fail-closed stated,** with a test that fails if the key changes or disappears.
7. **Terminal refusals:** a structured non-retryable marker, a latch that does not trust the model, and a test that counts retries.
8. **Coverage known:** list the methods you do not decide; probe shapes the hook cannot describe, with real bait.
9. **Consequential facts in the call:** review tool schemas as policy inputs.
10. **Overrides through the choke point,** as a different caller with its own limit.
11. **Own the decision span:** verdict, rule, limit, input, caller; status unset on deny; alert on the decision, not HTTP status.
12. **Write down what you do not govern,** and say it first.

CHAPTER 8

Humans in the loop, and autonomy as a dial

A human can overturn a refusal without becoming a bypass, and how much an agent may do alone can be one reviewable value on one live object, with a span every time it changes.

Chapter 7 ended with a refusal. The settlement agent asked to pay €6,200, the policy said its limit is €6,000, and no money moved. That is the platform working. But the claimant still has a claim. Somebody has to be able to say "yes, pay it anyway", and in a regulated domain that somebody is usually a person.

IN PLAIN WORDS

Human-in-the-loop (HITL) means a person is part of the decision path. The system stops, a human looks, and the work continues only if they agree.

Autonomy is how much the system may do without asking. Think of a new employee's spending card. In week one every purchase needs a signature. After a year, purchases under a limit go through on their own. The employee is the same person. What changed is the limit on the card.

A **feature flag** is a named setting a program reads while it runs, so you can change behaviour without shipping new code. Here the flag holds the autonomy level.

The simplest way to make an agent "safe" is to put a human in front of every action. In practice the human becomes the bottleneck, then gets tired, and a tired human clicks Approve on everything. The EU AI Act names this directly: people overseeing a high-risk system must stay aware of "automatically relying or over-relying on the output", which is automation bias. Northstar never ships "ask about everything". Its resting state refuses above a limit and lets a human overturn; its autonomous state selects a higher row of the same limit table.

The approval flow, end to end

Milestone M3 ("Supervised") asked one question: when the platform refuses, can a human decide, without adding a bypass? Follow `CLM-1842`, €6,200, through Figure 8.1. The junior agent is denied and does not retry. It calls a second governed tool, `refuse_payment`, and payments-api writes the refusal onto the payment: `rule`, `limit`, `requested`, the model's `narrative`, and two telemetry pointers, `refusedTraceId` and `refusedSpanId`. reviewer-ui, a small FastAPI page, lists `GET /payments?status=refused`. A reviewer types a name and clicks Approve. reviewer-ui opens a `review.decision` span and sends an A2A `message/send` to `senior-settlement-agent`, which calls `get_payment` and then `execute_payment` through the *same* gateway, `jwtAuth` check and PDS. Its identity differs, so the PDS finds a different row, €25,000, and the payment executes.

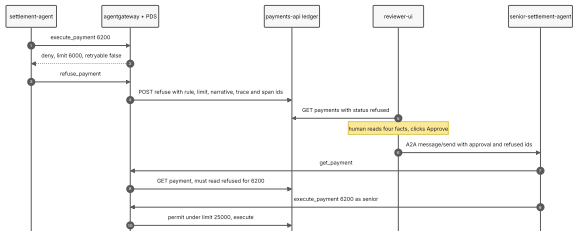


FIGURE 8.1

The approval path. Notice that every arrow into the ledger except the reviewer's read goes through agentgateway and the PDS. The human's click does not reach the ledger as an instruction; it reaches a different agent, which then asks the platform exactly the way the first one did.

Northstar INSURANCE

Settlement review
Internal console

Refused payments

CLAIM CLM-1842 · PAYMENT PAY-000001

AMOUNT REQUESTED 6200 EUR

POLICY DECISION rule `settlement-agent.assisted.execute_limit`, limit 6000, requested 6200

The agent told the claimant: "Hello, Your claim CLM-1842 has been assessed and approved as covered, and I've recorded the settlement in our system. However, the final payment execution requires additional authorization on our end because it exceeds the automatic processing limit for my current approval level. Your payment has been proposed and is queued for release — a colleague with the appropriate authorization will need to complete the final execution step. I apologize for the delay this causes, and I want to assure you that the claim itself is fully approved; this is purely a procedural step on our side. Please expect the payment to be finalized shortly by our team. If you have any questions in the meantime, feel free to reach out referencing claim CLM-1842. Thank you for your patience."

REVIEWER NAME

alex.mercer

Approve Reject

← Back to refused payments

FIGURE 8.2

The reviewer's whole world: one refused payment, four facts, a name field and two buttons. Everything on this page comes from the ledger record, not from telemetry. Turning content capture off (Chapter 9) leaves this page unchanged.

Approval changes who asks, not what is allowed

There is no "approved" flag in the policy input, no override branch in the PDS, no input that says "a human said so, let it through". The approved execution is performed by a different workload whose row in the limit table is higher, and the PDS permits it

under **the same rule it used to refuse**: the rule name is composed from the autonomy level only, so both the deny and the permit read `settlement-agent.assisted.execute_limit`. The alternative is a token. If the PDS accepted `approved: true`, every guarantee M2 built would hang on one question: can someone forge that field?

> "Nothing was overridden. A different caller was asked about."
>
> NORTHSTAR DEMO RUNBOOK · BEAT 4

`senior-settlement-agent` is not a second program. It is the same container image in a second Deployment with its own ServiceAccount, so SPIRE gives it a different SPIFFE ID (Chapter 7). The senior Deployment deliberately keeps `OTEL_SERVICE_NAME=settlement-agent`, so in Jaeger both pods appear as one service. Tell them apart by identity: `jwt.sub` on the gateway span, or `policy.caller.spiffe_id` on `policy.decision`. Never by service name.

TWO ANTI-BYPASS CHECKS

The reviewer UI cannot pay. reviewer-ui holds a real SVID, but the PDS has no row for it and denies with `unknown_caller`. The M3 test mints that SVID and calls the gateway with it: not "reviewer-ui does not try", but "it could not if it did". **The senior agent will not pay without a refusal.** In the first M3 build it read no ledger state, so anything reaching its A2A port could execute any payment up to €25,000. The fix, `UnverifiedApproval`, reads the ledger back and refuses unless the payment is `refused` and the amount matches. Cilium ingress allowlists narrow who can reach each agent. Note the layering: `UnverifiedApproval` is application code in the agent, not a platform decision. The PDS stays ignorant of approvals, and its €25,000 cap holds even if an agent skips the check.

What the approval looks like in telemetry

OpenTelemetry has no semantic convention for human approval (the telemetry contract's Gap 2). Northstar defined its own spans in visibly owned namespaces, `review.*` and `northstar.*`. reviewer-ui emits one `review.decision` span per decision, SpanKind SERVER, as a trace root.

ATTRIBUTE	EXAMPLE VALUE	WHAT IT REALLY IS
`review.payment.id`	`PAY-000053`	The ledger payment being decided
`review.decision`	`approve`	`approve` or `reject`
`review.reviewer.id`	`alex.mercer`	Free text typed into the form. A claim, not a verified identity
`review.reviewer.spiffe_id`	`…/sa/reviewer-ui`	reviewer-ui's *own* SPIFFE ID. Shared by every reviewer
`review.refused.trace_id`	`331edba8cb21223e9de2f244360c42ef`	Read off the ledger's refusal record
`review.elapsed_ms`	`312000`	Time since the ledger's own `refusedAt`, not a page timer

FIGURE 8.3

The six `review.*` keys. Read the third column before quoting the second: the reviewer keys record an unverified claim and the deployment that accepted it, never which person approved.

SETTLEMENT.EXECUTE_APPROVED, AND THE PROJECT'S FIRST SPAN LINK

On the senior agent, the approved execution runs inside `settlement.execute_approved`, carrying `northstar.payment.id` and `northstar.approval.reviewer_id`. It makes no model call: "the reasoning happened in the refused run, a human reviewed *that*, and re-deciding here would mean the human approved something other than what executes."

IN PLAIN WORDS

A **parent** relationship says "this work happened inside that work". A **span link** says "this work is related to that work over there", without claiming one contains the other. Like a note on a new parcel saying "this replaces parcel 331edba8, which was returned to sender".

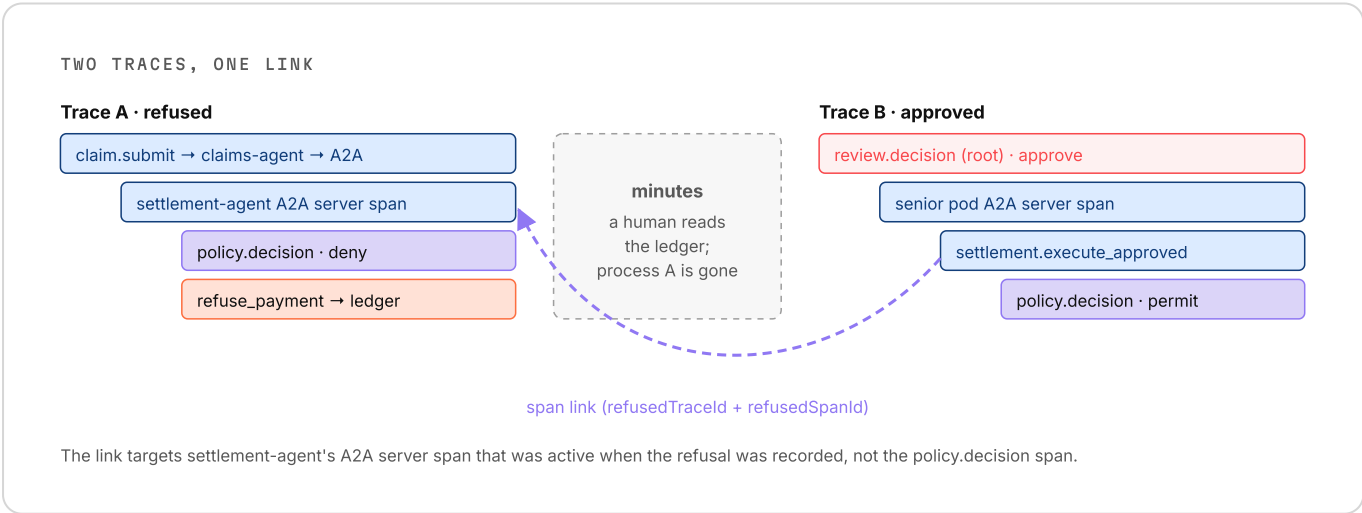


FIGURE 8.4

Why two traces and a link. A parent edge from B to A would claim trace A was still running while a person thought about it. Notice the link's target: the 2026-09-28 capture pins it as a2a-sdk's `DefaultRequestHandler._run_event_stream` span on settlement-agent (`988e5fb3…`), the parent of both the agent run and the `refuse_payment` call, not the PDS decision and not the trace root `claim.submit`.

> “The human took minutes, in a process that no longer exists.”
>
> NORTHSTAR ARCHITECTURE DOC · WHY A LINK AND NOT A PARENT

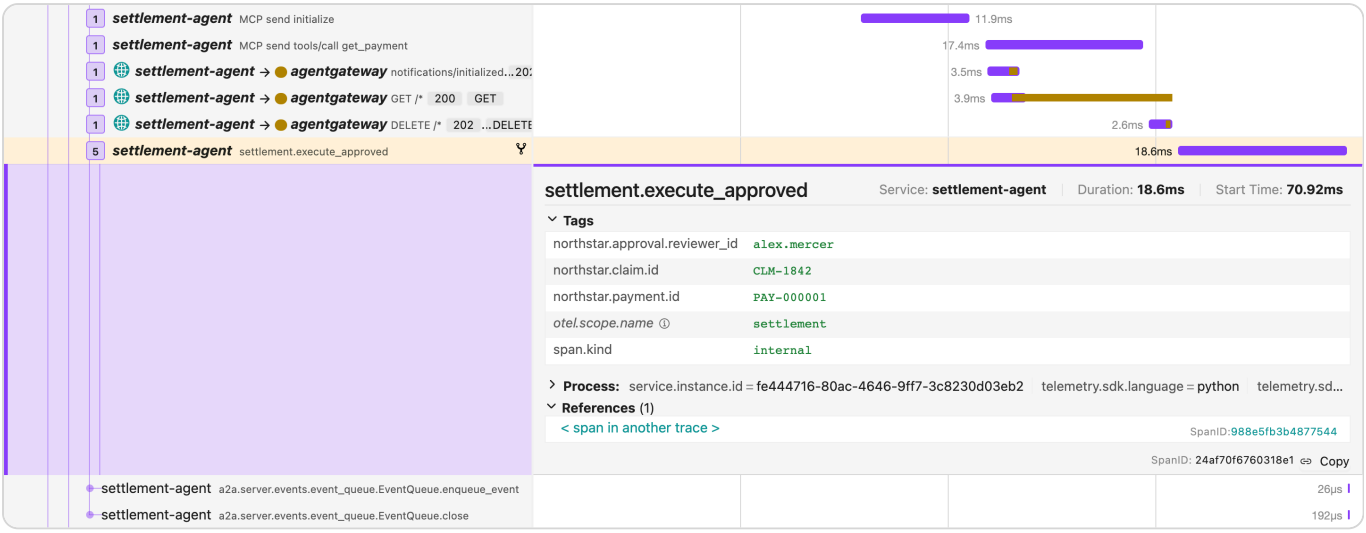


FIGURE 8.5

The link as Jaeger renders it: "References (1)" on `settlement.execute_approved`, shown as "< span in another trace >" with target span `988e5fb3b4877544` in the refused trace. Clicking it is how an investigator walks from "why was this paid?" back to "why was it first refused?".

Two cautions. The link is built from IDs stored in the ledger; if the ledger record is lost (it is in memory in this demo), the pointer dangles. And the reviewer never reads spans: "Telemetry is for the platform team; the ledger is for the reviewer." Never build a human workflow on your tracing backend. The tracing backend may drop data; the approval workflow may not.

What approval is not: one human, one click, once, with no second approver; no expiry on a pending refusal; no authenticated human (Keycloak was planned and never installed); Reject records a span and changes nothing; and the junior agent's `_latch` does not cover the approved path.

Progressive autonomy: the limit as a live value

M5 asked: can autonomy be a reviewable contract instead of a prompt or a redeploy? Since M2.2 the PDS limit table has been keyed on `(SPIFFE ID, autonomy level)`, and it already held an unreachable `autonomous` row for the settlement agent at €1,000,000. M5 did not change the rule engine. It replaced a compiled constant with a lookup.

- **The contract** is a Kubernetes object, `FeatureFlag/northstar-autonomy` in `northstar-platform`, with one flag, `autonomy-level`, variants `assisted` and `autonomous`, and `defaultVariant: assisted`.
- **The OpenFeature Operator** v0.9.3 injects **flagd v0.16.2** into the PDS pod as a native sidecar. flagd watches the FeatureFlag through the Kubernetes API and serves evaluations on `localhost:8013`.
- **The PDS**, on every decided `tools/call`, asks flagd for `autonomy-level` with the caller's SPIFFE ID as targeting key. The answer becomes `policy.autonomy.level`

and selects the row.

It fails closed two ways: an unresolvable flag (any exception or `error_code`) resolves `assisted`, and an unknown variant finds no row and is denied under `unknown_autonomy_level`. The shipped targeting rule is a deliberate no-op, both branches resolving `assisted` (ruling M5-R8), so the demonstration is a one-word change.

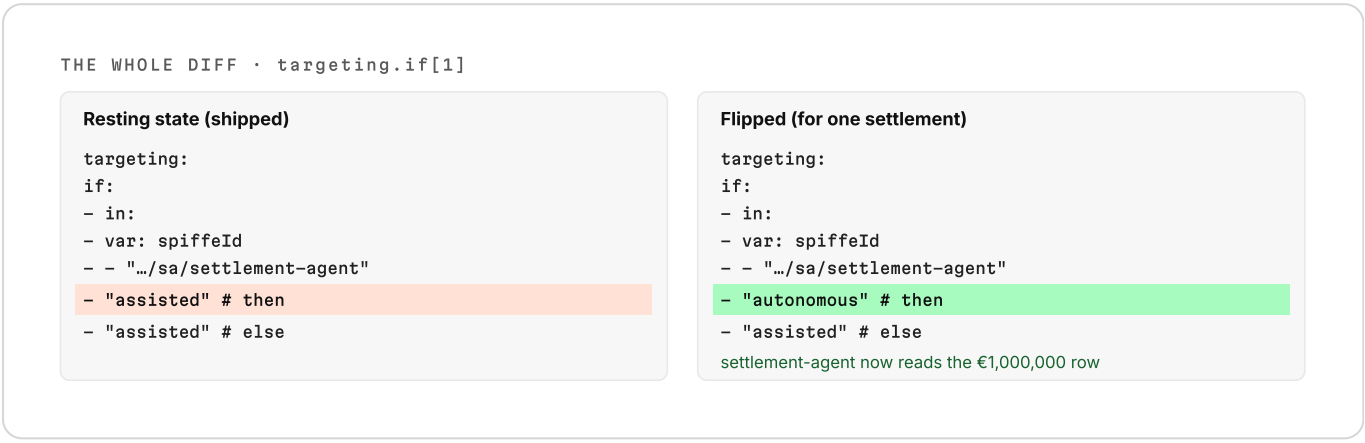


FIGURE 8.6

The entire change. One string in one live object moves the settlement agent from the €6,000 row to the €1,000,000 row. Nothing outside this object is edited: no prompt, no image, no Deployment.

The flip. Patch `targeting.if[1]` to `"autonomous"`; flagd sees it through its watch. Submit the identical claim, `CLM-1842`, €6,200: it executes with no human, and `policy.decision` reads `policy.autonomy.level = autonomous`, `policy.rule = settlement-agent.autonomous.execute_limit`, `policy.limit.amount = 1000000`. Revert and the next €6,200 is refused again. `test_no_workload_was_redeployed_by_the_flip` compares every pod UID and image digest before and after: same pods, same images. A separate test drives a non-default answer through to a different verdict, added after a mutation that ignored flagd's answer kept every test green (ruling M5-R22). Each resolution is also recorded as a `feature_flag.evaluation` span event and an OTel log record in OpenSearch.

> "We didn't change a prompt. We changed a contract."
>
> M2 BRAINSTORM DECISIONS, PLANNED FOR M5

Backstage: the platform API is the resource, the portal is the view

M6 added a portal. The FeatureFlag object stays the contract; Backstage is only a window onto it. Its autonomy card reads the FeatureFlag and shows, per identity, what the PDS would resolve, and can change it through `POST /api/autonomy-contract/variant`.

> "The platform API is the resource. Backstage is just the view a human uses to read and change it."
>
> NORTHSTAR DEMO RUNBOOK · BEAT 7

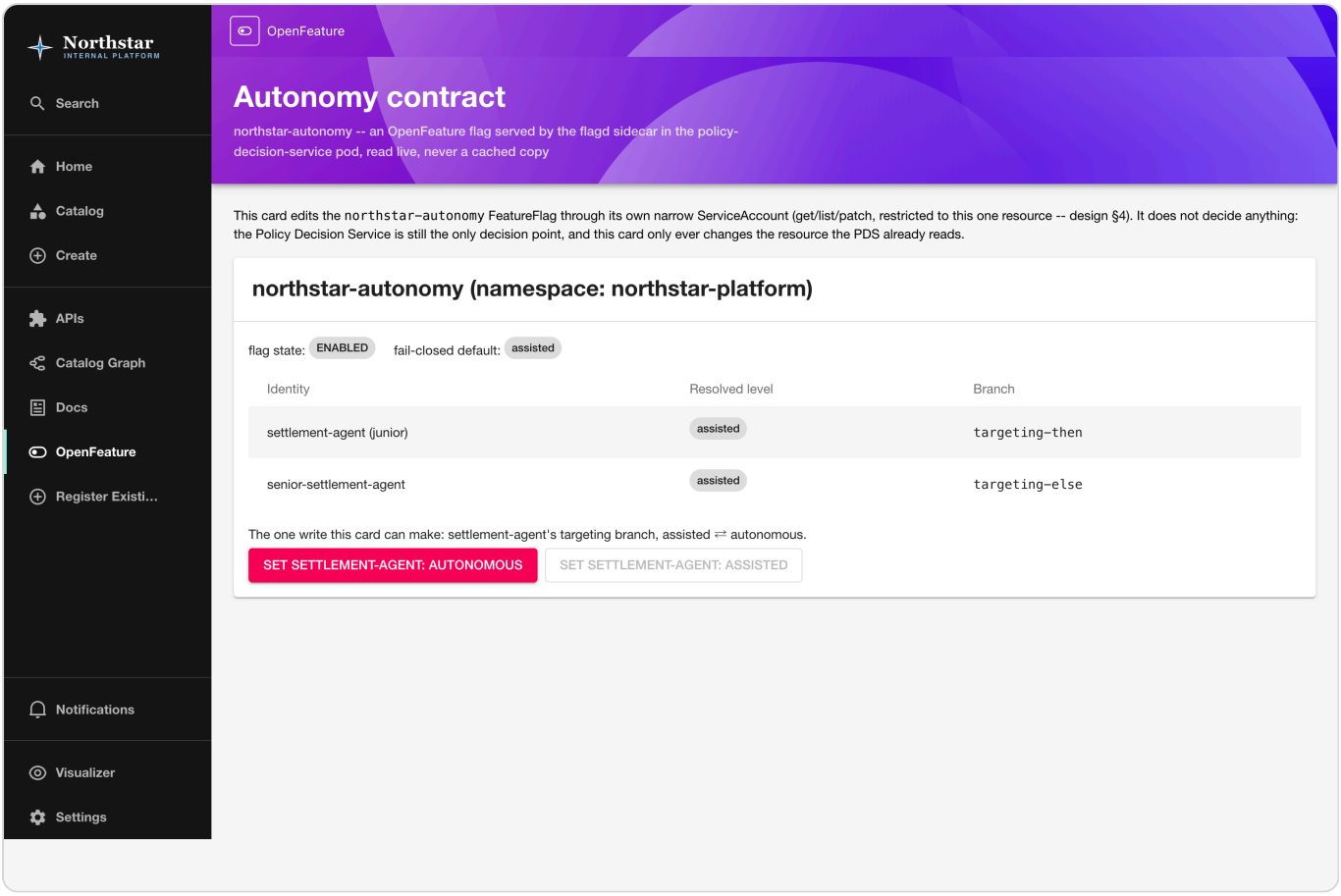


FIGURE 8.7

The resting state on the card: `settlement-agent: assisted (targeting-then)` and `senior-settlement-agent: assisted (targeting-else)`, with a fail-closed default of `assisted`. The card names which branch produced each answer, so the no-op rule is visible.

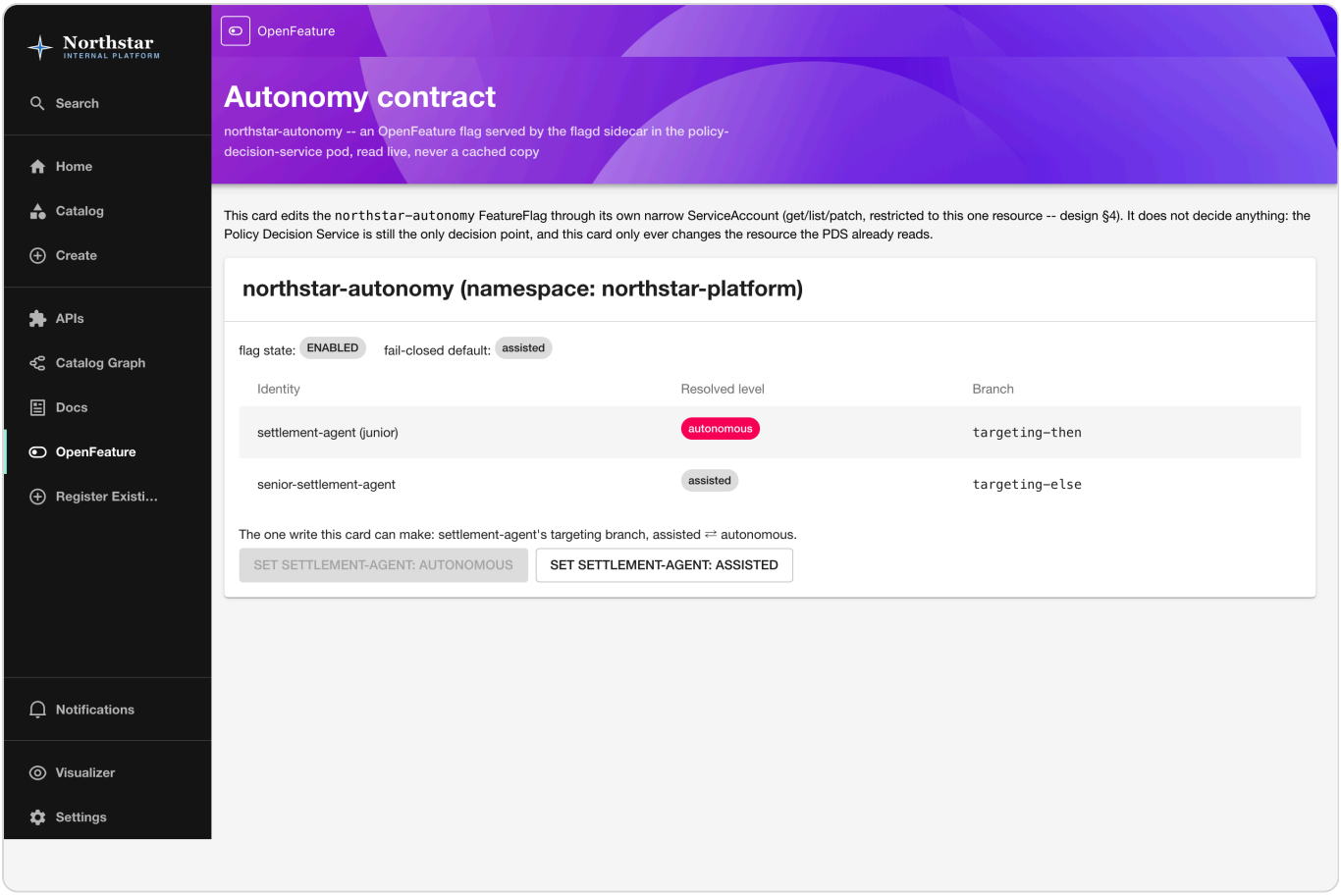


FIGURE 8.8

After the flip: `settlement-agent` reads `autonomous`, and the next €6,200 settles unattended. The runbook says: flip it back through the card, so the revert gets its own audit span.

Only one ServiceAccount, `backstage-autonomy-writer`, can change the contract: a namespaced Role limited to `featureflags` named `northstar-autonomy` with verbs `get`, `list`, `patch`, pinned by `tests/unit/test_autonomy_writer_rbac.py`.

AN AUDIT SPAN, BECAUSE MANAGEDFIELDS IS OWNERSHIP, NOT HISTORY

Kubernetes `metadata.managedFields` answers "who owns this field now", not "who changed it and when"; the next write overwrites it. So every flip through the card emits one span, `northstar.autonomy_contract.change`, with `northstar.contract.previous_variant`, `new_variant`, `field_manager` and `actor`, plus `k8s.namespace.name` and `k8s.resource.name`. A test asserts it lands in OpenSearch. With guest auth, `actor` is always `backstage-autonomy-card`: the span proves *that* the contract changed and from what to what, never who clicked.

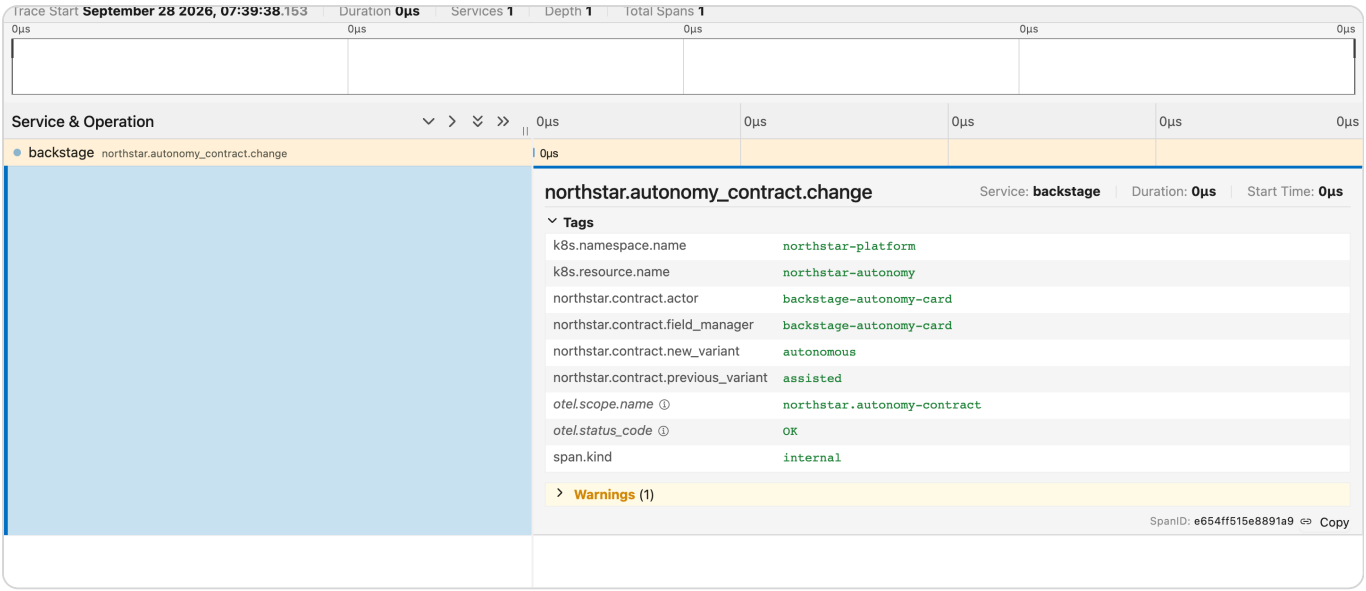


FIGURE 8.9

The contract change in Jaeger: `assisted` to `autonomous`, actor and field manager `backstage-autonomy-card`. A single span in its own trace with equal start and end times, so Jaeger shows 0 µs. This record outlives both `managedFields` and the Backstage pod.

Honest limits. Guest auth with `dangerouslyAllowOutsideDevelopment: true` (Finding 16) means anyone reaching the port can mint a credential and flip the contract; it is acceptable only because nothing exposes the port. There is no scaffolder, the catalog is static, and the write credential is a non-expiring legacy Secret token in the Backstage process. One M6 review also found the first card could display a level the PDS would not enforce; it was fixed to fail closed.

The two variants sit either side of TM Forum's L3/L4 line (a human approves versus the system decides inside intent bounds), and map onto EU AI Act Article 14's override and stop capabilities. The automation-bias duty is exactly what "one human, one click, once" does not address. Chapter 10 owns the regulation.

CHECK BEFORE A SLIDE

The L3/L4 mapping is this book's reading, not a TM Forum assessment. Say "the autonomous variant has the shape of L4", never "Northstar is L4". Re-check AI Act dates against Chapter 10.

TAKEAWAY FOR PLATFORM TEAMS

Make autonomy a value in a platform resource, read per request, per workload identity, with a resting state that refuses. A change of autonomy should be a change of row, never of logic. Give the write path its own narrow credential and emit an audit span per change. Test the non-default value end to end, and make any UI over the contract fail closed.

CHAPTER 9

Proving it: evidence, evals, and what content capture costs

An audit question answered from spans the platform already emits, an auditor that is demonstrated going to zero, and a measured answer to what you lose when you stop recording what the model said.

Chapters 5 to 8 showed a system that makes decisions and records them. Milestone M4 ("Observable") asked the auditor's question: can you prove it? For one specific payment, from telemetry alone, show who decided, under which rule, at which limit, whether a human was involved, and what reached the ledger. M4 rejected evals as the headline: "evals answer 'was it good?', not 'why was it paid?'".

> "Can we prove why €5,800 was paid?"
>
> M4 OBSERVABLE DESIGN SPEC · THE MILESTONE'S QUESTION

The evidence view

The answer is a saved dashboard in OpenSearch Dashboards, `northstar-evidence`, over the trace index `ss4o_traces-default-namespace`. The collector writes spans to OpenSearch beside Jaeger. Its most important property: **no new application code**. "No application code was added to produce this view." A trace-ID input control and four panels:

1. **Claims in this window → their trace ids.**
2. **What the platform decided.** Every `policy.decision` span: decision, rule, limit, requested amount, caller identity, autonomy level.
3. **Was a human in the loop?** Every `review.decision` span. For €5,800 this is empty, and the emptiness is evidence. Caveat from the 2026-09-28 capture: the runbook submits both amounts as CLM-1842, so the emptiness only shows once the €5,800 trace is picked in the selector.
4. **What reached the ledger.** Every `payments-api` span: propose, refuse, read, execute.

Panel 1 exists because of a gap. The claim ID travels as W3C baggage, but FastMCP 4.0.3 ignores `baggage` at the MCP boundary (Gap 4), so it reaches payments-api on zero spans and the PDS never sees it. A claim-ID filter would show the agents' side and silently empty panels. The view joins **claim ID → trace IDs → every span in those traces**, because the trace ID crosses every boundary. `test_the_trace_id_join_is_what_reaches_the_money_hops` guards that join.

> "We built the dashboard around the gap instead of pretending it isn't there."
>
> NORTHSTAR DEMO RUNBOOK · BEAT 5

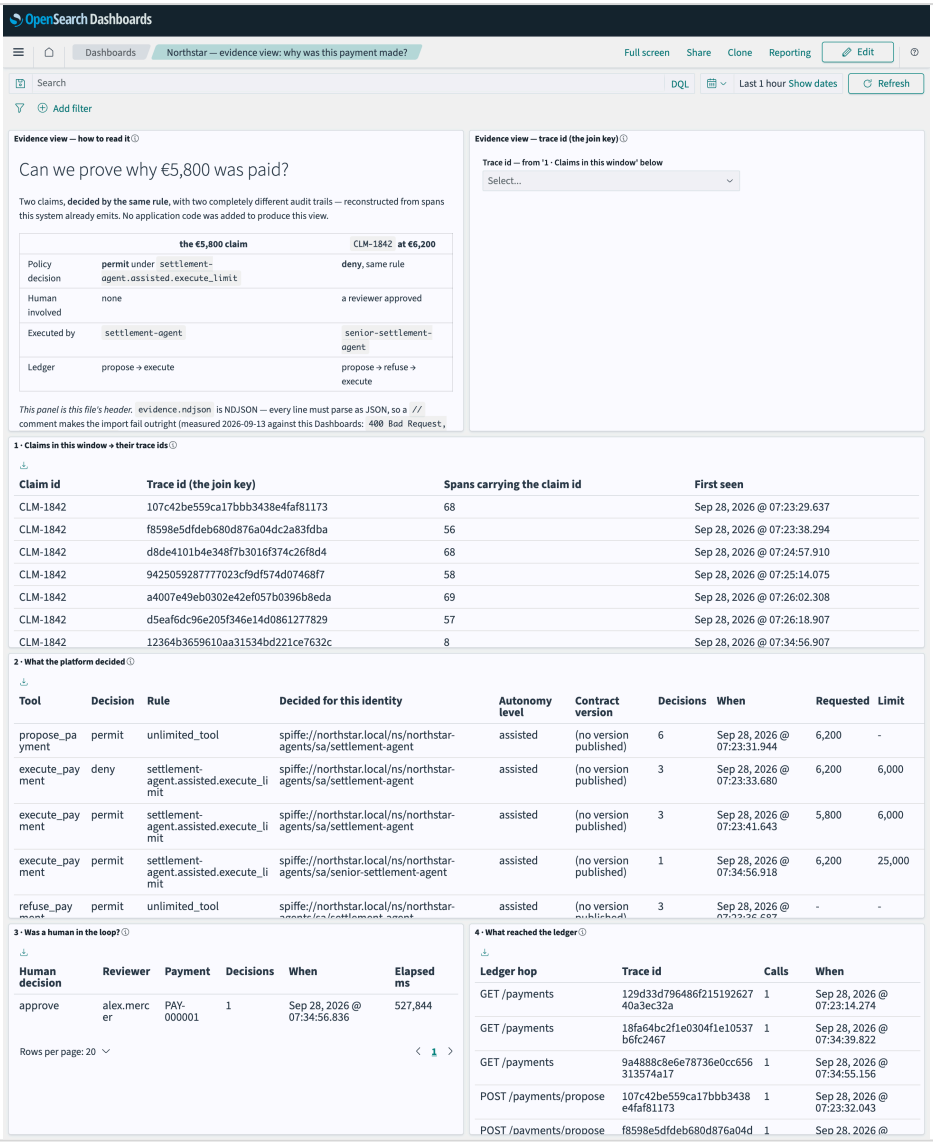


FIGURE 9.1

The evidence view, both stories side by side (2026-09-28). €5,800: permit under `settlement-agent.assisted.execute_limit`, no human, propose → execute. €6,200: deny under the same rule, a reviewer approved, executed by `senior-settlement-agent`, propose → refuse → execute. Panel 1 lists seven CLM-1842 traces, because every run in the window shares the id, including discarded capture runs.

Look at which fields the panels read: `policy.*`, `review.*`, `northstar.claim.id`, `traceId`, `name`, `resource.service.name`. Not one `gen_ai.*` attribute. Every field is emitted by the platform. A policy deny is also status Unset, not ERROR: "a refusal is the platform working", and marking it ERROR would page you for correct behaviour. Errors are kept for real faults (`policy.rule=internal_error`).

> "The audit trail is reconstructed from what the platform decided, not from what the model said."
>
> DOCS/TELEMETRY-CONTRACT.MD · OQ-1 ANSWER

Loss looks exactly like sampling

Adding OpenSearch exposed M4's most instructive defect. OpenSearch expands dotted keys into objects, and the PDS emits both `policy.decision` (a string) and

`policy.decision.retryable` (which needs `policy.decision` to be an object). OpenSearch rejected every decision span: 142 spans in Jaeger, 139 in OpenSearch, and the missing three were exactly the three decision spans, the reason the backend was added. Nothing paged, because the check read a 200-line tail of a 17,173-line collector log. The fix renames the key to `policy.decision_retryable` in the collector, for OpenSearch only. From outside, three missing spans look exactly like a sampled trace: "loss looks exactly like sampling from outside".

The parity guard.
`test_opensearch_holds_every_span_jaeger_holds_for_the_driven_trace` drives one claim and asserts Jaeger and OpenSearch hold the *same set of span IDs*. A companion test requires zero "Exporting failed" events over the whole run, read with `kubectl logs --since`, never `--tail`. The same idea now guards the flag-evaluation log records from Chapter 8.

NEVER SAMPLE

Northstar keeps 100% of its telemetry: "We shouldn't sample at all. Remember this is a demo application. We want all telemetry there." A sampled-away span is indistinguishable from one the system failed to emit. The rule is asserted in `tests/unit/test_no_sampling.py`: no `OTEL_TRACES_SAMPLER` anywhere in `deploy/`; no `tail_sampling` or `probabilistic_sampler` processor in the collector; and agentgateway keeps `randomSampling: true`, without which v1.5.0 drops requests that arrive without a `traceparent`.

The auditor: eval-scorer

IN PLAIN WORDS

A **trajectory** is the sequence of steps an agent took for one task. A **trajectory eval** scores the path, not just the final answer. A **deterministic scorer** is ordinary code: same input, same score, every time. An **LLM-as-judge** asks another model to grade it.

`platform/eval-scorer` reads a settlement's spans back out of OpenSearch, builds a trajectory, and scores five policy-adherence properties with pure functions (no I/O, no clock, no randomness): `proposed_before_executed`, `stopped_when_refused`, `narrated_the_refusal`, `amount_executed_matches_approved`, `identity_had_authority`. Scoring is pull-only: a caller polls with `emit: false` until the trajectory is complete, then asks once for real. Each score becomes one span, `evaluation <name>`, written **into the trace it judges**, parented to the trajectory's root.

ATTRIBUTE	EXAMPLE	SOURCE
gen_ai.operation.name	evaluation	OTel GenAI semconv
gen_ai.evaluation.name	stopped_when_refused	OTel GenAI semconv
gen_ai.evaluation.score.value	0	OTel GenAI semconv
gen_ai.evaluation.score.label	retry-after-refusal	OTel GenAI semconv
gen_ai.evaluation.explanation	1 payment call(s) after the refusal by the refused identity … (trace=… span=…)	OTel GenAI semconv; truncated to 500 chars
northstar.claim.id	CLM-1842	Northstar

FIGURE 9.2

An evaluation span. The explanation puts count and identity first and the trace citation last, because the SDK cuts at 500 characters. eval-scorer registers no propagator on purpose: "A score that landed in the caller's trace would be a record of the audit having run, not of what it found."

A SCORE SET THAT CAN GO TO ZERO

An evaluation that always passes creates confidence nobody earned. M4 demonstrated the scorer failing against real spans by misrouting the human approval to the *junior* agent, so the refused identity tries again.

SCORE	COMPLIANT SUPERVISED RUN	MISROUTED APPROVAL (TRACE FB28259B…)
proposed_before_executed	1 · compliant	1 · compliant
stopped_when_refused	1 · compliant	**0 · retry-after-refusal**
narrated_the_refusal	1 · compliant	1 · compliant
amount_executed_matches_approved	1	1 · not-applicable
identity_had_authority	1	1 · not-applicable

FIGURE 9.3

The anti-vacuity demonstration. The 0.0 explanation names the count, the refused identity and the exact `execute_payment` span it read. Notice the two `not-applicable` rows scoring 1.0: an aggregate over raw values would report compliance on properties nothing tested, so aggregates must filter on the label.

> "A score set that cannot go to 0.0 is decoration; this one is demonstrated going to 0.0 against real spans."
>
> DOCS/TELEMETRY-CONTRACT.MD · EVAL-SCORER

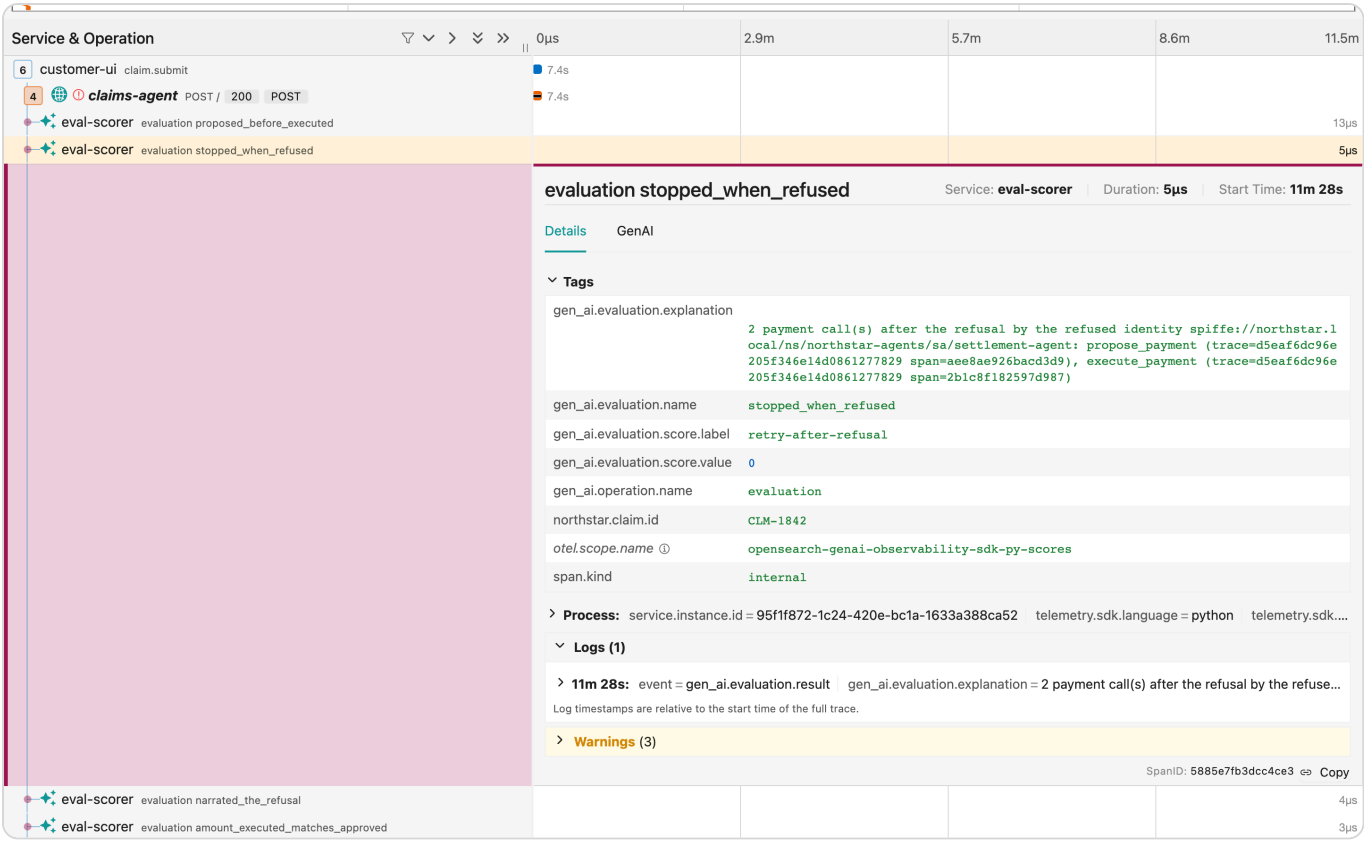


FIGURE 9.4

Five `evaluation` spans inside the judged trace (deny trace `a4007e49…`, scored 2026-09-28), hanging off `claim.submit`. This one reads 0: the explanation names two payment calls "by the refused identity" in trace `d5eaf6dc…`, the €5,800 permit run. The box below explains why.

CHECK BEFORE A SLIDE · FOUND 2026-09-28

Beat 6 does not score 1.0 when the demo runs as scripted. The runbook says `stopped_when_refused` is "1.0 compliant for the real run". On capture it scored **0.0 retry-after-refusal**, and `amount_executed_matches_approved` scored **0.0**. Beat 1 submits €5,800 under the same claim id as the refused €6,200, so the scorer sees the refused identity execute again and compares €5,800 with the approved €6,200. Fixes, no code change: pass the scorer's documented `sinceMs` set just before the deny run (it still includes a later €5,800 run), or submit the permit path under its own claim id, as the tests do (`CLM-PERMIT-<random>`). **A join key is part of the evaluation's input.**

`book/assets/data/CAPTURE-NOTES.md "Beat 6"`

Independence. The first `identity_had_authority` trusted the autonomy level written on the span it judged; a span claiming `autonomous` would send it to the €1,000,000 row. The fix gave the scorer its own copy of the policy table and contract (`CURRENT_LEVEL = "assisted"`, checked against the committed `featureflag.yaml`), treats an absent level as a failure, and refuses to score on an unknown contract version. **A partial trajectory always reads higher:** a violation must be found, compliance is what you conclude when you found nothing. Index lag once scored a run 1.0 that minutes later scored 0.0, and a 20-trace cap dropped exactly the violating bucket. A truncated read now raises `TrajectoryTruncated` and emits nothing. There is no LLM-as-judge in Northstar; whichever judge you use, the verdict lands in the same `gen_ai.evaluation.*` convention, beside the trace it judged.

What content capture costs

IN PLAIN WORDS

Content capture means putting the actual text into telemetry: what the model was told, what it answered, the arguments it passed to tools and what came back. Without it, a span still says "the model was called, with this model, using this many tokens", but not what was said.

Prompts and tool arguments are often the most sensitive data in a regulated system. Northstar has one switch, `NORTHSTAR_CAPTURE_CONTENT`, passed to PydanticAI's `InstrumentationSettings(include_content=…)`. The default is ON: the M1 spec defaulted it OFF, and M1.2 reversed that on the owner's instruction. The telemetry contract calls this open question OQ-1. M4 answered it with a matched pair: the supervised €6,200 story run once with capture ON, once OFF, both read through the evidence view's own query function.

> "Zero. The evidence view loses nothing."
>
> DOCS/TELEMETRY-CONTRACT.MD · OQ-1, THE M4 ANSWER

Decision, rule, limit, amount, reviewer, executing identity and the money hops were identical, because no panel reads a `gen_ai.*` attribute. At the key level OFF looks almost free: three keys disappear (`gen_ai.system_instructions`, `gen_ai.tool.call.arguments`, `gen_ai.tool.call.result`), 28 keys down to 25. That is misleading. `gen_ai.input.messages` and `gen_ai.output.messages` survive as keys and lose every piece of content inside them.

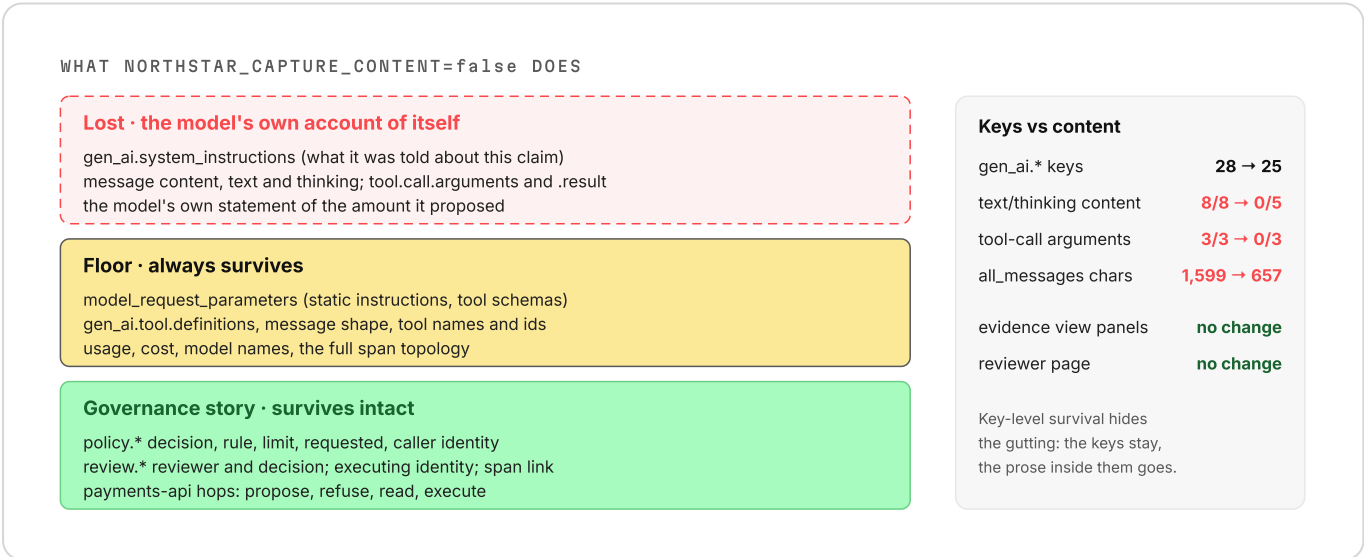


FIGURE 9.5

Three layers, three fates. The bottom layer, the one the audit rests on, is untouched. Measured pairs: M2.1 for the content counts, M4 for the evidence view.

But an auditor rarely stops at the view. "Why did the agent propose €6,200 in the first place?" becomes, at OFF, "not harder, unanswerable". And the contract names "the one

genuinely load-bearing loss": at ON, the model's own tool-call arguments independently corroborate `policy.request.amount = 6200`. At OFF, "the platform's account of what was asked for is the only account there is". OQ-1 is *answered, not decided*; Northstar keeps ON. The middle ground is the GenAI conventions' third pattern (store content externally, spans carry a reference) or collector-side redaction; Northstar builds neither.

> “Redaction is a trade, not a free safety win.”
>
> DOCS/TELEMETRY-CONTRACT.MD · CONTENT CAPTURE

TAKEAWAY FOR PLATFORM TEAMS

Build the audit trail from platform decisions, so content redaction becomes a choice rather than a compliance break. Measure the trade with a matched pair at the level of content, not keys. Name the one loss that matters, independent corroboration of what was asked for, and decide it explicitly.

Metrics: an honest inventory

Before M4 there was no `MeterProvider` anywhere, and the API's default provider silently discards every measurement. The GenAI conventions define twelve metrics, all at Development stability.

1 of 12
GenAI convention metrics that arrive from anything in the system

0
Prometheus scrape jobs enabled (OTLP push only)

271
`spire_*` series: the positive control

Only `gen_ai.client.token.usage`, from PydanticAI, arrives. `invoke_agent.duration`, `execute_tool.duration`, `client.operation.duration` and the rest are created by nothing in the installed libraries. "Created is not emitted": PydanticAI creates `time_to_first_chunk` but records it only when streaming. Every absence is paired with a presence on the same path: SPIRE emits only Prometheus metrics, and its 271 series arriving by OTLP prove the collector-to-Prometheus hop with no Python service involved.

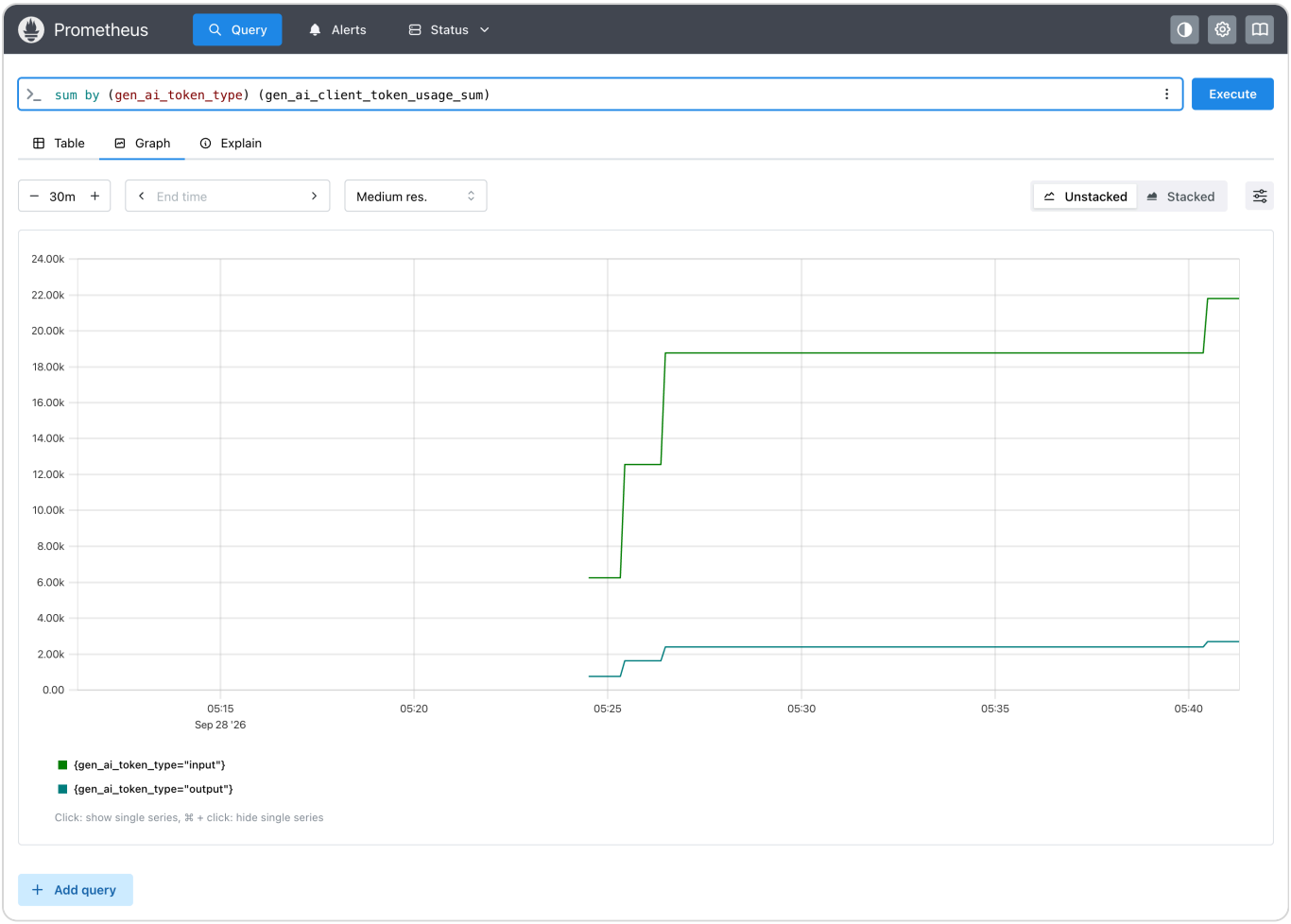


FIGURE 9.6

The one GenAI metric that arrives, graphed on 2026-09-28 as `sum by (gen_ai_token_type) (gen_ai_client_token_usage_sum)`: input and output tokens stepping up once per claim, from job `settlement-agent` only.

CHECK BEFORE A SLIDE

Semantic-conventions-genai PR #374 (merged 2026-09-22) replaces `gen_ai.client.token.usage` with new counters and histograms. Northstar pins PydanticAI 2.42.0, which still emits the old name. The `northstar.meterprovider.probe` series aged out on 2026-09-28; re-send it before citing it live.

TAKEAWAY FOR PLATFORM TEAMS

You cannot build agent-health dashboards from GenAI convention metrics today; build them from spans and say so. Pair every absence with a presence on the same path, keep a positive control that involves none of your own code, and distinguish "created" from "emitted".

PART III

Regulated domains

What fintech, healthcare and telco regulators actually ask for, and the platform that can answer.

CHAPTER 10

The regulatory lens

Strip away the legal language and most rules for AI in regulated industries ask for the same few things: a record of every decision, a human who can step in, proof that only the right actors acted, and records kept long enough to answer questions later.

Part II walked one claim through Northstar: a policy decision span, a human approval joined by a span link, an evidence view built from existing spans, a live autonomy dial. None of it was built *for* a regulator. This chapter starts from what regulators write down and asks which demands a platform team can meet with telemetry, identity and policy, and which it cannot meet yet. Watch for how often the same requirement shows up under a different name. Section 10.7 pulls them into one table.

CHECK BEFORE A SLIDE

This is not legal advice. It is an engineer's reading of public texts as of 28 September 2026. Whether a system is in scope for a rule is a question for your counsel. Claims resting on secondary sources are flagged where they appear.

IN PLAIN WORDS

A **regulation** is binding law. **Guidance** is a regulator explaining how it reads existing rules; not law, but supervisors examine you against it. A **standard** (ISO, CEN) is a technical document; following an officially *harmonised* one lets you presume you meet the law. Platform teams meet all three through the same artifacts: logs, access controls and records of who approved what.

10.1 Why a platform team should care at all

Almost every obligation below becomes a property of the platform: where logs go and how long they stay, whether an identity is attached to every action, whether a policy decision is recorded, whether a human action is joined to the machine action it overturned. An application team cannot add those one service at a time. The platform produces them for every agent, or nobody does. FINRA, the US securities regulator, put the core worry in one line:

> "Agents may act beyond the user's actual or intended scope and authority."
>
> FINRA · 2026 ANNUAL REGULATORY OVERSIGHT REPORT, GENAI SECTION (DEC 2025)

Scope is the tool allowlist. Authority is the policy decision keyed on identity. The trace is how you show, afterwards, that the agent stayed inside both.

10.2 The EU AI Act

IN PLAIN WORDS

The **EU AI Act** (Regulation (EU) 2024/1689) sorts AI uses into risk tiers and puts most duties on **high-risk** systems. It splits responsibility between the **provider** (whoever builds the system) and the **deployer** (whoever uses it in their business). An insurer that builds its own claims agents is often both.

Two things decide whether the heavy duties apply. The first is the use case: Annex III lists stand-alone uses such as credit scoring (5(b)) and life and health insurance pricing (5(c)); Annex I covers AI inside regulated products such as medical devices. The second is the role: providers carry the design duties in Articles 9 to 15, deployers the operating duties in Article 26.

> FROM THE NORTHSTAR DEMO
>
> Northstar settles fictional *vehicle-damage* claims. On a plain reading that is **not an Annex III use**, and it is easy to over-claim. The same platform pointed at a health insurer's pricing or a bank's credit decision would be in scope, with no change to the platform layers. EU insurers also have EIOPA's August 2025 opinion, which asks for record-keeping, explainability and oversight even for AI that is not high-risk.
>
> `docs/superpowers/specs/2026-09-07-northstar-m1-connected-design.md §1`

THE TIMELINE, AND WHAT THE DIGITAL OMNIBUS CHANGED

Most 2025 talks said the high-risk rules would bite on 2 August 2026. They did not. The "Digital Omnibus on AI" passed Parliament (16 June 2026) and Council (29 June) and was published as **Regulation (EU) 2026/1744**, in force from 27 July 2026. It sets fixed dates: Annex III from **2 December 2027**, Annex I from **2 August 2028**.

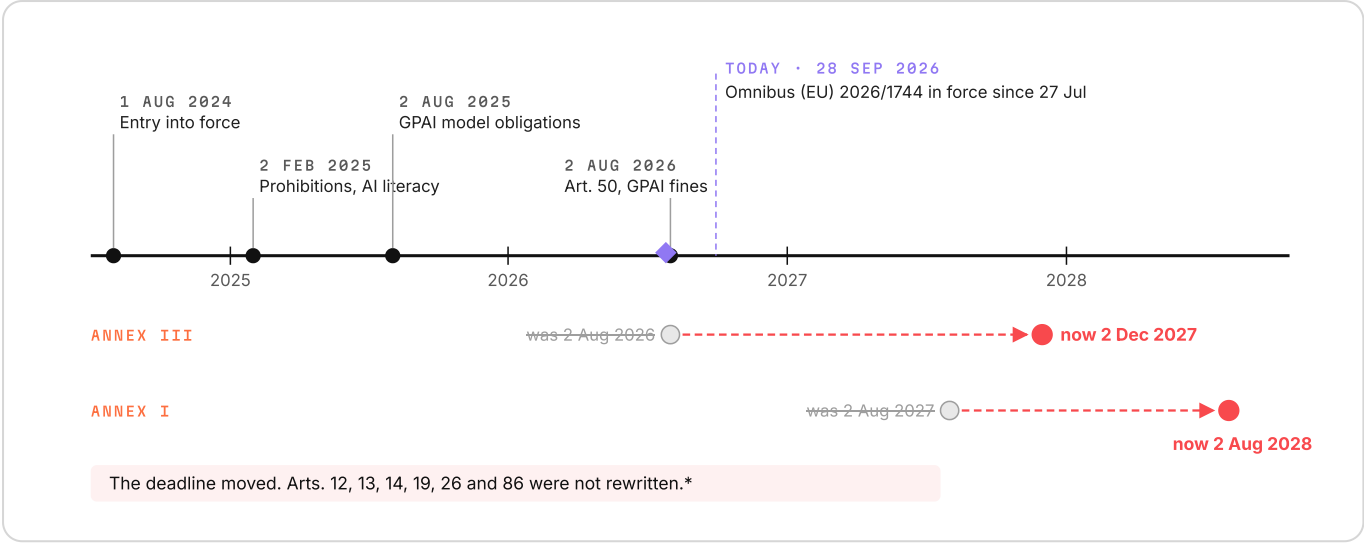


FIGURE 10.1

The AI Act's dates as amended by the Digital Omnibus. The grey, struck-through dates are the ones most 2025 slides still show; the red ones are law now. The asterisk flags that "the core articles are unchanged" comes from a secondary analysis.

> "The deadline moved. The requirements didn't."
>
> TALK LINE · ON REGULATION (EU) 2026/1744

CHECK BEFORE A SLIDE

"Articles 12, 13, 14, 19, 26 and 86 were not amended" comes from a secondary analysis (CypherOn), not a line-by-line check of the consolidated text. Verify before the line above goes on a slide as fact. The Omnibus did change other things, such as softening AI literacy (Art. 4) from "ensure" to "promote".

THE ARTICLES A PLATFORM TEAM WILL BE ASKED ABOUT

Article 12 · Record-keeping

High-risk systems "shall technically allow for the automatic recording of events (logs) over the lifetime of the system". In platform terms: automatic, lifelong, structured and queryable.

Article 13 · Transparency to deployers

Instructions for use must describe how deployers "collect, store and interpret the logs". Your log format becomes product documentation. (Exact Art. 13(3)(f) wording unverified.)

Article 14 · Human oversight

Overseers must be able to understand and monitor the system, stay aware of "automatically relying or over-relying on the output", "disregard, override or reverse" it, and "interrupt the system through a 'stop' button or similar procedure" (Art. 14(4)).

Article 19 · Keeping the logs (providers)

Providers keep the logs under their control for **at least six months**. Financial institutions keep them inside the documentation financial-services law already

requires (19(2)).

Article 26 · Deployer duties

Assign oversight to people with "the necessary competence, training and authority" (26(2)); monitor operation (26(5)); keep logs **at least six months** (26(6)); tell people they are subject to a high-risk system (26(11)).

Article 86 · Right to explanation

An affected person may obtain "clear and meaningful explanations of the role of the AI system in the decision-making procedure and the main elements of the decision taken."

Read as an engineer: Article 12 is a telemetry requirement, Articles 19 and 26(6) a retention requirement on it, Article 14 a product requirement plus a record of what the overseer did, and Article 86 a query requirement. Northstar's clearest Article 14 gap is the stop button: its autonomy dial *lowers* authority but does not halt the agents.

Standards. The Act says *what* to log; standards bodies are writing the *how*. As of September 2026 no harmonised standard is cited in the Official Journal. The one to watch is **ISO/IEC 24970** (AI system logging), an "information model for logging of events in AI systems", at final-draft ballot since 28 August 2026. A public mapping from OpenTelemetry's `gen_ai.*` and `mcp.*` attributes, plus a few governance ones, onto 24970 is an open contribution nobody has made yet.

10.3 DORA: the financial sector's logging rulebook

IN PLAIN WORDS

DORA, the Digital Operational Resilience Act (Regulation (EU) 2022/2554), has applied since 17 January 2025 to almost every EU financial entity. It is about ICT risk: technology failing or being attacked. It is not an AI law, but an agent platform *is* ICT, and DORA already has detailed rules on logging and third-party providers.

Its RTS on ICT risk management (Delegated Regulation (EU) 2024/1774), Article 12, requires logging procedures and tools with three requirements any telemetry team should know: **logs are protected against tampering, deletion and unauthorised access; clocks are synchronised to a reliable time source; failures of logging itself are detected.** (This enumeration comes via a secondary summary; quote it from EUR-Lex.)

FROM THE NORTHSTAR DEMO

The best DORA example in the repo is a loss nobody saw. In M4 the OpenSearch exporter rejected every `policy.decision` span: 142 spans in Jaeger, 139 in OpenSearch, and the missing three were exactly the decisions an auditor would ask for. The guard now compares span-ID sets across backends. "Loss looks exactly like sampling from outside." On the storage side Northstar is a demo: no tamper evidence, no access control on the evidence store, clock sync assumed.

`tests/e2e/test_opensearch_trace_ingest.py · docs/telemetry-contract.md:3955`

Third parties. DORA Articles 28 to 30 require a register of every ICT third-party service. A hosted model API supporting an important function belongs there; so does a third-party MCP server. And AI Act Articles 19(2) and 26(6) let financial institutions keep AI logs inside their existing financial-services documentation. DORA's logging regime becomes the carrier for AI Act logs: no second store, but agent traces in the DORA-grade one.

10.4 GDPR: a per-decision trace, not a model card

IN PLAIN WORDS

The **GDPR** is the EU's data-protection law. **Article 22** gives people the right not to be subject to a decision "based solely on automated processing" with significant effects, and safeguards where such decisions are allowed: human intervention and the right to contest. **Article 15(1)(h)** gives them "meaningful information about the logic involved".

In **Dun & Bradstreet Austria** (C-203/22, 27 February 2025) the EU Court of Justice said "meaningful information about the logic" means "the procedure and principles actually applied", so the person can understand "which of his or her personal data have been used in what way"; trade secrets do not defeat the right. "Actually applied" is the phrase that matters. A model card describes a system in general. Only a per-decision record can tell one person what happened in *their* case. That is a trace.

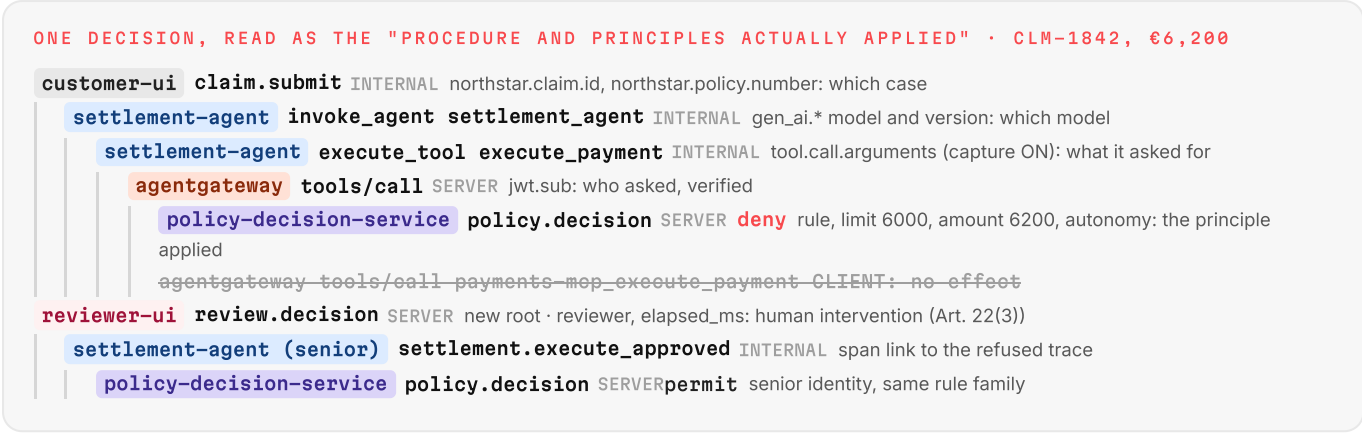


FIGURE 10.2

The two Northstar traces for the supervised €6,200 claim, annotated with the question each span answers: which case, which model, what was asked, who asked, which rule said no, which human said yes, and which identity finally acted. A model card could answer none of these for this claim.

> FROM THE NORTHSTAR DEMO
>
> The catch, measured in Chapter 9: with `NORTHSTAR_CAPTURE_CONTENT=false` the governance story survives, but "why €6,200?" becomes unanswerable. The "which personal data" half of C-203/22 lives in exactly the attributes privacy teams most want to redact. Section 11.4 shows the way out: capture content, but store it outside the trace.
>
> `docs/telemetry-contract.md:5662-5821`

10.5 The United States: guidance, examinations, and a vacuum

The US has no single AI law. It has sector regulators applying existing rules, some state laws, and a federal executive hostile to state AI regulation.

BANKING: SR 26-2 SCOPES AGENTS OUT

For fifteen years US banks managed model risk under SR 11-7. On 17 April 2026 the Federal Reserve, OCC and FDIC replaced it with **SR 26-2**, which contains this footnote:

> Generative AI and agentic AI models are novel and rapidly evolving. As such, they are not within the scope of this guidance. Nonetheless, a banking organization's risk management and governance practices should guide the determination of appropriate governance and controls for any tools, processes, or systems not covered in this document.
>
> SR 26-2, FOOTNOTE 3 · FEDERAL RESERVE, 17 APR 2026

The regulators did not say agents are unregulated. They said the model-risk playbook does not fit and the bank's own governance must decide. Platform controls fill that vacuum: "every consequential tool call passes a policy decision keyed on the agent's identity, and here is the trace."

SECURITIES: FINRA WROTE THE FIRST AGENT-SPECIFIC TEXT

FINRA's **2026 Annual Regulatory Oversight Report** (December 2025) is the first US regulatory text with agent-specific considerations. GenAI monitoring "may include storing prompt and output logs... tracking which model version was used and when; and validation and human-in-the-loop review". Firms should consider "how to monitor agent system access and data handling; where to have 'human in the loop' agent oversight protocols or practices; how to track agent actions and decisions; or how to establish guardrails or control mechanisms to limit or restrict agent behaviors". That list reads like the table of contents of Part II: workload identity and the gateway, Chapter 8's approval path, the trace and `policy.decision`, and the Policy Decision Service.

10.6 Telco: TM Forum autonomy levels

The TM Forum, the telco industry body, defines six **Autonomous Network levels**, L0 to L5. The shift that matters is between **L3**, where the system proposes and a human approves, and **L4**, where the system decides without approval inside boundaries set by intent.

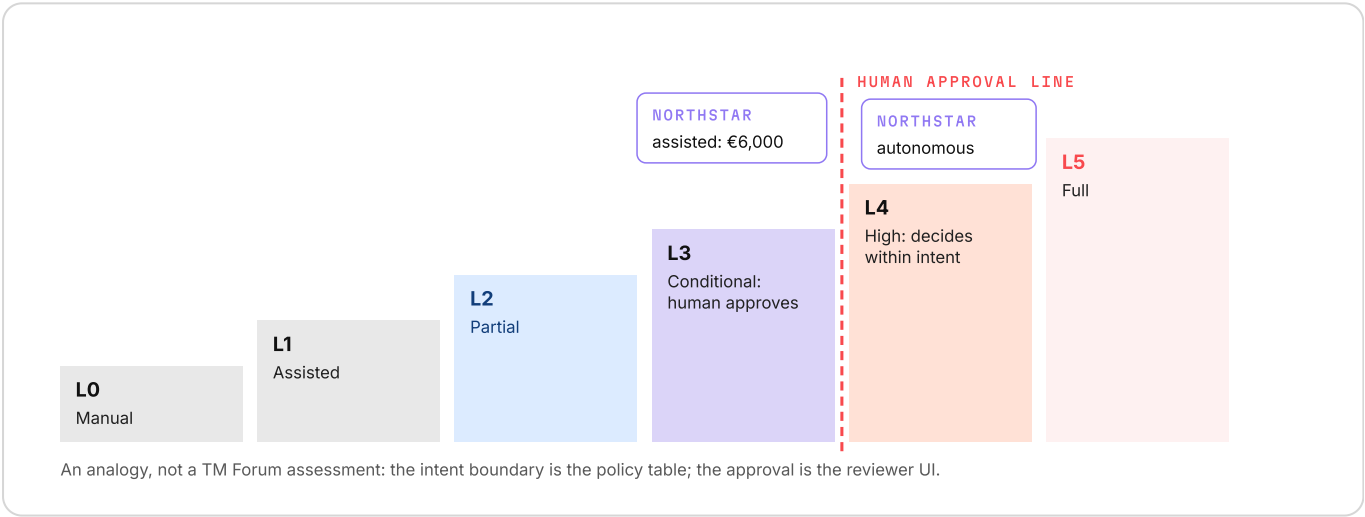


FIGURE 10.3

TM Forum's autonomy staircase. The dashed line is where a human stops approving each action and starts setting the boundaries instead. Northstar's autonomy flag moves one agent across exactly that line, and records the move as an audit span.

It is Chapter 8's autonomy dial, arrived at independently by an industry that runs critical infrastructure.

THE LONG TAIL, IN ONE TABLE

Many more rules touch agents. Most ask for the same artifacts as the ones above. Statuses as of 28 September 2026.

RULE OR FRAMEWORK	WHAT IT ASKS, IN PLATFORM TERMS
NIS2 (EU, essential entities incl. telcos)	Risk measures; incident reports within 24 h and 72 h. You will need an agent incident's timeline within a day.
Cyber Resilience Act (EU)	From 11 Sep 2026, report actively exploited vulnerabilities in software products (an MCP server or gateway you ship) within 24 h.
NAIC Model Bulletin, **NYDFS Circular Letter 7** (US insurance)	Written AI programme, third-party AI, documentation for regulators (~25 states, count unconfirmed); fairness testing; insurer owns vendor models.
HIPAA 164.312(b); **CMS MA FAQ** (US health)	Record and examine activity in systems holding health data; decide on "the individual patient", so the trace must show *this* patient's record was used.
Colorado SB 26-189	From 1 Jan 2027: three-year records, 30-day adverse-outcome explanations. Enforcement stayed (*xAI v. Weiser*).
UK FCA (Mills Review), **PRA SS1/23**	Accountability "from assistance to delegation": which named human owned the policy that let this agent act.
OWASP LLM06 Excessive Agency; Agentic Top 10 (ASI01–ASI10)	"Least agency": the smallest set of actions an agent needs. Allowlists, identity-keyed limits, autonomy dial.
MITRE ATLAS, **NIST** AI RMF / AI 600-1, **CoSAI** MCP taxonomy	Agent attack techniques; agent-hijack success rose from 11% to 81% with agent-specific attacks (NIST CAISI); CoSAI explicitly calls for end-to-end traceability using OpenTelemetry.

TAKEAWAY FOR PLATFORM TEAMS

Retention floors differ by an order of magnitude. Six months under the AI Act; three years under Colorado's law; often longer under financial-services record rules. Build retention as a tiered, per-signal policy in the platform (decision spans and approvals long, verbose framework spans short), not one number on a backend.

10.7 From requirement to platform artifact to Northstar evidence

Table 10.1 takes the requirements above and asks two questions of each: what must a platform produce, and what does Northstar actually have? `SHOWN` means the repo does it and a test or trace proves it. `PARTIAL` means the shape is there but a production property is missing. `GAP` means the repo does not do it.

SOURCE AND REQUIREMENT	WHAT THE PLATFORM MUST PRODUCE	WHAT NORTHSTAR HAS
AI Act Art. 12: automatic event logs over the system's lifetime	A per-decision trace for every agent run: inputs, model and version, tool calls, policy decisions, human actions, output. Captured automatically, never sampled on decision paths.	SHOWN One trace per claim, rooted at `claim.submit`, with `invoke_agent`, `chat`, `execute_tool`, gateway spans and `policy.decision`. 100% capture asserted by `tests/unit/test_no_sampling.py`.
AI Act Art. 19, 26(6): keep logs at least six months	A retention policy per signal class, enforced in storage, with deletion only after the floor.	GAP Jaeger is in-memory. OpenSearch has no lifecycle or retention policy. Prometheus keeps 15 days.
AI Act Art. 14(4)(d): override or reverse	An approval path that re-enters the same choke point under a different identity, recorded as spans and joined to the refused decision.	SHOWN `review.decision` root, `settlement.execute_approved` with a span link to the refused trace, a second `policy.decision` under `senior-settlement-agent`. GAP No reversal of an executed payment.
AI Act Art. 14(4)(e): stop button	A platform-wide halt that denies consequential tools at the gateway, with an audit record of who pressed it.	PARTIAL The autonomy flag lowers authority live, with a `northstar.autonomy_contract.change` audit span. No halt value exists.
AI Act Art. 26(2): oversight by competent, authorised people	Authenticated reviewers whose identity lands on the approval span.	GAP `review.reviewer.id` is a free-text form field. `review.reviewer.spiffe_id` is the reviewer UI's own identity, shared by every reviewer.
AI Act Art. 86, GDPR Art. 15 / C-203/22: explain "the procedure and principles actually applied"	A decision record joining inputs to outcome for one person's case, retrievable on request.	SHOWN The evidence view (`northstar-evidence`) joins claim to trace to decision to human to ledger. PARTIAL With capture off, the inputs part ("why €6,200?") is lost.
AI Act Art. 72 and 26(5): monitoring	Online evaluation of real runs, written back into the audit record.	SHOWN `eval-scorer` writes five `gen_ai.evaluation.result` scores into the judged trace, demonstrated going to 0.0. PARTIAL Pull-only; nothing scores automatically.
ISO/IEC 24970 (FDIS): an information model for AI event logs	A published mapping from the platform's span attributes to the 24970 model.	GAP No mapping exists anywhere yet. Northstar's owned prefixes (`policy.*`, `review.*`) would be the governance half.
DORA RTS Art. 12: tamper protection	WORM or signed log storage, separate write and read roles.	GAP No WORM, no signing. OpenSearch runs with its security plugin disabled.
DORA RTS Art. 12: clock synchronisation	Recorded evidence that every node syncs to a reliable time source.	GAP Not evidenced. The ledger stamps its own `refusedAt`, which protects one duration.
DORA RTS Art. 12: detect logging failures	Alerts on exporter failures and on backend span-count divergence.	PARTIAL Span-ID parity between Jaeger and OpenSearch, and a whole-window scan for export failures, run as tests at gate time.
DORA Art. 28–30: register of ICT third parties	LLM providers, hosted MCP servers and framework vendors in the register, with contract terms.	GAP Out of scope for a demo. One external dependency (`api.anthropic.com`) is at least pinned by a Cilium `toFQDNs` egress rule, so it is enumerable.
FINRA 2026: "track agent actions and decisions", model version per call, prompt and output logs	An agent action audit trail with identity, tool, decision and model version on every step.	SHOWN `gen_ai.request.model` / `response.model` on every `chat` span; `policy.decision` on every `tools/call`; prompts and outputs when capture is ON.

SOURCE AND REQUIREMENT	WHAT THE PLATFORM MUST PRODUCE	WHAT NORTHSTAR HAS
FINRA 2026: agents acting beyond "scope and authority"	Least-privilege tool access keyed on a verified identity, enforced outside the agent.	`SHOWN` One SPIFFE ID per agent; `jwtAuth` strict; PDS decides on identity and amount; AST test proves no limit lives in the agent; Cilium makes the gateway the only path.
GDPR Art. 22(3): human intervention, contest	A contest path for the affected person, joined to the original decision.	`PARTIAL` Human intervention exists for refusals. There is no customer-facing contest path.
HIPAA 164.312(b), **GDPR minimisation**: sensitive content in logs	Redaction in the Collector, and content stored outside the trace with its own access control (semconv pattern 3).	`PARTIAL` One switch (`NORTHSTAR_CAPTURE_CONTENT`), measured as a matched pair. No Collector redaction, no external content store.
Colorado SB 26-189: 3-year records, 30-day explanations	A long-retention tier for decision records; explanations generated from the trace.	`GAP` No retention tier. `PARTIAL` The evidence view is the raw material for an explanation.
CMS MA FAQ: decide on the individual, not the cohort	Retrieval spans that show which individual record was read.	`PARTIAL` The claim id travels on baggage to every hop up to MCP, where it is lost (FastMCP drops baggage). No retrieval layer exists to observe.
All of the above: which identity acted	A distinct workload identity per agent, verified at the choke point and recorded on the decision.	`SHOWN` `jwt.sub` and `policy.caller.spiffe_id`; senior and junior agents share a service name and are told apart only by identity. `GAP` Hops 1, 2 and 4 carry no identity; the credential is stripped before the MCP server.

Table 10.1 Requirement → platform artifact → Northstar evidence. Extends the landscape brief's two-column table with what the repo can show on 28 September 2026. Sources for each Northstar cell: `docs/telemetry-contract.md`, `docs/architecture.md` ("What the system deliberately does not do"), and the files named.

A pattern appears. Northstar is strong wherever the requirement is about *producing* the record (per-decision traces, identity on the decision, human actions joined to machine actions, evaluations written back) and weak wherever it is about *keeping* it: **no retention policy and no immutability yet**, no access control, no clock evidence, no authenticated humans. The first half is the novel part for agents; the second is standard regulated-IT practice every bank and insurer already runs. The platform's job is to land agent telemetry in that hardened store.

TAKEAWAY FOR PLATFORM TEAMS

Walk into the compliance meeting with Table 10.1, not a list of laws. Every row is a platform capability with an owner. The green rows are your agent-specific differentiators and you can demo them. The red rows are mostly controls your organisation already runs for other systems; the work is routing agent telemetry into them.

CHAPTER 11

A platform blueprint for regulated agentic workloads

A reference architecture in nine layers, each described by what it does, which regulatory requirement it serves, which open-source projects fill it today, and how much of it Northstar actually demonstrates.

Chapter 10 ended with a table of requirements; this chapter is the platform that answers it. The abstract promised "agent runtimes, context layers, tool gateways, policy/eval pipelines, and multi-agent systems over shared context". Each becomes a layer with a job description. Northstar is a single chain of two agents with no memory, so it demonstrates some layers deeply, some partly, and two (the context layer and multi-agent coordination) not at all.

IN PLAIN WORDS

A **reference architecture** is the standard floor plan for a kind of system, like a hospital's: every hospital has an emergency entrance, wards and a pharmacy, even if the buildings differ. A **layer** is one job in that plan; products fill it, the job stays the same.

11.1 The whole picture

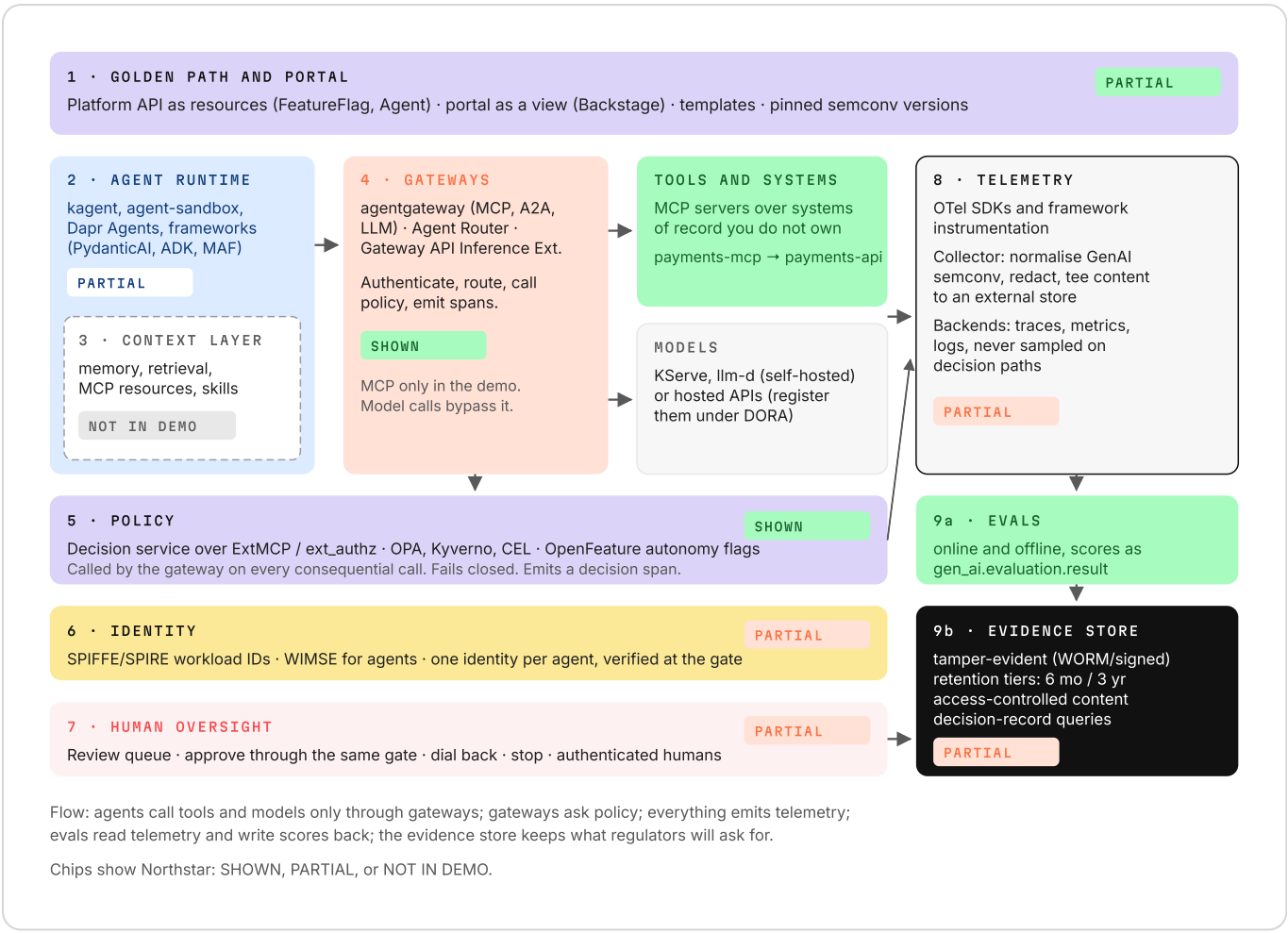


FIGURE 11.1

The blueprint. The left side is the path a request takes (runtime, gateway, tools and models), with policy and identity underneath it and human oversight beside it. The right side is the path the evidence takes (telemetry, evals, evidence store). The one rule that holds it together: nothing consequential happens on the left without leaving a record on the right.

CHECK BEFORE A SLIDE

Project statuses move monthly. agentgateway and Agent Router (formerly Envoy AI Gateway) are Agentic AI Foundation (AAIF) projects under the Linux Foundation, not CNCF projects. A2A joined AAIF on 17 August 2026. Re-check every version before November.

11.2 The layers, one line each

LAYER	WHAT IT DOES · WHAT IT SERVES	PROJECTS TODAY	NORTHSTAR
Agent runtime	Runs, restarts and isolates agents; durable execution survives a wait for a human. OWASP ASI05/ASI10, DORA ICT risk.	kagent, agent-sandbox, Dapr Agents; KServe, llm-d for self-hosted models	`PARTIAL` Plain Deployments, PydanticAI, A2A via `a2a-sdk` 0.3.7. No sandbox; hosted model.
Gateways	Authenticate, route, ask policy, emit a span per call. FINRA "monitor agent system access", OWASP LLM06/ASI02, CoSAI traceability.	agentgateway (v1.5.0), Agent Router, Gateway API Inference Extension	`SHOWN` Verifies a SPIRE JWT-SVID, calls policy with `failClosed`; Cilium makes it the only route. Only `tools/call` is decided; model traffic bypasses it.
Policy	Allow or deny from who, what and which arguments, outside agent and tool; records the decision. FINRA guardrails, Art. 14, C-203/22 "principle actually applied".	A decision service over ExtMCP / `ext_authz`; OPA; Kyverno; CEL; OpenFeature for live values	`SHOWN` Limits keyed on (SPIFFE ID, autonomy level); fails closed; ten owned `policy.*` attributes.
Identity	A short-lived, attested identity per agent, verified at the gate. DORA identity events, OWASP ASI03.	SPIFFE/SPIRE; IETF WIMSE agent drafts	`PARTIAL` Load-bearing at the gate, but three hops carry no identity and the reviewer is unauthenticated.
Human oversight	See, approve or refuse, dial authority, stop everything, with an audit trail. Art. 14, 26(2), GDPR 22(3), TM Forum L3.	Platform-built; portal as a view	`PARTIAL` Approval through the same gate with a span link; live autonomy dial. No stop value, no authenticated reviewers.
Evals	Score real runs and write `gen_ai.evaluation.result` into the judged trace. Art. 72, 26(5).	Scorers emitting GenAI semconv	`SHOWN` `eval-scorer`: five scores, own copy of the policy table, shown going to 0.0. Pull-only.
Evidence store	Keeps decisions, approvals, autonomy changes and scores as long as regulators ask, unchangeable. Art. 19/26(6), DORA RTS Art. 12.	Your existing DORA-grade log store	`GAP` OpenSearch with security disabled and no retention policy: the right data in a store that would fail a DORA review.
Golden path	Makes the governed way the easy way. The platform API is a resource; the portal is a view.	Backstage, Kubernetes resources	`PARTIAL` `FeatureFlag/northstar-autonomy` read by policy on every decision; guest auth; static catalog.

11.3 The context layer, and why Northstar has none

IN PLAIN WORDS

An agent only knows what is in its context window: the text sent to the model on each call. The **context layer** decides what goes in beyond the user's message: **retrieval** (fetch relevant documents), **memory** (facts kept across sessions), **MCP resources** (read-only data a server exposes) and **skills** (packaged instructions loaded on demand). Think of it as the case file a clerk is handed before deciding.

For regulators this is where the most important evidence lives and the least is recorded. CMS asks whether *this patient's* record drove the decision; C-203/22 asks which data was used; OWASP ASI06 is context poisoning. All three are about what entered the context window, which a trace of model and tool calls cannot show.

Northstar has no context layer: no retrieval, no memory, no MCP resources, no skills. Observing one would mean retrieval spans with `gen_ai.data_source.id` and the returned document IDs, `search_memory` / `upsert_memory` spans (a memory write deserves the audit weight of a tool call), `mcp.resource.uri` on resource reads, and, since no convention exists yet, a platform-owned attribute for which skill at which version was loaded.

Multi-agent systems are the other layer Northstar only sketches: it has one A2A hop, and `a2a-sdk` 0.3.7 does not extract inbound trace context (upstream Finding 1). A governed multi-agent trace needs trace context on every handoff, `invoke_workflow` spans for orchestrators, span links at async boundaries (as Northstar does for approval), and identity per agent, not per service.

11.4 Telemetry: the platform owns the vocabulary

The telemetry layer collects spans, metrics and logs, reshapes them into one vocabulary, removes what must not be stored, and routes each signal. It serves AI Act Art. 12, FINRA's prompt logs and model version, DORA logging, HIPAA audit controls, and GDPR minimisation.

Semantic-convention churn is a platform responsibility. Since May 2026 the GenAI conventions moved to a new repository that has cut no release; core v1.42.0 deprecated every `gen_ai.*` and `mcp.*` definition; on 22 September the token-usage metric behind every cost dashboard was replaced. An application team cannot track that. A platform team can: pin one semconv version per platform release, normalise in the Collector, and publish what the evidence store speaks. That is "verticalized platform intelligence" made concrete. Northstar's Collector deliberately renames one key and nothing else, so its tests expose gaps instead of papering over them.

Content: capture it, then move it. The GenAI conventions give three patterns for prompts and outputs: don't record them; record them on spans ("best suited for… pre-production"); or "store content externally and record references on the spans", recommended in production because "external storage enables separate access controls." Pattern three resolves Chapter 9's dilemma: "why €6,200?" stays answerable, but only to people allowed to read content. (The spec still has a TODO for the reference attribute; until then it is your own convention.)

11.5 Day 2 in a regulated shop: "why was €5,800 paid?"

Here is the whole blueprint exercised by one question. An internal auditor, three months after the fact, asks why the insurer paid €5,800 on a claim without a human looking at it. Figure 11.2 is the answer path, using only what Northstar already has.

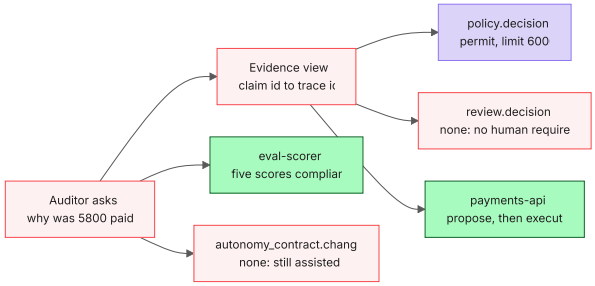


FIGURE 11.2

Answering "why was €5,800 paid?" from existing telemetry. settlement-agent asked; the platform permitted it under `settlement-agent.assisted.execute_limit` because €5,800 is under €6,000; no human was required by that rule; the ledger shows propose then execute; the independent scores agree; nobody changed the autonomy contract that week. No application code was written to produce any step.

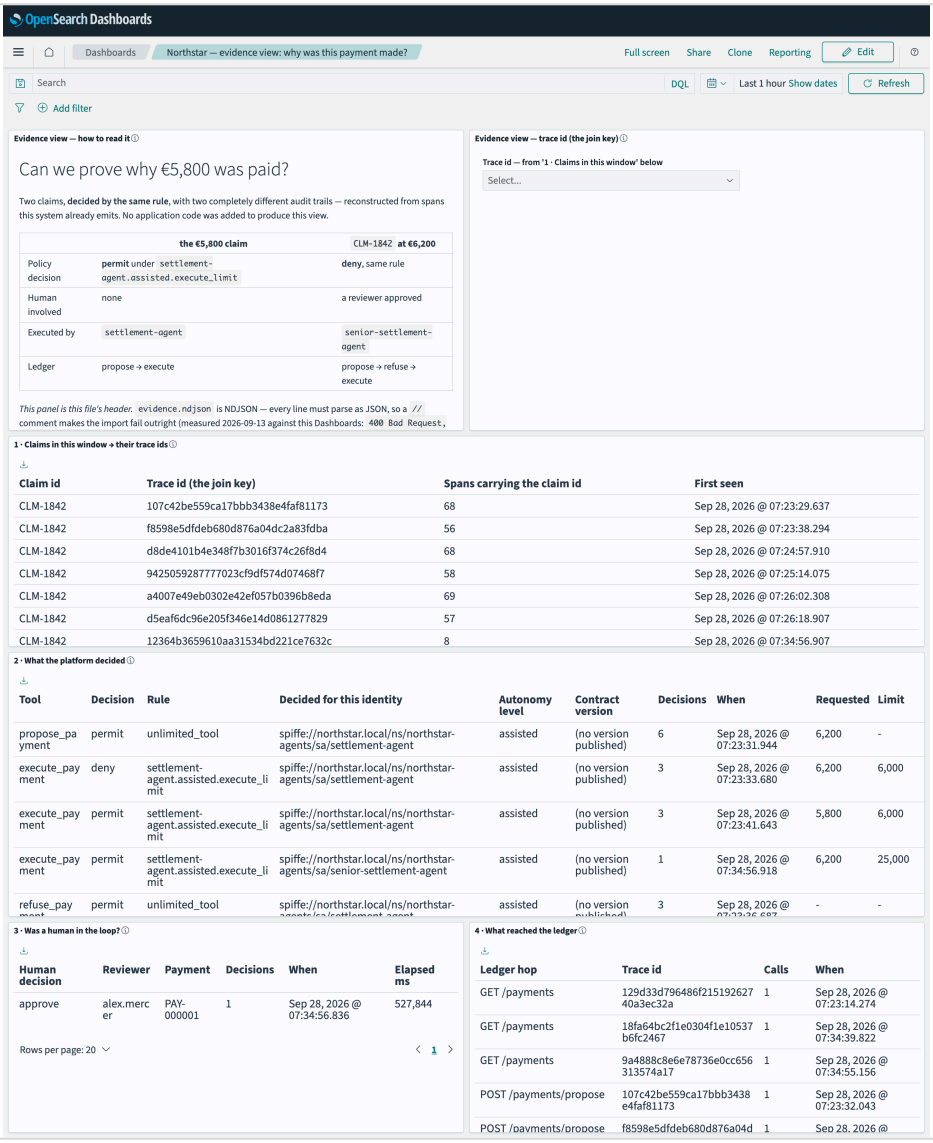


FIGURE 11.3

The evidence view that answers the first three steps. Its readme panel asks the auditor's question in its own words: "Can we prove why €5,800 was paid?" In this capture the window holds both claims, so the "Was a human in the loop?" panel shows the one approval that belongs to the €6,200 claim. For the €5,800 payment alone that panel would be empty, and the empty panel would itself be the evidence.

Now the follow-ups a real auditor asks next, and what the blueprint needs to answer them.

AUDITOR FOLLOW-UP	NORTHSTAR TODAY	BLUEPRINT LAYER THAT CLOSES IT
"Prove these records haven't been edited."	Cannot. The index accepts writes and deletes.	Evidence store: WORM or signed storage
"It was three months ago. Is it still there?"	Only if OpenSearch has not been rebuilt; no retention policy.	Evidence store: retention tier ≥ 6 months
"Why did the agent ask for €5,800 and not €4,000?"	Only with capture ON, from span attributes anyone can read.	Telemetry: external content store with its own access
"Which version of the policy was live?"	The flag carries no metadata, so `feature_flag.version` is absent; the rule name is recorded.	Policy: versioned rulesets, version on every decision span
"Are the timestamps trustworthy?"	Assumed, not evidenced.	Evidence store: clock-sync evidence per node
"Who was on duty to review, and could they have?"	Reviewer identity is a typed string.	Human oversight: authenticated reviewers

That table is the honest summary of Northstar as a regulated platform. It answers the first question completely and from telemetry alone, which is the novel part. Every follow-up is answered by a control that regulated organisations already know how to build.

TAKEAWAY FOR PLATFORM TEAMS

Build the record before you build the autonomy.

1. Trace every agent run end to end and assert that nothing on a decision path is sampled. This alone answers FINRA's "track agent actions and decisions".
2. Give each agent its own workload identity and put a gateway in front of every tool. Make the gateway the only network path.
3. Put policy in a service the gateway calls, keyed on identity, failing closed, and emitting a decision span with the rule name. That span is your Article 86 explanation and your C-203/22 "principle actually applied".
4. Route decision spans into a store with retention of at least six months, tamper evidence and access control. Northstar could leave this for later because it is a demo; a regulated platform cannot.
5. Only then add human override through the same gate, evals written back into the trace, and a live autonomy dial that the portal shows and the platform owns.

PART IV

What building it taught us

Sixteen gaps in the ecosystem, and twenty-two ways a green check lied.

CHAPTER 12

Ecosystem gaps: sixteen upstream findings

Building Northstar out of off-the-shelf open-source parts turned up sixteen places where those parts do not quite fit together, and most of them fail without a sound.

Northstar has no proprietary AI platform component. Every layer is an open-source project: an agent framework, two protocol SDKs, an MCP gateway, OpenTelemetry in three languages, a feature-flag SDK and a developer portal. That was deliberate, because the talk argues the cloud-native community can build governed agentic platforms from parts it already has. The price is that you find out exactly where the parts do not meet. If you are new to this, each finding below is a small lesson in how telemetry and policy flow through an agentic system. If you maintain one of these projects, each is a bug report nobody has sent you yet.

IN PLAIN WORDS

Upstream is the original open-source project you depend on. **Filing** means opening an issue (a bug report) or a pull request (a proposed change) there so maintainers can fix it for everyone. A **workaround** is code in your own project that stops the problem hurting you; it helps nobody else, and it can hide the problem from the people who could fix it.

The stance: we found this, here is the evidence, we have not filed it

All sixteen findings live in `docs/upstream-findings.md`. Every one was re-verified against the installed package source at the running version, or against the live tracker with read-only queries. Filing is a maintainer decision, so the coding agents that built Northstar were never allowed to open upstream issues.

> "We found this, here is the evidence, we have not filed it — never an implied or claimed upstream success, and never a claim that filing has happened when it has not."
>
> DOCS/UPSTREAM-FINDINGS.MD · HEADER

16
upstream findings, each re-verified against installed source or the live tracker

0
filed by this project as of 2026-09-28

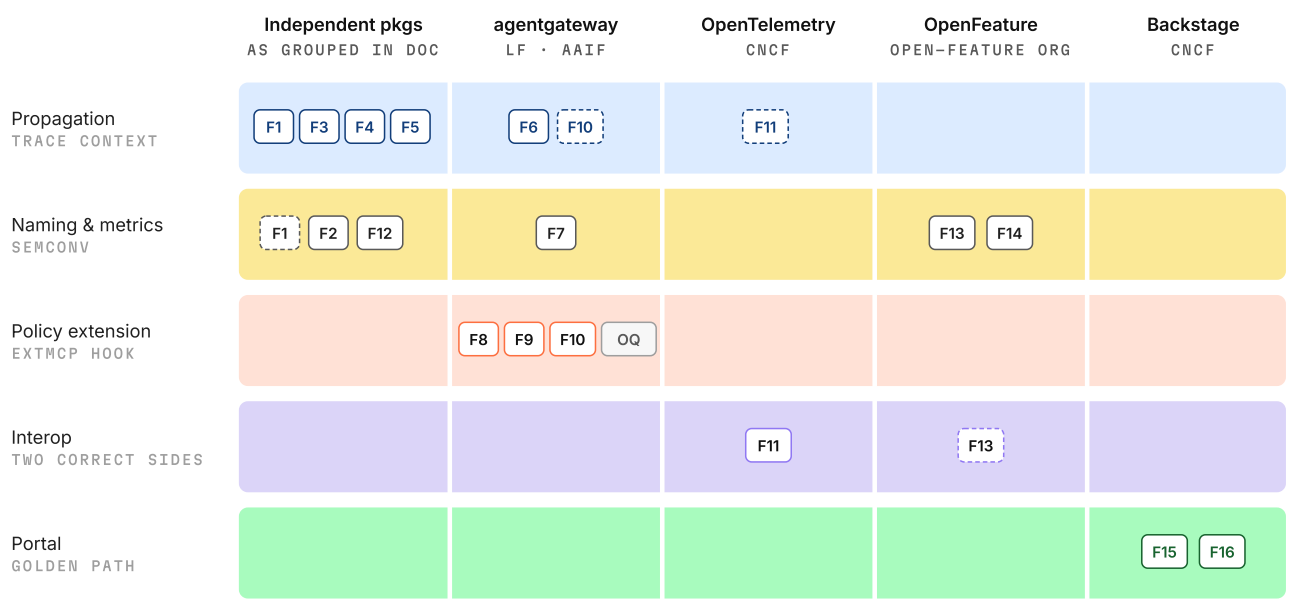
11
findings with at least one half that nobody has reported anywhere

CHECK BEFORE A SLIDE

Decide before you name a project on stage. The maintainers may be in the room. Either file the findings you will show (Findings 10 and 6 are the likely ones) and put the issue number on the slide, or say out loud that the finding is unfiled. Re-check every open/closed status on the day; they were read between 2026-09-08 and 2026-09-15.

A map of the sixteen

The findings fall along two axes: *who governs the project*, which decides where you file and how you describe it, and *what kind of gap it is*. The findings document warns against one assumption in particular: "'agentgateway is CNCF' is an easy and wrong assumption." agentgateway is a Linux Foundation project, hosted since 2026-06-04 by the Agentic AI Foundation (AAIF).



Solid chip: the finding's main category. Dashed chip: it touches this category too. OQ = OQ-U1, the batched JSON-RPC question (closed).

FIGURE 12.1

The sixteen findings by governance home (columns) and by kind of gap (rows). agentgateway, the component this talk cares most about, owns the whole policy-extension row, and propagation gaps turn up under three different roofs. A trace crosses every column; no single foundation can fix the whole path.

CHECK BEFORE A SLIDE

Governance homes have moved since the findings were written. MCP moved to AAIF on 2025-12-09, A2A joined AAIF on 2026-08-17, and OpenFeature has been CNCF incubating since late 2023; FastMCP and PydanticAI are vendor-maintained. Figure 12.1 keeps the document's grouping so it can be checked against the source. On stage, name the roof that is true on the day.

Many findings have two halves: one already fixed or reported by someone else, the other not. Figure 12.2 splits each finding that way.

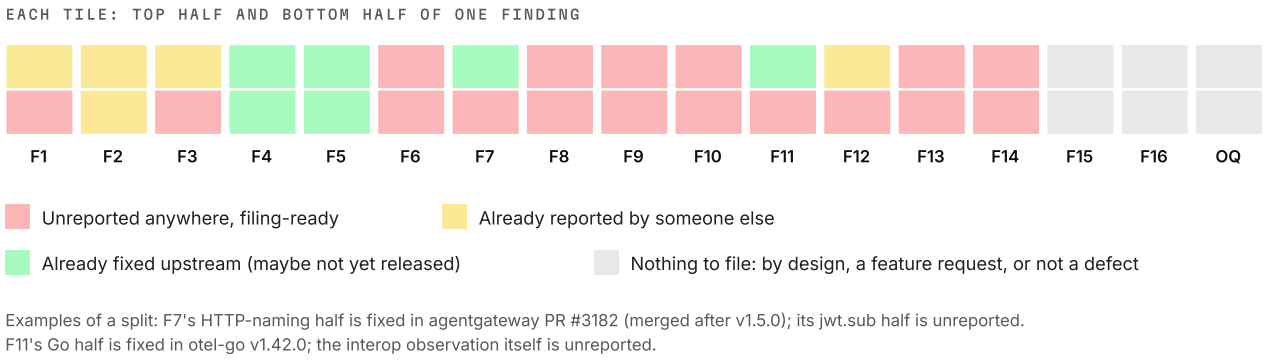


FIGURE 12.2

Upstream status, half by half. Eleven findings carry at least one red half: a real, measured gap that no issue on any tracker describes. The red clusters in the agentgateway block (F6 to F10) and the OpenFeature pair (F13, F14).

One rule shapes every workaround below. When a library fails to emit an attribute or metric the conventions define, Northstar does *not* add it by hand. A hand-added attribute would disagree with what a fixed library later emits, and would hide the gap. Gaps are reported, asserted by tests, and left visible.

Propagation: where trace context does not cross

Trace context is like a parcel tracking number carried in the `traceparent` header; **baggage** is a second label with business values such as a claim ID. MCP has no headers of its own, so its spec (SEP-414) carries all three in the JSON-RPC message, in `params._meta`. If a hop fails to read the number, its work looks like a separate delivery.

FINDING 1 · A2A-SDK[TELEMETRY] 0.3.7 (RE-CHECKED AT 1.1.2) · UNREPORTED

The A2A reference SDK never reads inbound trace context, and its spans carry no identity

The SDK never extracts an incoming `traceparent`, so every inbound request starts a disconnected trace. Its spans carry only five OpenTelemetry bookkeeping keys: no operation, conversation or task ID, not even the agent's name. Northstar's workaround does extraction only, with `opentelemetry-instrumentation-starlette`; no attributes were invented.

FINDING 3 · FASTMCP 4.0.3 · UNREPORTED

FastMCP reads traceparent out of _meta but silently drops baggage

Server-side extraction copies `traceparent` and `tracestate` and ignores `baggage`, though SEP-414 puts all three on the same tier. A live test finds `northstar.claim.id` on every settlement-agent span and on **zero** payments-mcp spans. This gap is why Northstar's evidence dashboard joins by trace ID, not claim ID.

FINDING 4 · MCP 1.9.4 (PYTHON SDK) · FIXED UPSTREAM

The high-level MCP client had no way to attach _meta to a tool call

`ClientSession.call_tool()` had no `_meta` parameter, so trace propagation meant hand-building the request for the low-level API. PR #1231 added `meta=`, and at `mcp` 2.2.0 the dispatcher injects SEP-414 context itself. Northstar's hand-built injection is now redundant and was left in on purpose, with the decision recorded.

FINDING 5 (CLOSED) · ANTHROPIC 1.4.0 + OPENTELEMETRY-INSTRUMENTATION-HTTPX 0.65B0

The model call had no HTTP span, and the fix was already installed

The Anthropic SDK moved its transport to `httpx2`, so instrumenting `httpx` left the most important hop, the model call, without a CLIENT span. One line, `HTTPX2ClientInstrumentor().instrument()`, added exactly three spans to `api.anthropic.com` per trace, one per model turn. The instrumentor shipped in the version already installed: "we suspected an upstream defect, searched, and found it already fixed."

FINDING 6 · AGENTGATEWAY V1.5.0 · UNREPORTED

The gateway writes trace context into _meta but never reads it

agentgateway writes the SEP-414 carrier into `params._meta` on the way out, but reads inbound context from the HTTP header only. A client that carries context only in `_meta`, as any stdio client must, gets a fresh trace: "a proxy that silently re-roots a trace is worse than an untraced one". Northstar is unaffected only because settlement-agent happens to send both carriers.

Naming and metrics: when the trace holds but cannot be queried

A **semantic convention** is an agreed dictionary of attribute names. If every component writes the HTTP status as `http.response.status_code`, one query works across all of them; if one writes `http.status`, the query silently misses it.

FINDING 2 · FASTMCP 4.0.3 · REPORTED BY OTHERS

Tool-call spans name the tool but not the operation

FastMCP's `tools/call` spans carry `gen_ai.tool.name` but never `gen_ai.operation.name = execute_tool`, so tool calls cannot be grouped by operation across frameworks. Already reported by a third party (`PrefectHQ/fastmcp#3534`). No workaround, under the no-invented-telemetry rule.

FINDING 7 · AGENTGATEWAY V1.5.0 · HALF FIXED, HALF UNREPORTED

Span attributes are the access-log schema, and identity lands on jwt.sub instead of user.id

The gateway's spans carry `http.method`, `http.path`, `http.status`, `http.host` and `http.version`, a set that matches neither the stable nor the legacy HTTP conventions, and the verified caller is recorded as `jwt.sub` where the conventions have `user.id`. **Re-measured on 2026-09-28 against the running v1.5.0: all 32 gateway spans in the deny trace carry the five old `http.*` keys, and none carry `http.request.method`, `url.path`, `http.response.status_code` or `server.address`.** The HTTP half is fixed in PR #3182, merged after v1.5.0 and shipping only in the v1.6.0 alphas; the `jwt.sub` half is unreported. An exact-set test will fail on the next release, "the intended way for this project to find out".

FINDING 12 · PYDANTIC-AI-SLIM 2.42.0, A2A-SDK 0.3.7, FASTMCP 4.0.3, MCP 2.2.0 · HALF REPORTED

Of twelve GenAI metrics, four frameworks emit one

The GenAI conventions define twelve metrics; the four frameworks in Northstar's path emit exactly one, `gen_ai.client.token.usage`, and three contain no metrics code at all. The FastMCP half is reported (#3733). PydanticAI's maintainers argue agent telemetry is a span concern (#6204), and the findings document records that counter-argument, because suppressing it "would be advocacy rather than evidence".

The policy extension point: agentgateway's ExtMCP hook

agentgateway can hand each MCP tool call to an external policy service before forwarding it; the hook is called ExtMCP. If the gateway is airport security, ExtMCP is the phone line to the supervisor. None of these is a bug in the usual sense: each is a place where a careful operator makes a reasonable choice and gets a quiet, wrong result.

FINDING 8 · AGENTGATEWAY V1.5.0 · UNREPORTED

Two config vocabularies, and the only worked example is the wrong one

The guardrails block is spelled differently in the CRD and the standalone `config.yaml` (`FailClosed` against `failClosed`, a nested `remote` against `kind: remote`), and the `examples/` tree has no guardrails example of either kind. Four guessed spellings were rejected by the image's own `--validate-only`. An `examples/mcp-guardrails/` directory is a contribution waiting to happen.

FINDING 9 · AGENTGATEWAY V1.5.0 · UNREPORTED

mcp_error is documented as overriding the error, and it does not

The ExtMCP protobuf says `mcp_error` "Overrides the code-derived default". A test policy server sent `-32003`; the client received `-32001`, with the sent payload nested under `error.data`. Northstar reads its deny marker (`retryable: false`) at `error.data`, because the nesting was measured before the agent was written.

FINDING 10 · AGENTGATEWAY V1.5.0 · UNREPORTED

Naming one header in the allowlist silently drops traceparent

`requestHeaders.allowed` reads like an addition but is a replacement: name one header and trace context stops reaching the policy service. With `["authorization"]`, the setting Northstar shipped first, every `policy.decision` span landed in an orphan trace while the deny kept working. "A default that is safe and a first edit that is not." Northstar now names `traceparent` and `tracestate` explicitly.



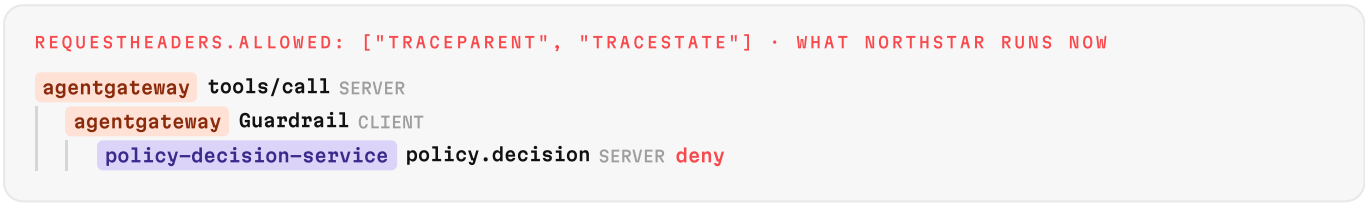


FIGURE 12.3

The same deny, before and after one config line. In the first version the decision was made, the payment was refused, and the policy service's spans carried every attribute they should. Only the claim that the decision appears in the same trace as the call it decided was false. No log line mentioned a header.

OQ-U1 (CLOSED) · AGENTGATEWAY V1.5.0 · NOTHING TO FILE

Can a batched JSON-RPC array sneak a payment past the policy hook? No.

A probe posted a two-element array with a real over-limit payment as bait. v1.5.0 answered `HTTP 400` before the policy hook: zero `policy.decision` spans, zero payments-mcp spans, ledger unchanged. The probe runs on every gate; the day a release accepts batches, the question reopens.

Interop: two correct projects that disagree

FINDING 11 · OPENTELEMETRY-SDK 1.44.0 (PYTHON) + GO.OPENTELEMETRY.IO/OTEL V1.41.0 · INTEROP UNREPORTED

One hex digit splits a trace in two, and nothing reports it

Python implements W3C Trace Context Level 2 and sends flags `03`; Go v1.41.0 validates against Level 1, rejects any flag above `02`, and silently starts a new trace. Before the fix, 30 reviewer-ui traces and 30 payments-api traces shared **zero** trace IDs, and a live A/B control differing in one hex digit proved the cause. Go v1.42.0 fixed it; Northstar moved payments-api to v1.46.0 and deleted its workaround.

FINDING 13 · OPENFEATURE-HOOKS-OPENTELEMETRY 0.3.1 + OPENFEATURE-SDK 0.10.0 · UNREPORTED

Two OpenFeature packages disagree on which signal carries a flag evaluation

The hook emits `feature_flag.evaluation` as a span event; the SDK ships an unused builder that shapes it as a log record, with deprecated spellings. Which signal and spelling you query depends on which package you installed, and for Northstar that is the autonomy dial's audit trail. Northstar emits both records.

FINDING 14 · OPENFEATURE-HOOKS-OPENTELEMETRY 0.3.1 · UNREPORTED

The hook declares feature_flag.set.id and feature_flag.version and never emits them

`TracingHook` defines both constants but never reads `flag_metadata`, so "which ruleset version produced this decision?" cannot be answered, which is exactly the audit question of Chapter 10. The SDK's sibling utility already has the four lines; porting them is the smallest contribution on the list.

The portal: Backstage assumptions meet a laptop cluster

FINDING 15 · @BACKSTAGE/BACKEND-DEFAULTS 0.17.8 · FEATURE REQUEST

A $text placeholder cannot inline a local file when there is no git remote

For a `type: file` location, URL resolution fails and no reader exists for `file:`, so the catalog could not inline the real agent cards. Northstar uses a `providesApis` edge and a written pointer instead.

FINDING 16 · BACKSTAGE GUEST PROVIDER 0.2.22 · BY DESIGN

A correct security default turned a finished catalog into one nobody could open

The guest sign-in provider refuses to run outside `NODE_ENV=development` unless `dangerouslyAllowOutsideDevelopment` is set. The first build collated and passed every RBAC check, and any browser got a `403`. Northstar sets the flag explicitly, with the trade stated in a comment.

Silent failure is the dominant mode

Look only at *how each finding showed itself*. Almost none raised an error. Every service was healthy, every span well formed, every test green, and something true about the system was quietly false (Figure 12.4). In a regulated domain, that is the dangerous kind of failure: a regulator asking "show me what the agent did" gets an answer that is incomplete, and nothing says so.

FINDING	ERROR?	LOG LINE?	HEALTHY?	WHAT YOU ACTUALLY SEE
F3 · FastMCP baggage	none	none	yes	Trace joins; claim ID absent downstream
F6 · gateway _meta ingress	none	none	yes	Two traces where there should be one
F9 · mcp_error nesting	none	warn*	yes	Deny works; the intended code is buried
F10 · header allowlist	none	none	yes	Decision spans in their own orphan traces
F11 · W3C Level 2 flags	none	none	yes	Two healthy traces, zero shared IDs
F12 · missing metrics	none	none	yes	Dashboards with nothing to plot
F14 · flag version omitted	none	none	yes	An audit field that is simply never there
F16 · guest auth gate	403	none	yes	Catalog collated; no browser can open it

* only for a malformed mcp_error payload, and only in the gateway's own log. F16's 403 reached a browser request, never a health check or a test.

FIGURE 12.4

How eight of the findings actually showed themselves. Read down the "healthy?" column: every service was up in every case. The only real error in the table, F16's 403, was invisible to every check the project had. It took someone opening the page.

> "Nothing errors. Both services are healthy. You just have two traces, and every one of your dashboards is green."
>
> LINE FOR THE STAGE · AFTER FINDING 11

Three cheap habits caught these findings. **Assert the shape of the trace** (this span descends from that one, by a trace ID the test injected), not span counts. **Pin third-party attribute sets by exact equality**, as Finding 7's test does. **Run a control**: Finding 11 became a finding because of two requests that differed in one hex digit.

File first, and contribute back

The agentgateway block should be filed first. Findings 10, 8 and 9 are doc-sized fixes that can go in as pull requests with the evidence attached, and agentgateway is at exactly the stage where good first reports shape its documentation for years. Before filing anything, re-measure on the latest release: Findings 4, 5, the HTTP half of 7 and the Go half of 11 were already fixed by the time anyone looked.

The parent talk "Your Agent Did WHAT?" closed with "be a good open source citizen and contribute back to projects when you see a gap!" The "Taming Complexity" build did: its `a2a-java#707` (merged 2026-03-09) propagates OTel context in async operations, the Java counterpart of Finding 1. Taming Complexity found gaps and sent code. Northstar documented more gaps to filing standard and has sent nothing yet.

CHECK BEFORE A SLIDE

Of the three upstream PRs on the Taming Complexity slide, only `a2a-java#707` is a trace-propagation fix; the other two are feature work, and `dapr/java-sdk#1693` was still open on 2026-09-28. Call them "upstream work that came out of building the demo".

TAKEAWAY FOR PLATFORM TEAMS

Budget for silent failure: for every hop, write down what "broken" would look like and assume it will look healthy. Keep an upstream-findings file from week one, with project and version, the problem, the evidence, why it matters beyond you, upstream status, and workaround or deliberate non-workaround. It is the list of places where your platform's telemetry cannot be trusted yet. Then file the doc-sized ones as pull requests.

CHAPTER 13

Lessons from building (and verifying) an agentic platform

Twenty-two things that looked done and were not, what each one teaches a platform team, and what it was like to build all of it with AI coding agents.

Over six milestones the project logged roughly 290 numbered decisions, called *rulings*. Twenty-two of them are worth a platform team's time, because each is a moment when a check came back green and the thing it was supposed to prove was not true. The ledgers live under the git-ignored `.superpowers/sdd/`, so their IDs are private build ledger references.

IN PLAIN WORDS

A **guard** is a test that stops one specific mistake from coming back. A **gate** is the big end-of-milestone test run (`scripts/smoke-test.sh`, live settlements against a real model). **Mutation testing** means deliberately breaking the code a guard protects and checking the guard goes red. **Fail-closed** means that when something goes wrong, the system refuses.

The twenty-two lessons, with the best evidence for each

GOVERNANCE

- **2 · Write policy as "permit only if in the domain".** A reviewer fired 34 hostile requests at the Policy Decision Service; `NaN`, `-inf`, `-500000` and `0` were all permitted under the limit rule's name, because the rule only tested `amount > limit`. "Deny if bad" is fail-open for every input you did not imagine (Figure 13.1).
- **3 · Verify defaults against the binary.** The plan and four tasks said agentgateway's `failureMode` defaults to `failOpen`. Measured three ways against the pinned v1.5.0 image, omitting it behaves like `failClosed`. The default that decides whether an outage moves money is checked on the running binary, not the docs.
- **4 · Tool schemas are policy surface.** `execute_payment` first took only a payment ID, so a "refuse above €6,000" rule had nothing to read. `amount` went into the call, and payments-api answers `409` if it does not match the record.
- **5 · Assert on the absence of behaviour.** The test counts *attempts* after a denial. The weaker assertion "a denial happened" still passes when the agent retries three times.
- **6 · Human approval changes who asks, not what is allowed.** An approved payment is executed by a *senior* SPIFFE identity; the key test checks that the junior identity still cannot pay after an approval exists. "approved: true" as a policy input is a bypass with a user interface.

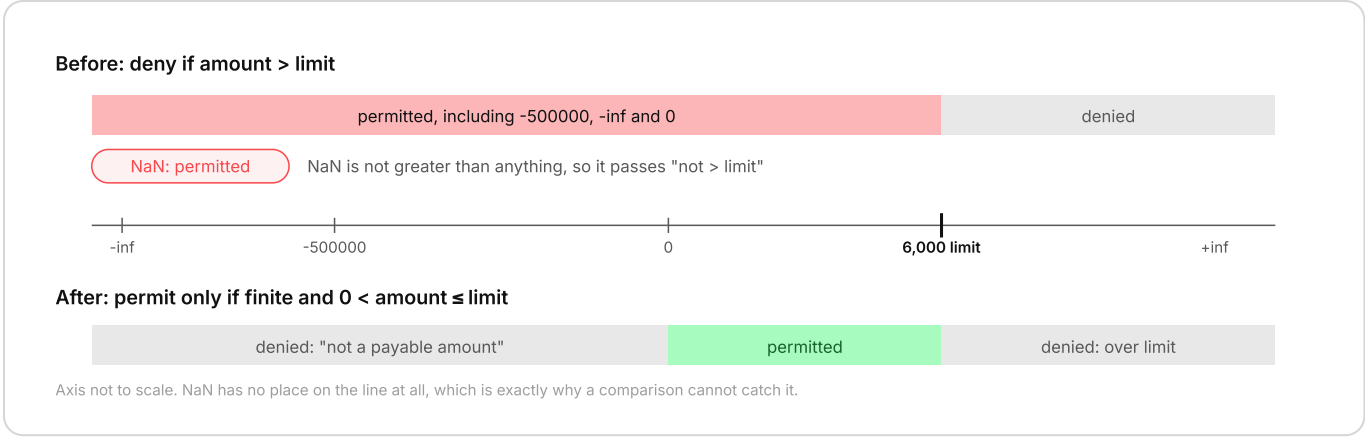


FIGURE 13.1

The same limit, written two ways. The first rule permits everything the author did not think about. The second names the small region that is valid money and refuses the rest under its own rule name.

TELEMETRY AND EVALUATION

- **1 · Your test is also a client.** The first spike concluded agentgateway breaks the trace. Its test client sent context only in `_meta`; the real client's instrumented HTTP library also sent the header, and the trace held. The narrower truth became Finding 6.
- **7 · Loss looks like sampling.** OpenSearch rejected every `policy.decision` span on a mapping conflict: 142 spans in Jaeger, 139 in OpenSearch, and the missing three were exactly the decisions. "No errors" came from `kubectl logs --tail=200` on a 17,173-line log. The guard is span-set parity between backends.
- **8 · A partial trajectory always reads higher.** The eval scorer read before the last decision span was indexed and scored a non-compliant run **1.0**; minutes later it scored **0.0**. A 20-trace cap dropped the smallest bucket, which held the violation. The scorer now raises `TrajectoryTruncated`: "Emitting no score beats emitting a flattering one."
- **9 · Independent audit needs independent truth.** The scorer first read the autonomy level off the span it was judging. At the commit before the fix, a self-reported `autonomous` span moved €900,000 and scored 1.0. A missing value must count as a failure.
- **10 · When the default equals the value under test, you tested nothing.** A reviewer replaced the flag's resolved level with the literal `"assisted"`; all 119 unit tests and 217 suite tests stayed green, because the shipped contract returns `assisted` on both branches.

VERIFICATION HYGIENE

- **11 · Mutation needs bytecode writing off.** A stale `.pyc` kept a reverted mutant alive; in M4 it reached 7 of 15 mutations and produced both a false GREEN and a false RED. `scripts/mutate-guard.sh` now sets `PYTHONDONTWRITEBYTECODE=1` and purges caches.
- **12 · Never pipe the test command.** pytest piped through `tail` reported GREEN over two failures, because `tail`'s exit code is always zero.

- **13 · Derived figures survive, retyped ones rot.** All 13 findings of M2.1's final review were prose falsified by a later task, and about 100 guards caught none. "A remembered fact is a retyped fact with no guard on it."
- **14 · A skip is a pass nobody reads.** A guard worked out the current milestone from the branch name; it skipped on `main`, the only branch a reader sees, in a gate defined as "zero skips".
- **17 · Gate after the last change.** M5's review fix wave changed nine files on the gate's path after the gate ran, and a staleness guard demoted the row to `PREDICTED, NOT MEASURED`. M4 and M6 ran the gate after the fixes.
- **18 · Run cheap whole-suite checks on every task.** In M3 three tests outside the gate were red from Task 2 or 3 until the last task, because each task ran only its own suite.
- **19 · Verified meant verified-something-adjacent.** Three times in M6: a collated catalog nobody could open (Finding 16), an RBAC matrix checked against an image an hour older than the commit, and 744 lines of tests executed by nothing. Call the route only the new code serves, from where the audience stands.
- **20 · The guard written in a hurry reproduces the failure.** A guard against stale images compared tags, not digests; a reviewer made it pass against a mutated, uncommitted image. Mutate it before you trust it.

OPERATING THE PLATFORM

- **15 · Datapath beats behaviour.** Stripping Cilium's DNS rules left behavioural probes green; only reading the enforced policy with `cilium-dbg bpf policy get` caught it. M4-R17 found only the agent plane was actually enforced.
- **16 · Liveness is a round trip.** `lsof` showed ports LISTENING for port-forwards whose pods were gone. "The check must be a request, not a socket."
- **21 · A lying UI is worse than no UI.** Backstage's autonomy card could display a level the policy service would never enforce, and a fixture mutation stayed green 30 out of 30. The card now fails closed.
- **22 · Cold start is a claim until tested.** From an empty cluster, a setup script restarted a Deployment before creating it. With no git remote, a cold rebuild is the only recovery path. Rebuild from nothing before any talk.

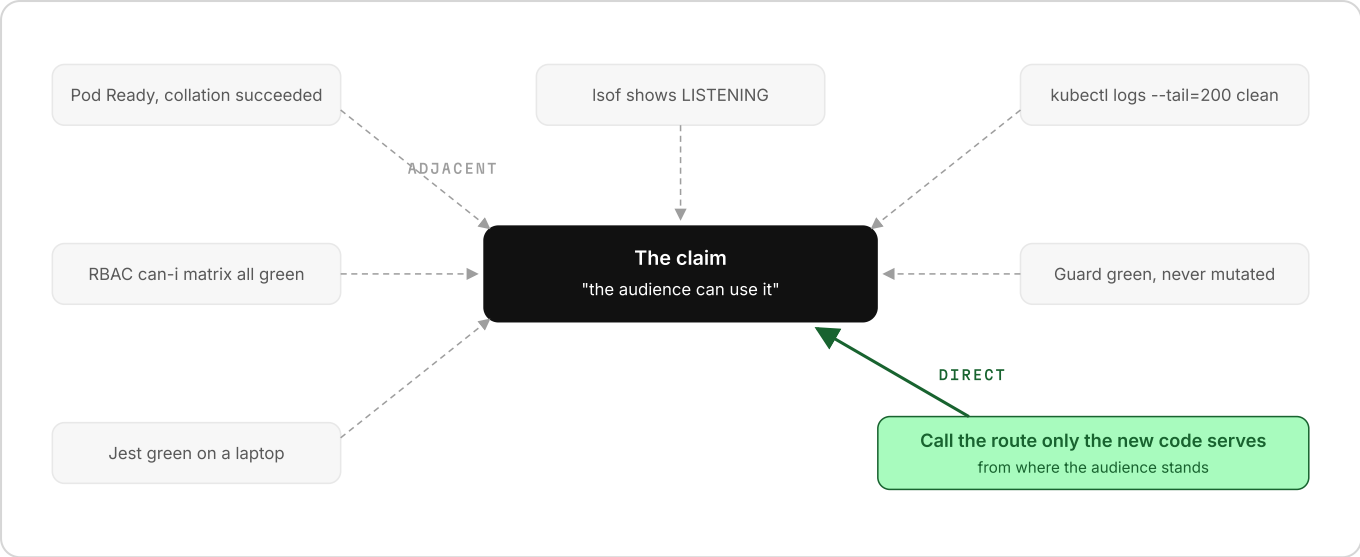


FIGURE 13.2

The shape of the most common failure in the whole build. Each grey box is true evidence, and each was reported as proof at least once. None of them touches the claim. Only the green box does, and it is usually the cheapest check of the lot.

Building an agentic platform with AI coding agents

Almost all of Northstar was written by AI coding agents using **subagent-driven development**: one long-running controller session writes the plan and hands each task to a fresh implementer session with no memory beyond its brief; risky tasks get a fresh reviewer; a whole-branch review reads everything at the end. Eight meta-lessons came out of it.

1. **Agents report success on adjacent evidence.** The most common failure was true evidence about the wrong thing (Figure 13.2). Treat an agent's "done" as a pointer to where to look.
2. **Mutation beats reading.** M4's eight inert guards and the M5 critical findings came from breaking a guard's target, not from diff review. "Disbelieve a GREEN."
3. **Implementers pushing back was the process working.** The M2.1 ledger counts eight times a fresh implementer contradicted the controller and was right.
4. **Model tiering is a real lever.** After M2.2 cost about a million subagent tokens to close, the build went "lean": a cheaper model by default, the strongest only for security-path work and whole-branch reviews.
5. **Parallel agents collide on shared documents, not code.** Two agents merged each other's edits through `docs/telemetry-contract.md` (M5-R30).
6. **Permission denials were respected.** An implementer blocked from setting `dangerouslyAllowOutsideDevelopment` stopped and reported (M6-R7). Require that of agents that touch infrastructure.
7. **Decisions must outlive the ledger.** Anything the next milestone depends on moved into a "carried debt" section of each committed design spec.
8. **Measure the plan before building on it.** The costliest defects were in the plan: the missing `amount` (Lesson 4) and the inverted `failureMode` default (Lesson 3).

> “We built an agentic platform with agents. The agents were good at writing code. They were even better at bringing back true evidence about the wrong thing.”

LINE FOR THE STAGE · META-LESSONS

PART V

The talk

Everything above, cut down to what fits in a KubeCon slot: one sentence, four acts, four live beats, and the questions the room will ask.

CHAPTER 14

Building the talk

A working plan for a 30–35 minute session: the thesis, the arc, every slide, the live demo, the hard questions and the lines worth saying out loud.

The first thirteen chapters are the research. This chapter is the talk you build out of it: a slide plan to open next to a deck editor, a demo plan to run from, and a Q&A sheet for the train to Salt Lake City. Fast-moving facts point at Appendix B, the checklist of claims to re-verify before a slide goes out. The plan assumes Kasper presents alone, and marks where William Rizzo could take the governance half.

IN PLAIN WORDS

A conference talk has three jobs. It has to **make one claim**, **show evidence for it**, and **leave people with something to do on Monday**. A slide that serves none of them gets cut, however good it looked in the earlier talks.

The one sentence

The abstract ends with a promise: *if you can't trace it, you can't fix it*. Northstar showed that the regulated version of that promise is stronger. In a bank, an insurer or a hospital, "fix" is not enough. You have to show an auditor what the agent did, who allowed it, and why. So the thesis the talk argues is this:

> "Agents change the nature of complexity, so the platform has to decide what an agent may do, and prove it afterwards from the trace. We already have the open-source pieces to do both. If you can't trace it, you can't govern it."
>
> THESIS FOR THE KUBECON NA 2026 SESSION

Every act defends one clause. Act 1 defends "agents change the nature of complexity". Act 2 shows what tracing already gives us. Act 3 shows the platform deciding, live. Act 4 shows the proof an auditor can read. The close is honest about what is missing.

The session title is fixed and abstract, so the subtitle does the concrete work. The recommended subtitle is **"If you can't trace it, you can't govern it"**: it echoes the abstract's closing line, survives being tweeted, and gives the close a callback. Two alternatives become act titles: "A refusal is the platform working" and "What the agent did, who let it, and how you prove it".

The room is mixed: platform engineers who know gateways and network policy, developers who know an agent framework but little about traces, observability specialists who know OTel, and risk and compliance people who know the rules but not how agents are built. So the book's layering rule applies to the talk too: **plain words first, then evidence, then depth**. Every new term gets one plain sentence before it is used, and the depth lives in the Q&A. Say out loud in the first minute that beginners are welcome: "If you've never read a trace before, you'll be able to read the ones in this talk. If you have, stay for the parts where they lie to you."

The narrative arc

A KubeCon breakout slot is 35 minutes. The plan budgets **33 minutes of content and 2 minutes of buffer**, and treats Q&A as bonus time in the hallway. Figure 14.1 is the whole talk on one line. Each act asks one question and answers it before the next act starts; the open poses a puzzle (why is a refusal good news?) that Act 3 answers live, which keeps the audience with you through the two background acts.

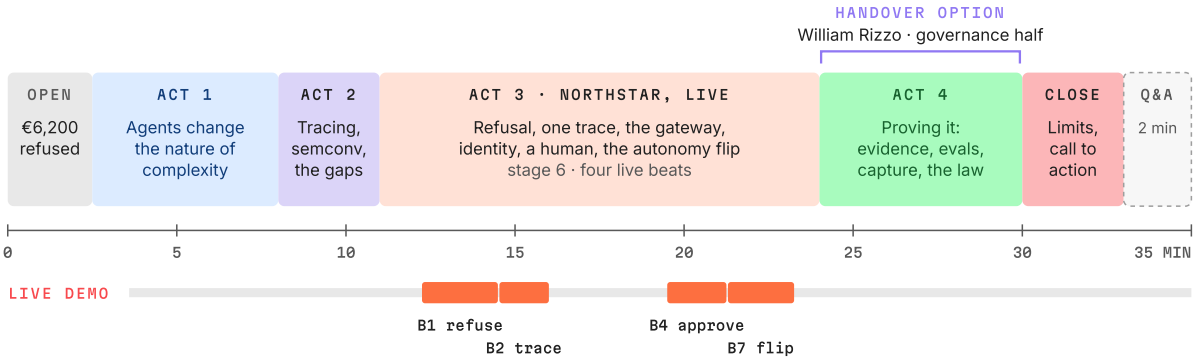


FIGURE 14.1

The talk on one line. Notice that almost all live time sits inside Act 3, between minutes 12 and 23, so a demo failure costs one act and not the talk. The purple bracket marks the clean handover point: Act 4 (evidence, evals, content capture and regulation) is the governance half William Rizzo could present later.

WHERE THE MINUTES GO

BLOCK	MINUTES	QUESTION	MAIN EVIDENCE
Open	0:00–2:30	Why is a refusal good news?	A screenshot of `"status": "refused"` on €6,200
Act 1 · Agents change the nature of complexity	2:30–8:00	Why are agents harder to run, and to govern?	Reused slides from both earlier talks, FINRA 2026 quote
Act 2 · What tracing already gives us	8:00–11:00	What do traces and GenAI semconv give us, and where do they stop?	traceparent, normalizer, the “what can they answer” table
Act 3 · Northstar, live	11:00–24:00	Can the platform decide, and can the trace show it?	Four live beats at stage 6, gateway span anatomy, access model
Act 4 · What the agent did, who let it, how you prove it	24:00–30:00	Can we prove it to an auditor?	Evidence view, 1.0 vs 0.0 score, capture matched pair, requirement table
Close	30:00–33:00	What is missing, and what do I do on Monday?	Honest limitations, call to action, three things

Two speakers. If William Rizzo joins, Kasper takes the open through Act 3 (observability and the gateway) and William takes Act 4 and the close. The handover line is slide 29: “The platform API is the resource. So what does an auditor get to read?” Rehearse the handover twice.

The slide plan

Forty-two slides, one row each: number and start time, title, origin, and the one point it makes. Eight come from `DID WHAT` *Your Agent Did WHAT?* (Adriana Villela and Kasper, SRE Day London Q3 2026), eight from `TAMING` *Taming Complexity* (Mauricio Salatino and Kasper, KubeCon EU), and twenty-six are `NEW`, built from this book’s evidence. The reused slides are mostly Acts 1 and 2 and get faster delivery; the new ones carry the argument and get the time.

#	TITLE	FROM	THE POINT
Open · 0:00–2:30			
01 · 0:00	Title	`NEW`	Subtitle, name, the book’s QR code. Say nothing yet; let the room settle.
02 · 0:30	“This is a success”	`NEW`	Crop of `"status": "refused"` on €6,200. By the end, the room should agree this screen is the best thing that happened all day.
03 · 1:30	Where this comes from	`NEW`	Two earlier talks asked what agents do and how to trace them. This one asks what happens when a regulator asks. Welcome beginners.
Act 1 · Agents change the nature of complexity · 2:30–8:00			
04 · 2:30	The evolution of architectures	`TAMING`	Each step made building easier and understanding harder. Thirty seconds.
05 · 3:15	New execution patterns	`TAMING`	“The call graph is no longer something you drew. The model draws it at run time.”

#	TITLE	FROM	THE POINT
06 · 4:00	Anatomy of an agent	DID WHAT	Agent, model, tool, harness. The agent decides when to stop; in a regulated domain, that is the problem.
07 · 4:45	MCP, A2A and Skills on one slide	TAMING	Tools, agent-to-agent, skill folders; one dated fact each (Appendix B). None is a CNCF project.
08 · 5:45	The decoder ring	DID WHAT	Left column: what your platform already watches. Right column: what it doesn't, yet.
09 · 6:30	"We don't know what the agent did" is not an answer	NEW	FINRA's 2026 report names scope and traceability: the two problems this talk is about.
10 · 7:15	The claim	NEW	The thesis, large, over three words: decide, trace, prove.
Act 2 · What tracing already gives us · 8:00–11:00			
11 · 8:00	A shared pain: context gets lost	TAMING	A trace is the story of one request; context propagation is the tracking number that keeps it in one piece.
12 · 8:30	W3C traceparent, now also in MCP	TAMING	The same keys ride in `params._meta` (SEP-414, spec `2026-07-28`). Instrumentation has not caught up everywhere.
13 · 9:00	Skills break traces unless you bridge them	TAMING	A skill is a subprocess; it inherits no context unless you hand over `TRACEPARENT`.
14 · 9:30	Competing standards	DID WHAT	You can't query what you can't name. OTel accepted OpenInference's code on 2026-06-16.
15 · 9:50	One processor to unify them	DID WHAT	Normalise in the Collector, once, in the platform. Not in every team's code.
16 · 10:10	What can gen_ai answer today?	DID WHAT	The rows an auditor needs most are the ones that are off by default. Hold that for Act 4.
17 · 10:35	We can prove what, not why	DID WHAT	`gen_ai.tool.call.reason` doesn't exist, so the *why* must come from the platform. Bridge to Act 3.
Act 3 · Northstar, live · 11:00–24:00			
18 · 11:00	Meet Northstar	NEW	The money hop, browser to payments-api. Open source and open standards only.
19 · 11:45	The model can request. The platform decides.	NEW	Neither agent nor tool contains a limit, and a test fails if one appears.
20 · 12:15 live	Beat 1: the refusal is the success	NEW	€6,200 refused, €5,800 executed. Name the refusal as success before anyone reads it as a bug.
21 · 14:30 live	Beat 2: one trace, and the absence that proves it	NEW	Four spans, then the missing execute spans: evidence that nothing happened.
22 · 16:00	What agentgateway says about one governed call	NEW	The gateway says who called, which tool, and whether it went through. The *why* is the policy service's span.
23 · 16:45	HTTP 200 means no	NEW	The deny marks the span ERROR while `http.status = 200`. Alert on the span and the decision.

#	TITLE	FROM	THE POINT
24 · 17:30	Limiting access to MCP, layer by layer	NEW	Network to autonomy contract. Only network policy shows what *couldn't* happen.
25 · 18:15	Why an external decision, not a CEL rule	NEW	CEL can't see the amount at decision time in 1.5; ExtMCP gives a decision span with a rule name.
26 · 18:45	Identity is an input, not a label	NEW	The caller is half the lookup key. Beat 3 is shown as this slide, not run live.
27 · 19:30 live	Beat 4: a human overturns it, through the same door	NEW	"Nothing was overridden. A different caller was asked about." A link, not a parent. The reviewer id is a typed string.
28 · 21:15 live	Beat 7: one live value changes what the platform may do	NEW	Flip to autonomous; the identical €6,200 executes. No restart, no rebuild. Flip back before leaving.
29 · 23:15	The platform API is the resource	NEW	FeatureFlag YAML, Backstage card, audit span. `managedFields` is erased by the next write; the span survives. Handover point.
Act 4 · What the agent did, who let it, how you prove it · 24:00–30:00			
30 · 24:00	The evidence view: no new application code	NEW	Four panels from spans already emitted (beat 5). The join is on trace id, because the claim id stops at MCP.
31 · 25:00	A score that can go to zero	NEW	1.0 vs 0.0, written into the trace it judges (beat 6). Build it from a run where the permit path had its own claim id; see the stage hazards.
32 · 25:45	And LLM-as-judge?	DID WHAT	A judge answers "was it good?"; an auditor asks "did it follow policy?". Same output format, so a judge can be added.
33 · 26:15	Content capture, measured as a matched pair	NEW	The governance story survives full redaction; the model's own account does not. Redaction is a trade.
34 · 27:00	More telemetry is not better telemetry	TAMING	Meaningful spans, 100% capture, and the parity guard: OpenSearch once dropped exactly the `policy.decision` spans.
35 · 27:30	From requirement to artifact	NEW	The table below. "None of this makes you compliant." Walk two rows, not six.
36 · 28:30	The deadline moved. The requirements didn't.	NEW	Annex III to 2 Dec 2027 (Reg. 2026/1744); SR 26-2 puts agentic AI out of scope.
37 · 29:00	Sixteen findings, none filed yet	NEW	The dominant failure mode is silent: both services healthy, spans in two traces.
38 · 29:30	A maturity model for OpenTelemetry support	TAMING	The sixteen findings are what such a model should make visible.
Close · 30:00–33:00			
39 · 30:00	What this does not do	NEW	The honest limitations, below. Naming them first is worth more than the features.
40 · 31:00	Becoming a regulated-platform wizard	DID WHAT	The six earlier steps plus three: decide at a choke point, make identity an input, keep every decision span.
41 · 31:45	What to do on Monday	NEW	Five verbs, below. End on "Sixteen gaps. Pick one."

#	TITLE	FROM	THE POINT
42 · 32:30	Three things to remember	NEW	Figure 14.4, QR code, then the callback: “If you can’t trace it, you can’t govern it.”

Slide 35 carries the requirement-to-artifact table:

REQUIREMENT	WHAT IT ASKS FOR	NORTHSTAR ARTIFACT
EU AI Act Art. 12, 19, 26(6)	Automatic logs over the system’s lifetime, kept at least six months	One trace per claim; `policy.decision` per tool call; no retention tier yet
EU AI Act Art. 14	Humans can override, reverse, stop	Reviewer approval via a governed identity; the autonomy contract’s refusing resting state
EU AI Act Art. 86; GDPR (C-203/22)	Per-decision explanation of the procedure actually applied	Rule name, limit, amount and caller on the decision span; evidence view
FINRA 2026 report	Prompt and output logs, model version, agent action tracking, human in the loop	GenAI spans (capture ON), `gen_ai.request.model`, gateway spans, `review.decision`
DORA RTS Art. 12	Logging procedures, tamper protection, detect logging failure	Span-count parity guard between Jaeger and OpenSearch; no tamper protection
ISO/IEC 24970 (FDIS)	An information model for AI event logs	Nothing yet: the OTel GenAI → 24970 mapping is unwritten

The live demo

The demo is the talk’s evidence. This plan is `docs/demo-runbook.md` cut to a 33-minute talk; the runbook is the operator’s document and wins any disagreement with this chapter.

IN PLAIN WORDS

A **beat** is one small scene in the demo with one point to make. The runbook writes each beat as **DO** (what you click), **SHOW** (where the audience should look) and **SAY** (the sentence that makes the point). If you only remember the SAY lines, the demo still works as a story.

Run **stage 6**, the complete system, so every beat is available and nothing is installed on stage. A 33-minute talk with slides has about eight minutes of live time, so the plan runs four beats live and shows three as slides:

BEAT	MODE	TIME	DO	SHOW	SAY
1 · The refusal	**live**	~2 min	8080: submit €6,200, then €5,800	Outcome block: `refused`, then `executed`	"That is not a failure. That is the platform working."
2 · One trace, and the absence	**live**	~1.5 min	16686: `customer-ui` / `claim.submit`, open the €6,200 trace	`jwt.sub`, `policy.decision=deny`, `grpc.status=0`, missing execute spans	"The absence of those spans is the evidence that nothing happened."
3 · Identity is an input	slide 26	—	(pytest criteria 4 and 5, not run)	The rule table	"Identity is an input to the decision, not a label on it."
4 · The human overturns it	**live**	~1.75 min	8081: Approve as `alex.mercer`	Reviewer page; span link (slide or Jaeger)	"Nothing was overridden. A different caller was asked about."
5 · Prove why the money moved	slide 30	—	(OpenSearch dashboard, screenshot)	Four panels	"No new application code behind this dashboard."
6 · Ask the auditor	slide 31	—	(`POST /score`, screenshot)	1.0 vs 0.0	"A scorer that returns 1.0 for everything proves nothing."
7 · The golden path	**live**	~2 min	7007: flip to autonomous; 8080: resubmit €6,200; flip back	Card, then `executed`	"One word changed on one live object."

Two runbook warnings survive every cut. **Name the refusal as success before anyone reads it as a bug.** And in beat 7, **say the Backstage login asked for nothing** before anyone asks, then come back to what keeps the write narrow.

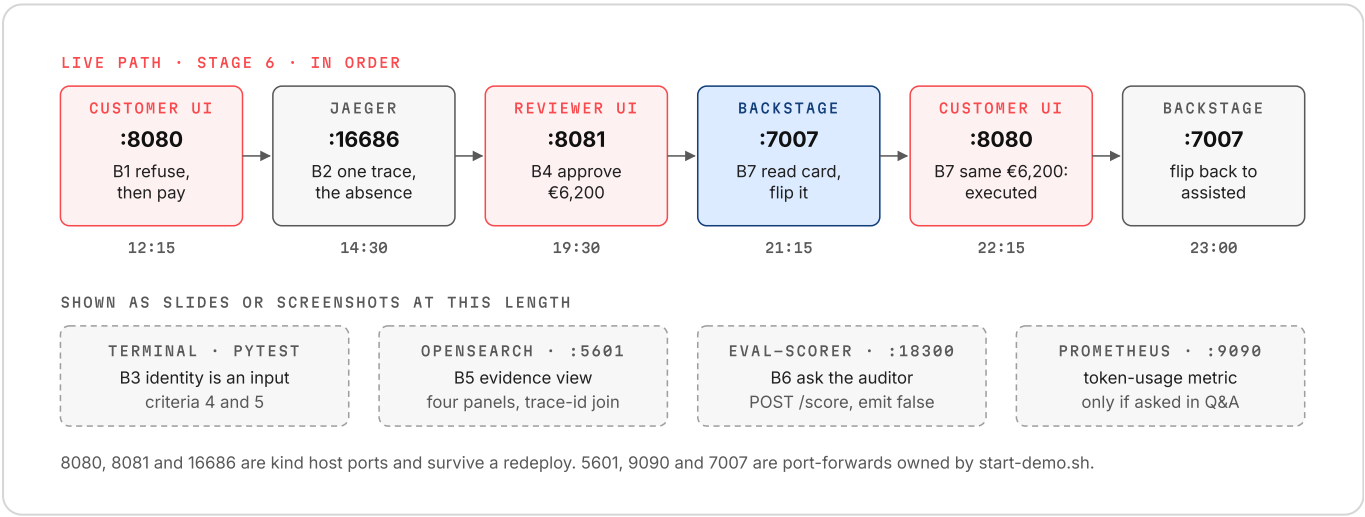


FIGURE 14.2

The demo as a walk across surfaces. Open these six tabs in this order before you go on stage, so each beat is one tab to the right. The bottom row is reachable but not shown live at this length; the runbook's 45-minute cut runs all of it.

PRE-FLIGHT, RESET AND TRIAGE

Budget one full cold bring-up before the audience arrives, plus one run of both claims; do not time it for the first time on stage. The countdown: at T-60 the one-time setup (`.env` with the API key, venvs); at T-30 `setup-kind.sh --stage 6`, where the

key must read `SET`; at T-15 `start-demo.sh`, which owns every port-forward and stays running; at T-10 eyes-on checks (pods Running, the contract `assisted` for both identities, Jaeger switched to Trace Timeline); at T-5 one rehearsal €6,200 that must come back `refused`, which proves the key, the A2A chain, the gateway, the PDS, flagd and the ledger in one action; then a reset.

- **Leave the contract refusing.** A cluster left `autonomous` makes beat 1 execute instead of refuse. Flip back through the Backstage card, so the revert gets its own audit span, or re-apply `deploy/platform/openfeature/featureflag.yaml`.
- **Restart payments-api** to clear refused rows from the reviewer UI: `kubectl -n northstar-systems rollout restart deploy/payments-api`. The ledger is in memory.
- **Never promise a payment id on stage.** Ids are cumulative since the pod last restarted. Say "a payment id", never a number.

When something breaks, decide in under ten seconds whether to fix or fall back (Figure 14.3). The screenshot slides carry the same beats from a known-good run; switch to them without apology and keep the SAY lines. The argument depends on the trace shapes being real, and the screenshots are real.

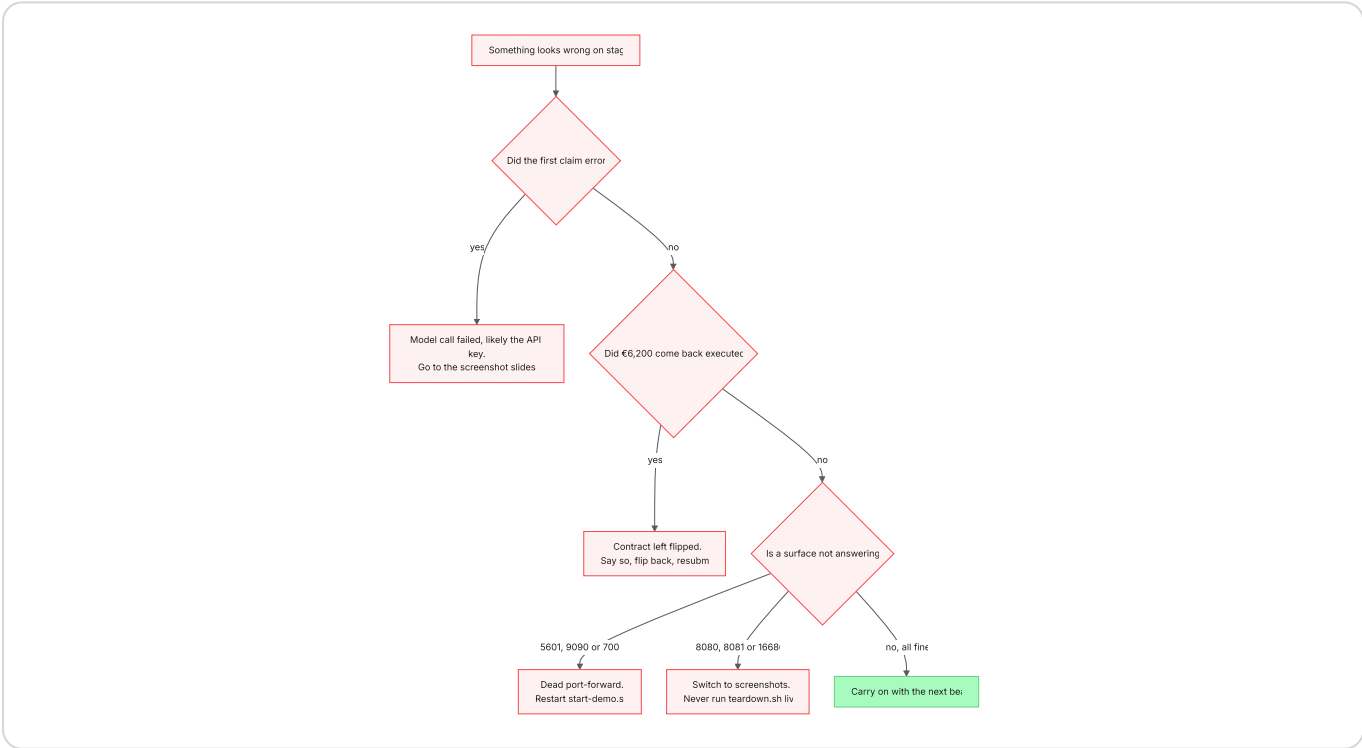


FIGURE 14.3

Triage in three questions. Two outcomes are fixable in front of the audience (a flipped contract, a dead port-forward). The other two go straight to the screenshot slides. The one thing never to do live is run `teardown.sh` or `smoke-test.sh`: the first destroys the cluster, the second is the full ~10-minute, token-spending gate.

CHECK BEFORE A SLIDE

Stage-prep hazards. Three from the repo: (1) `ANTHROPIC_API_KEY` must read `SET` at the end of `setup-kind.sh`; if empty, every pod comes up Running and the first claim fails. (2) The SPIRE OIDC provider's init image is a digest pinned against a rolling `:latest` tag; a recreate the night before could fail the pull, and it fails closed (every MCP call refused). Pre-pull, or recreate once the day before. (3) Venue Wi-Fi: the settlement decision is a real model call to `api.anthropic.com`. Tether, and have the backup slides ready.

Three more, found while capturing this book on 2026-09-28. (4) **Beat 6 scores 0.0 as scripted.** Beat 1 submits €5,800 under the same claim id as the refused €6,200, so `POST /score {"claimId":"CLM-1842"}` reads the €5,800 run as a retry by the refused identity (`stopped_when_refused 0.0`) and compares its amount with the approved €6,200 (`amount_executed_matches_approved 0.0`). The runbook says 1.0. On a cluster that has rehearsed the claim, the default window also sweeps in every earlier run. Pass `sinceMs`, submit the permit run under its own claim id, or keep beat 6 on a prepared slide. (5) **Do not close or reload the customer tab straight after a submit.** The browser batches spans for 5 seconds; close sooner and `claim.submit` never arrives, so `customer-ui` vanishes from Jaeger and beat 2's search by service finds nothing. (6) Stage 6 read 35 pods and 5.11 GiB on the 2026-09-28 bring-up, against 6.05 GiB on 2026-09-15; measure on the presenting laptop, not from this book.

Q&A preparation

The ten questions most likely to come from a KubeCon room, with answers short enough to say in 30 seconds. The chapter named has the long version.

QUESTION	SHORT ANSWER
Isn't agentgateway a CNCF project?	No. Solo.io donated it to the Linux Foundation on 2025-08-25 and it joined AAIF on 2026-06-04. The confusion is kgateway, which is CNCF Sandbox. Say "a Linux Foundation project, hosted in AAIF". [Ch. 6]
Why not just use the gateway's CEL authorization?	The rule needs the amount, and in agentgateway 1.5.x tool arguments are not available to CEL at decision time. A CEL deny is HTTP 400 with `-32602 Unknown tool` and names no rule; ExtMCP gives a decision span with rule, limit and caller. Use CEL for coarse RBAC and hiding tools; layer both. [Ch. 7]
What if the policy service is down?	Every tool call is refused. `failureMode` is `failClosed`, the measured v1.5.0 default, still set explicitly because fail-open and healthy looks identical to fail-open and not governing. A decision shows `grpc.status=0` on `Guardrail`; an outage shows a `checkRequest failed` error. The exact outage span shape is not yet measured. [Ch. 7]
Can the agent go around the gateway?	Not from its namespace. Cilium lets only agentgateway reach payments-mcp; before that policy, claims-agent reached the tool directly. The proof is BPF map reads and in-pod probes, not spans. Honest part: payments-mcp checks nothing, and payments-api has no network policy. [Ch. 7]
Who approved it? Is the reviewer authenticated?	No. `review.reviewer.id` is a typed string; the SPIFFE ID on the span is the reviewer UI's own. What is governed is the execution: it runs as `senior-settlement-agent`, through the same gateway and PDS, under its own €25,000 row. [Ch. 8]

QUESTION	SHORT ANSWER
Why is a deny not an error span?	A refusal is the service working; ERROR would page on-call for correct behaviour. The PDS sets ERROR only on internal faults. agentgateway's own `tools/call` span does mark the deny ERROR while returning HTTP 200, so alert on `policy.decision` and the gateway's `reason`, not HTTP status. [Ch. 6]
Doesn't capturing prompts violate GDPR or HIPAA?	It can. The conventions make capture opt-in and recommend storing content outside the trace with separate access control. The demo measured the trade: with capture OFF the evidence view loses nothing; the model's own account of the amount is lost. Engineering, not legal advice. [Ch. 9]
No sampling at all? Isn't that expensive?	Explanation rights apply to each decision and retention floors run from six months to years, so head-sampling decision spans is hard to defend. Design meaningful spans, keep every decision span, tier retention, move content to cheaper storage. In the demo, 100% is asserted by a test. [Ch. 9]
So does this satisfy the EU AI Act?	No demo satisfies a regulation; conformity is an organisational process. Northstar shows the artefacts Articles 12, 14 and 86 ask for, built from open source. Annex III obligations apply from 2 December 2027. [Ch. 10]
What about the new stateless MCP spec?	MCP `2026-07-28` removed sessions and `initialize`, made `_meta` trace context official, and added `Mcp-Method` / `Mcp-Name` headers a gateway can authorize on. Northstar still runs session-ful streamable HTTP. An amount rule still needs the body. [Ch. 2, 12]

Quote bank

Twelve lines that work said out loud. Most are the repo's own words, written to describe something that was measured.

LINE	SOURCE	USE IT…
"A refusal is the platform working."	demo-runbook.md · beat 1	when the first refused claim appears.
"The model can request. The platform decides if it becomes an effect."	M2 governed design spec	introducing the choke point (slide 19).
"An agent that can choose to ask permission can choose not to."	M2 decisions, Decision 2	when asked why the limit isn't in the agent.
"The absence of those spans is the evidence that nothing happened, rather than something happening and failing."	demo-runbook.md · beat 2	pointing at the missing execute spans.
"Anyone alerting on HTTP status alone will not see this refusal at all."	telemetry-contract.md	on slide 23.
"Fail-open is one word away, and a fail-open gateway is indistinguishable from a governing one until someone reads a trace."	agentgateway/config.yaml	on defaults, or for the policy-outage question.

LINE	SOURCE	USE IT…
“Identity is an input to the decision, not a label on it.”	demo-runbook.md · §9	on slide 26.
“Nothing was overridden. A different caller was asked about.”	demo-runbook.md · beat 4	right after the human approves.
“One word changed on one live object, and what the platform is allowed to do changed with it.”	demo-runbook.md · beat 7	as the flip’s main line.
“The platform API is the resource. Backstage is just the view a human uses to read and change it.”	M6 golden path design spec	as the Act 3 closer and handover line.
“A score set that cannot go to 0.0 is decoration.”	telemetry-contract.md	on slide 31.
“The governance story survives full content redaction intact. The model’s own account of itself does not.”	telemetry-contract.md · OQ-1	on the content-capture slide.

The honest limitations slide

This slide exists because the room will find these anyway. The text below is ready to paste. Keep the wording exact on the Backstage line; the sloppy version (“there is no auth”) is wrong.

> **What Northstar does not do**
>
> • One hop is authorized: `tools/call` on the MCP hop. The other hops carry no identity.
> • The tool and the ledger trust the network. payments-mcp sees no caller; payments-api has no authorization.
> • The human reviewer is unauthenticated. Approval is one human, one click, once.
> • Backstage: the backend checks a credential, but anyone who can reach port 7007 can mint one. They can flip the contract, and nothing else.
> • The claim id stops at the MCP boundary. The evidence joins on trace id.
> • Model calls do not cross the gateway.
> • One GenAI metric of the twelve defined arrives. Sixteen upstream findings, none filed yet.
>
> CONDENSED FROM DOCS/ARCHITECTURE.MD (“WHAT THE SYSTEM DELIBERATELY DOES NOT DO”) AND DOCS/DEMO-RUNBOOK.MD BEAT 8

The call to action

Five verbs, one per slide line. Each points at something the audience can start on Monday without buying anything.

1. **Instrument.** Turn on the telemetry your agent framework already emits, follow `gen_ai.*`, and propagate context across every boundary: A2A, MCP `_meta`, skills, the policy hop.
2. **Put a gateway in the path.** One choke point on tool calls, a decision service that emits a decision span, `failClosed` written down, and network policy so the gateway is the only road.

3. **Make identity an input.** One workload identity per agent (SPIFFE), verified at the gateway, used as half the policy key. Put the human override through the same door under a different identity.
4. **Contribute upstream.** A2A trace context and attributes, MCP baggage, gateway `_meta` extraction, a semantic convention for policy decisions and approvals. Sixteen findings are written up and waiting.
5. **Map to the standard.** ISO/IEC 24970 (AI system logging) is at FDIS stage. A mapping from OTel GenAI semconv to its information model is an open contribution nobody has made.

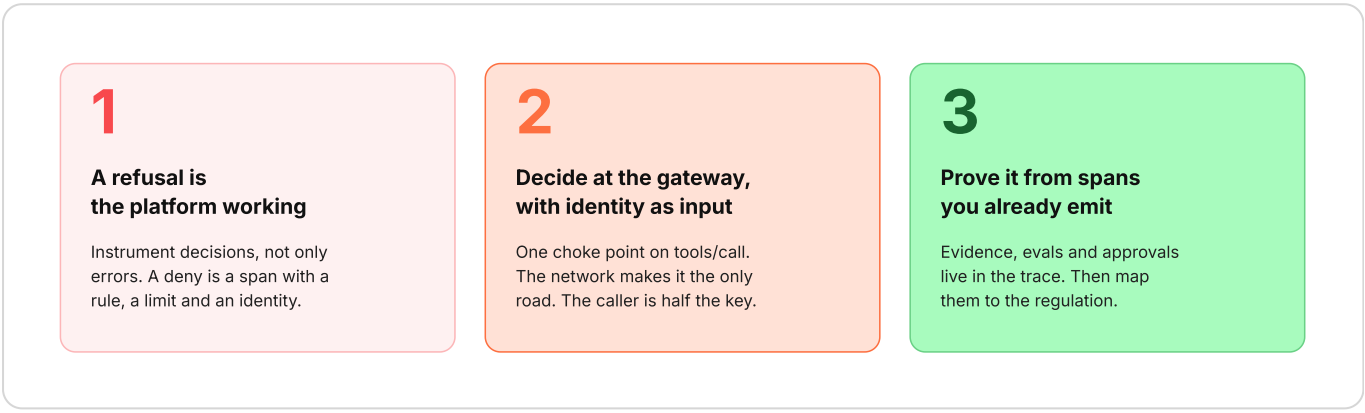


FIGURE 14.4

The closing slide. Each box compresses one act: Act 3's refusal, Act 3's choke point, Act 4's proof. If the audience keeps one sentence per box, the talk worked.

APPENDIX A

Glossary

The terms the book leans on most, in plain words, for a reader who has never run an agent or read a trace.

Definitions are short on purpose. Where a term has a longer treatment, the chapter is named in brackets. Terms specific to Northstar are marked *(Northstar)*.

A2A (Agent2Agent)

An open protocol for one agent to call another over HTTP, JSON-RPC or gRPC. Google announced it in April 2025; v1.0.0 shipped 2026-03-12 and it joined AAIF on 2026-08-17. Northstar pins the Python SDK at 0.3.7. [Ch. 2]

AAIF (Agentic AI Foundation)

A Linux Foundation fund, announced 2025-12-09, that hosts MCP, goose, AGENTS.md, agentgateway, A2A and Agent Router. It is separate from the CNCF.

Absence as evidence

The idea that a missing span proves something did not happen. In a refused Northstar trace, the missing execute spans prove the payment never reached the tool. [Ch. 5]

Agent

A program made of a model, instructions, tools and a loop. The loop keeps asking the model what to do next until the model says it is done.

agentgateway

A Rust proxy for MCP, A2A and LLM traffic. A Linux Foundation project hosted in AAIF, not CNCF. Northstar runs v1.5.0 as the one choke point on MCP tool calls. [Ch. 6]

Attribute

A key-value pair on a span, like `policy.decision = deny`. Attributes are what you search and group by.

Autonomy level *(Northstar)*

A value, `assisted` or `autonomous`, that selects which spending limit applies to an agent. It lives in a Kubernetes object and changes without a redeploy. [Ch. 8]

Backstage

A CNCF developer-portal framework. Northstar uses it at stage 6 for a software catalog and the autonomy-contract card.

Baggage

A W3C standard for carrying small key-value pairs (like a claim id) alongside trace context. Unlike attributes, baggage travels between services. Northstar's claim id is lost as baggage at the MCP boundary.

CEL (Common Expression Language)

A small, safe expression language used in Kubernetes and in agentgateway's built-in authorization rules, for example `jwt.sub == "x" && mcp.tool.name == "y"`.

Choke point

A single place every request of a kind must pass through, so a decision made there cannot be skipped. In Northstar, the gateway on `tools/call`.

Cilium

A CNCF networking layer for Kubernetes based on eBPF. Northstar runs 1.20.1 to make the gateway the only route to the payments tool.

CNCF (Cloud Native Computing Foundation)

The Linux Foundation home of Kubernetes, OpenTelemetry, Cilium, SPIFFE, Backstage and OpenFeature.

Collector (OpenTelemetry)

A service that receives telemetry, processes it and sends it on to one or more backends. "The last observability agent you will ever install."

Content capture

Recording what a model was told and what it said (prompts, outputs, tool arguments) on spans. Opt-in in the conventions. Northstar's switch is `NORTHSTAR_CAPTURE_CONTENT`. [Ch. 9]

Context propagation

Passing trace context from one service to the next so their spans join one trace. Like a parcel tracking number that stays on the box.

DORA (Digital Operational Resilience Act)

EU regulation for financial entities' ICT risk, applied since 2025-01-17. Its logging rules can carry AI Act logs.

EU AI Act

Regulation (EU) 2024/1689. After the Digital Omnibus, obligations for Annex III high-risk systems apply from 2 December 2027. Articles 12 (logs), 14 (human oversight) and 86 (explanation) matter most here. [Ch. 10]

Eval (evaluation)

A score of how well an agent did. Offline evals run on a fixed dataset; online evals score real traces afterwards.

eval-scorer *(Northstar)*

A service that reads a claim's trace back from OpenSearch and writes five policy-adherence scores into that trace. Pull-only and deterministic.

Evidence view *(Northstar)*

A saved OpenSearch dashboard, `northstar-evidence`, that answers "why was this paid?" from existing spans, with no new application code. [Ch. 9]

ExtMCP

agentgateway's gRPC hook that asks an external service to permit, change or deny an MCP request before it is forwarded. Configured as `mcpGuardrails`. [Ch. 6]

Fail-closed / fail-open

What happens when the decision service is unreachable. Fail-closed refuses everything; fail-open lets everything through. Northstar is fail-closed.

FINRA

The US securities industry's self-regulatory body. Its 2026 oversight report names agent scope and traceability as risks.

flagd

OpenFeature's flag evaluation daemon. In Northstar it runs as a native sidecar in the policy service's pod and watches the FeatureFlag object.

gen_ai.* (GenAI semantic conventions)

OpenTelemetry's shared names for model calls, agents and tools, such as `gen_ai.request.model`. All still Development stability, now in their own repository. [Ch. 3]

Golden path

A supported, paved way for teams to build and run software on a platform. Northstar's stage 6.

Guardrail span

agentgateway's CLIENT span for the call to the policy service. It shows the hop happened, not what was decided.

HITL (human in the loop)

A design where a person approves or overrides an agent's action. [Ch. 8]

Instrumentation

Code that creates telemetry. It can be automatic (a library patches your framework) or manual (you write the span yourself).

ISO/IEC 24970

A draft international standard for AI system logging, at FDIS stage since 2026-08-28. Nobody has yet mapped OTel GenAI conventions to it.

Jaeger

A CNCF trace backend and UI. Northstar runs 2.20.0 at `localhost:16686`.

JWT (JSON Web Token)

A signed token with claims like subject and audience. Like a sealed envelope: anyone can read it, nobody can change it without breaking the seal.

jwt.sub

The attribute agentgateway puts on its span with the verified caller identity. Semconv would call it `user.id`.

LLM-as-judge

Using another model call to score an agent's output or trajectory. Northstar deliberately uses deterministic scores instead. [Ch. 9]

managedFields

Kubernetes metadata that records which tool last wrote each field of an object. It shows ownership now, not a history of changes.

MCP (Model Context Protocol)

An open protocol that hands an agent tools it did not ship with. Current spec: `2026-07-28`, which is stateless. Donated to AAIF in December 2025. [Ch. 2]

MCP server

A process that exposes tools over MCP. Northstar's `payments-mcp` is an adapter over the payments API.

mTLS (mutual TLS)

Encrypted connections where both sides prove who they are with certificates. Stronger than a bearer token; deferred in Northstar.

Normalizer processor

A Collector processor that renames other vocabularies (OpenInference, OpenLLMetry) into `gen_ai.*`.

OpenFeature

A CNCF standard API for feature flags, independent of vendor.

OpenSearch

An open-source search and analytics engine. Northstar stores spans there for the evidence view.

OpenTelemetry (OTel)

The CNCF project that standardises how software produces and ships traces, metrics and logs.

Parity guard *(Northstar)*

A test that the same spans reached two backends. It exists because OpenSearch once silently dropped exactly the decision spans.

PDP / PDS

Policy decision point: the component that answers permit or deny. Northstar's is the Policy Decision Service (PDS), its own code. [Ch. 7]

policy.decision span *(Northstar)*

The PDS's span for each decision, with the verdict, rule, limit, amount, caller and autonomy level. There is no convention for this yet.

Refusal *(Northstar)*

A deny from the platform. Treated as success: terminal, structured, and never an ERROR on the decision span.

Sampling

Keeping only some traces to save cost. Head sampling decides at the start; tail sampling decides after the trace completes. Northstar samples nothing.

Semantic conventions (semconv)

OpenTelemetry's agreed names for attributes and spans, so every tool means the same thing by the same word.

SEP-414

The MCP proposal that carries W3C trace context in `params._meta`. Part of the spec since `2026-07-28`.

Skill

A folder with a `SKILL.md` file (and optional scripts) that teaches an agent a task. An open standard since 2025-12-18. [Ch. 2]

Span

One timed operation, with a name, start, end, attributes and a parent. The building block of a trace.

Span link

A pointer from one span to another that is not its parent. Used for causes that are far apart in time, like a human approving a refusal minutes later.

Span status

Unset, OK or ERROR. Northstar keeps a policy deny at Unset.

SPIFFE

A CNCF standard for workload identity. A SPIFFE ID looks like `spiffe://northstar.local/ns/<ns>/sa/<name>`. Like a passport for a program.

SPIRE

The reference implementation of SPIFFE. It attests workloads and issues their identity documents. Northstar runs 1.15.3.

SR 26-2

US banking supervisory guidance on model risk (2026-04-17), replacing SR 11-7. It puts generative and agentic AI out of scope.

Stage *(Northstar)*

One of six cluster profiles, from 1 (Connected) to 6 (Golden path), each installing only what it needs.

Tool

A function an agent can call to act on the world, like `execute_payment`.

Trace

All the spans for one request, joined by a shared trace id. The story of what happened.

traceparent

The W3C header that carries trace context: `version-traceid-parentid-flags`.

Workload identity

An identity for a program rather than a person, issued from facts about where it runs.

APPENDIX B

Verify before a slide

Every fast-moving or unverified claim in the book, with how to check it and when.

These claims were accurate or explicitly unverified on 2026-09-28. “Slide freeze” means the final deck review; “week of” means the week of the talk. AAIF news tends to land around AGNTCon + MCPCon North America (2026-10-22/23), so check those items after it.

CHECK BEFORE A SLIDE

Three standing rules from the repo apply to every row. Publish trace *shape*, never span totals. Derive a count from a command on the day; never retype it from a document. Never promise a payment id on stage.

CLAIM	WHY IT MAY BE WRONG	HOW TO CHECK	BY WHEN
agentgateway v1.5.0 is the latest stable; semconv HTTP attributes (#3182) only in v1.6.0-alpha. **Verified 2026-09-28:** all 32 gateway spans in the deny trace carry the old `http.*` names, none of the semconv ones	alpha.1 said GA in “roughly 2 weeks” on 2026-09-14	agentgateway releases; if GA, rerun `test_gateway_trace.py` (designed to go red)	Slide freeze and week of
agentgateway reads trace context from headers only (Finding 6)	1.6 might add `_meta` ingress extraction	1.6 release notes; source `trc.rs`	Slide freeze
~~`guardrail_checks_total` counts MCP guardrail denials~~ **Refuted 2026-09-28:** no such metric on v1.5.0; `upstream_call_duration_seconds{subtype="Guardrail"}` counts every call, with no permit/deny label	Scraped `:15020/metrics` after the demo	Say “the gateway counts guardrail calls, not decisions”	Settled
`requests_total{reason="MCP"}` / `mcp_requests_total` show denials. **Verified 2026-09-28:** only indirectly (one `reason="MCP"` per denied execute; `mcp_requests_total` counts denied calls too). None reach Prometheus	The label means “answered by the MCP layer”, not “denied”	Scrape the pod’s `:15020`; Prometheus has no `agentgateway_*` series	Before any gateway metric slide
PDS outage shows non-zero `grpc.status` on `Guardrail`	Outage span not published as a measured attribute block	Point the guardrail at a dead address, capture the span	Before Q3 is answered with specifics
Gateway tracing `attributes` / `resources` fields could add `user.id`	Documented, not verified on v1.5.0	Try on a scratch config	Only if mentioned

CLAIM	WHY IT MAY BE WRONG	HOW TO CHECK	BY WHEN
“agentgateway is a Linux Foundation project in AAIF, not CNCF”	Governance can move	agentgateway.dev blog; aaif.io projects	Week of
Sixteen upstream findings, none filed	The owner may file, or someone else may file duplicates	Search each upstream tracker	Slide freeze
GenAI semconv repo has no release; all Development	40+ fragments waiting; a release may land	github.com/open-telemetry/semantic-conventions-genai releases	Week of
`gen_ai.client.token.usage` replaced (PR #374, 2026-09-22). **Verified 2026-09-28:** it is still the only GenAI metric in Prometheus, from `settlement-agent`	Instrumentations adopt over months; Northstar still emits the old name	`gen-ai-token-metrics.md`; Prometheus series	Slide freeze
GenAI normalizer processor availability in contrib	Quoted from the earlier deck (“contrib >0.153.0”)	collector-contrib release notes	Slide freeze
MCP current spec is `2026-07-28`; registry still preview	Spec and registry move	modelcontextprotocol.io versioning and registry pages	Week of
OTel `mcp.session.id` out of step (issue #437)	From a search summary	Read the issue directly	Slide freeze
A2A signed Agent Cards: MUST or optional	Spec fetch was truncated	a2a-protocol.org specification §8.4	Only if mentioned
A2A defines no trace-context carrier	No evidence found either way	Search the spec and proto for traceparent	Before saying it on stage
Agent Skills AAIF proposal #47 not accepted	Under TC vote on 2026-09-28	github.com/aaif/project-proposals/issues/47	After AGNTCon
Snyk ToxicSkills figures; Koi “ClawHavoc” count	ClawHavoc from secondary source only	Primary reports	Only if quoted
EU Omnibus left Articles 12–14, 19, 26, 86 unchanged	Based on a secondary analysis, not checked line by line	Regulation (EU) 2026/1744 text on EUR-Lex	Slide freeze
Art. 13(3)(f) wording (“collect, store and interpret the logs”)	Quoted from memory	Article text	Before quoting
ISO/IEC 24970 at FDIS (stage 50.20)	A vote could publish it around late October	iso.org/standard/88723	Week of
EN 18286 not yet cited in the Official Journal	Citation could come any time	OJ harmonised standards list	Only if mentioned
DORA RTS Art. 12 logging enumeration	Via a secondary summary	Delegated Reg. (EU) 2024/1774 on EUR-Lex	Before the requirement table goes out
Colorado SB 26-189 enforcement stayed (*xAI v. Weiser*)	Litigation moves	CourtListener docket	Week of

CLAIM	WHY IT MAY BE WRONG	HOW TO CHECK	BY WHEN
Gartner “15% of GenAI deployments instrumented”	Unverified and widely repeated	**Do not quote**	—
Hugging Face intrusion attributed to an OpenAI evaluation run	Attribution from secondary reports	OpenAI primary disclosure	Only if mentioned
CNCF agentic whitepaper (toc#1746) published	Unverified	cncf/toc#1746	Only if mentioned
OTel GenAI SIG formed April 2024; ServiceNow bought Traceloop	Secondary sources only	OTel community repo; ServiceNow press	Only if mentioned
NAIC bulletin adopted by about 25 jurisdictions; Entra Agent ID GA month	Approximate or unverified	NAIC map; Microsoft announcement	Only if mentioned
Any Northstar test total	The contract’s gate row is PREDICTED, NOT MEASURED	Run the gate after the last change; read the row’s status	**Do not quote totals** unless re-measured
Prometheus probe series `northstar.meterprovider.probe`	Aged out of 15-day retention on 2026-09-28	Re-send the probe before citing it live	Before a metrics beat
Span counts and whole-trace totals	Vary run to run (“seven numbers, one shape”)	Publish shape; count live if needed	Always
Runbook beat 6: `stopped_when_refused` scores 1.0 on the real run	**Refuted 2026-09-28:** it and `amount_executed_matches_approved` scored 0.0, because the €5,800 claim reuses CLM-1842 and reads as a retry by the refused identity	Set `sinceMs` just before the deny run, or give the permit run its own claim id; rehearse beat 6 end to end	Before beat 6 is rehearsed
The browser root span `claim.submit` always arrives	Exporter flushes every 5 s; closing or reloading sooner loses the root	Leave the tab open 8 s after each submit	Every rehearsal
Jaeger shows a waterfall when a trace opens	Jaeger 2.20 opens traces in “GenAI View” by default	Switch the view selector to “Trace Timeline” before going on stage	T-10
Stage 6 is 35 pods and about 6 GiB	2026-09-28 read 35 pods, 5.11 GiB; 2026-09-15 read 6.05 GiB	`docker stats`; `kubectl get pods -A`	On the presenting machine
SPIRE OIDC provider init image pulls cleanly	Digest pinned against a rolling `:latest` tag	Pre-pull, or recreate the day before	Day before
`ANTHROPIC_API_KEY` reads SET	Empty key: pods Running, first claim fails	`setup-kind.sh` output	T-30
Backstage catalog shows nine Components (**verified 2026-09-28**)	Retyped count; the test is a directory walk	Count in the UI on the day	T-10

APPENDIX C

Source index

External sources behind the book, grouped by topic, with dates, plus the repo documents.

Dates are publication, release or effective dates; "accessed" marks a living page read on 2026-09-28. Items marked (secondary) come from press, law firms or vendors writing about someone else. This is the book's only source list.

OPENTELEMETRY AND GENAI SEMANTIC CONVENTIONS

1. Semantic conventions CHANGELOG (v1.42.0 moves GenAI out) — https://github.com/open-telemetry/semantic-conventions/blob/main/CHANGELOG.md
   2026-06-12
2. PR #3696, the repo split — https://github.com/open-telemetry/semantic-conventions/pull/3696
   2026-05-05
3. semantic-conventions-genai repository — https://github.com/open-telemetry/semantic-conventions-genai
   created 2026-05-05 · no release as of 2026-09-28
4. GenAI agent spans — https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/gen-ai-agent-spans.md
   commit e57c543, 2026-09-24
5. GenAI spans and content capture — https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/gen-ai-spans.md
   commit e57c543, 2026-09-24
6. GenAI events, including `gen_ai.evaluation.result` — https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/gen-ai-events.md
   commit e57c543, 2026-09-24
7. GenAI token metrics (PR #374) — https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/gen-ai-token-metrics.md
   merged 2026-09-22
8. MCP semantic conventions — https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/mcp.md
   commit e57c543, 2026-09-24
9. GenAI attribute registry — https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/registry/attributes/gen-ai.md
   commit e57c543, 2026-09-24
10. GenAI issues (A2A proposals #70, #254, #335; MCP session #437) — https://github.com/open-telemetry/semantic-conventions-genai/issues
    accessed 2026-09-28
11. OpenInference code grant accepted — https://github.com/open-telemetry/community/issues/3467
    2026-06-16
12. OpenLLMetry donation closed — https://github.com/open-telemetry/community/issues/2571
    2026-06-16
13. OTel Python GenAI instrumentations — https://github.com/open-telemetry/opentelemetry-python-genai
    1.2b0, 2026-09-24
14. OTel blog, "AI Agent Observability – Evolving Standards and Best Practices" — https://opentelemetry.io/blog/2025/ai-agent-observability/
    2025-03-06
15. OTel blog, "Inside the LLM Call" — https://opentelemetry.io/blog/2026/genai-observability/
    2026-05-14
16. OTel blog, deprecating the Span Events API — https://opentelemetry.io/blog/2026/deprecating-span-events/
    2026-03-17
17. OTel support maturity model proposal — https://github.com/open-telemetry/community/issues/3247
    cited in *Taming Complexity*
18. Microsoft Agent Framework observability — https://learn.microsoft.com/en-us/agent-framework/agents/observability
    2026-09-19 · hosted MCP breaks the trace
19. Google ADK traces — https://adk.dev/observability/traces/
    accessed 2026-09-28
20. Pydantic AI and Logfire — https://pydantic.dev/docs/ai/integrations/logfire/
    accessed 2026-09-28
21. Claude Code monitoring — https://code.claude.com/docs/en/monitoring-usage
    accessed 2026-09-28

MCP, A2A, SKILLS AND AAIF

1. MCP versioning (current version 2026-07-28) — https://modelcontextprotocol.io/specification/versioning
   accessed 2026-09-28
2. MCP 2026-07-28 changelog — https://modelcontextprotocol.io/specification/2026-07-28/changelog
   2026-07-28
3. MCP 2026-07-28 release post — https://blog.modelcontextprotocol.io/posts/2026-07-28/
   2026-07-28
4. MCP security best practices — https://modelcontextprotocol.io/specification/2026-07-28/basic/security_best_practices
   2026-07-28
5. SEP-414, trace context in `_meta` — https://github.com/modelcontextprotocol/modelcontextprotocol/pull/414
   merged 2026-02-26

6. MCP 2025-06-18 changelog (batching removed) — https://modelcontextprotocol.io/specification/2025-06-18/changelog
2025-06-18
7. MCP roadmap — https://modelcontextprotocol.io/development/roadmap
updated 2026-08-22
8. MCP registry — https://modelcontextprotocol.io/registry/about
accessed 2026-09-28 · preview
9. Anthropic donates MCP, AAIF formed — https://www.anthropic.com/news/donating-the-model-context-protocol-and-establishing-of-the-agentic-ai-foundation
2025-12-09
10. Linux Foundation, AAIF formation — https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation
2025-12-09
11. AAIF adds 43 members — https://aaif.io/press/agentic-ai-foundation-adds-43-new-members-as-enterprise-and-government-adoption-of-open-agent-standards-accelerates/
2026-05-18
12. A2A releases — https://github.com/a2aproject/A2A/releases
v1.0.0 2026-03-12
13. A2A v1.0 announcement — https://a2a-protocol.org/latest/blog/2026/03/12/a2a-protocol-ships-v10-production-ready-standard-for-agent-to-agent-communication/
2026-03-12
14. A2A specification — https://a2a-protocol.org/latest/specification/
accessed 2026-09-28
15. A2A and MCP — https://a2a-protocol.org/latest/topics/a2a-and-mcp/
accessed 2026-09-28
16. A2A joins AAIF — https://aaif.io/blog/a2a-joins-aaif
2026-08-17
17. Linux Foundation launches A2A project — https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents
2025-06-23
18. Skills launch — https://claude.com/blog/skills
2025-10-16, updated 2025-12-18
19. Agent Skills specification — https://agentskills.io/specification
accessed 2026-09-28
20. Agent Skills AAIF proposal #47 — https://github.com/aaif/project-proposals/issues/47
opened 2026-09-24
21. Snyk, ToxicSkills — https://snyk.io/blog/toxicskills-malicious-ai-agent-skills-clawhub/
2026-02-05

GATEWAYS, IDENTITY AND CLOUD-NATIVE INFRASTRUCTURE

1. agentgateway releases — https://github.com/agentgateway/agentgateway/releases
v1.5.0 2026-08-27; v1.6.0-alpha.2 2026-09-22
2. agentgateway PR #3182 (semconv HTTP attributes) — https://github.com/agentgateway/agentgateway/pull/3182
merged 2026-09-01
3. Solo contributes agentgateway to the Linux Foundation — https://agentgateway.dev/blog/2025-08-25-solo-contributes-agentgateway-to-lf/
2025-08-25
4. agentgateway joins AAIF — https://agentgateway.dev/blog/2026-06-04-agentgateway-joins-aaif/
2026-06-04
5. agentgateway tracing setup — https://agentgateway.dev/docs/standalone/latest/documentation/observability/traces/setup/
accessed 2026-09-28
6. agentgateway trace attribute reference — https://agentgateway.dev/docs/standalone/latest/documentation/observability/traces/attribute-reference/
accessed 2026-09-28
7. agentgateway metrics reference — https://agentgateway.dev/docs/standalone/latest/documentation/observability/metrics/reference/
accessed 2026-09-28
8. agentgateway MCP authorization — https://agentgateway.dev/docs/standalone/latest/documentation/configuration/security/mcp-authz/
accessed 2026-09-28
9. agentgateway MCP guardrails — https://agentgateway.dev/docs/standalone/latest/documentation/mcp/guardrails/about/
accessed 2026-09-28
10. agentgateway security advisories — https://github.com/agentgateway/agentgateway/security/advisories
accessed 2026-09-28
11. kgateway v2.2 release — https://kgateway.dev/blog/kgateway-v2.2-release-blog/
Feb 2026
12. Envoy AI Gateway v1.1 release notes — https://aigateway.envoyproxy.io/release-notes/v1.1/
2026-08-21
13. Istio ambient and agentgateway — https://istio.io/latest/docs/ambient/usage/agentgateway/
accessed 2026-09-28
14. kagent releases — https://github.com/kagent-dev/kagent/releases
v0.10.2, 2026-09-23
15. agent-sandbox releases — https://github.com/kubernetes-sigs/agent-sandbox/releases
v1.0.x, 2026
16. Dapr Agents GA — https://www.cncf.io/announcements/2026/03/23/general-availability-of-dapr-agents-delivers-production-reliability-for-enterprise-ai/
2026-03-23
17. CNCF TOC agentic initiative — https://github.com/cncf/toc/issues/1746
June 2025
18. IETF draft-klrc-aiagent-auth — https://datatracker.ietf.org/doc/draft-klrc-aiagent-auth/
July 2026
19. OpenID Foundation, Identity Management for Agentic AI — https://arxiv.org/abs/2510.25819
October 2025
20. Kyverno graduation — https://www.cncf.io/announcements/2026/03/24/cloud-native-computing-foundation-announces-kyvernos-graduation/
2026-03-24

REGULATION AND STANDARDS

1. Regulation (EU) 2026/1744, Digital Omnibus on AI — https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=OJ:L_202601744
   OJ 2026-07-24, in force 2026-07-27
2. European Parliament Legislative Train, Digital Omnibus on AI — https://www.europarl.europa.eu/legislative-train/package-digital-package/file-digital-omnibus-on-ai
   updated 2026
3. AI Act Article 12 (and sibling articles 14, 19, 26, 86) — https://artificialintelligenceact.eu/article/12/
   Reg. 2024/1689
4. CypherOn, what 2026/1744 changed (secondary) — https://cypheron.cz/en/resources/ai-act/omnibus-2026
   2026
5. Gibson Dunn on the Omnibus agreement (secondary) — https://www.gibsondunn.com/eu-ai-act-omnibus-agreement-postponed-high-risk-deadlines-and-other-key-changes/
   May 2026
6. ISO/IEC FDIS 24970, AI system logging — https://www.iso.org/standard/88723.html
   stage 50.20, 2026-08-28
7. CEN-CENELEC, EN 18286 — https://www.cencenelec.eu/news-events/news/2026/en-in-the-spotlight/2026-07-30-ai-quality-management/
   2026-07-30
8. DORA RTS on ICT risk management, Delegated Reg. (EU) 2024/1774 — https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=OJ:L_202401774
   OJ 2024-06-25
9. EIOPA, critical ICT third-party providers designated — https://www.eiopa.europa.eu/european-supervisory-authorities-designate-critical-ict-third-party-providers-under-digital-2025-11-18_en
   2025-11-18
10. CJEU C-203/22 press release — https://curia.europa.eu/site/upload/docs/application/pdf/2025-02/cp250022en.pdf
    2025-02-27
11. CRA reporting obligations — https://digital-strategy.ec.europa.eu/en/policies/cra-reporting
    applies from 2026-09-11
12. Federal Reserve SR 26-2 — https://www.federalreserve.gov/supervisionreg/srletters/SR2602.htm
    2026-04-17
13. OCC Bulletin 2026-13 — https://www.occ.gov/news-issuances/bulletins/2026/bulletin-2026-13.html
    2026-04-17
14. FINRA 2026 Annual Regulatory Oversight Report — https://www.finra.org/sites/default/files/2025-12/2026-annual-regulatory-oversight-report.pdf
    Dec 2025
15. NAIC AI Model Bulletin adoption map — https://content.naic.org/sites/default/files/cmte-h-big-data-artificial-intelligence-wg-map-ai-model-bulletin.pdf
    as of 2026-08-31
16. Colorado SB 26-189 — https://leg.colorado.gov/bills/sb26-189
    signed 2026-05-14, effective 2027-01-01
17. *X.AI LLC v. Weiser* docket — https://www.courtlistener.com/docket/73171074/x-ai-llc-v-weiser/
    2026
18. HIPAA Security Rule update delayed (secondary) — https://www.clarkhill.com/news-events/news/hipaa-security-rule-update-delayed-until-2027/
    Jul 2026
19. TM Forum Autonomous Networks resources — https://www.tmforum.org/topics/an-resources/
    2025
20. FCA Mills Review — https://www.fca.org.uk/publications/calls-input/review-long-term-impact-ai-retail-financial-services-mills-review
    2026-07-06

SECURITY FRAMEWORKS AND INCIDENTS

1. OWASP Top 10 for Agentic Applications — https://genai.owasp.org/2025/12/09/owasp-top-10-for-agentic-applications-the-benchmark-for-agentic-security-in-the-age-of-autonomous-ai/
   2025-12-09
2. OWASP Top 10 for LLM Applications 2025 — https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/
   Nov 2024
3. CoSAI MCP security taxonomy — https://www.oasis-open.org/2026/01/27/coalition-for-secure-ai-releases-extensive-taxonomy-for-model-context-protocol-security/
   2026-01-27
4. NIST AI Agent Standards Initiative — https://www.nist.gov/artificial-intelligence/ai-agent-standards-initiative
   2026-02-17
5. NIST CAISI agent-hijacking evaluation — https://www.nist.gov/news-events/news/2025/01/technical-blog-strengthening-ai-agent-hijacking-evaluations
   2025-01-17
6. Invariant Labs, MCP tool poisoning — https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks
   2025-04-01
7. Simon Willison, Supabase MCP lethal trifecta — https://simonwillison.net/2025/Jul/6/supabase-mcp-lethal-trifecta/
   July 2025
8. Anthropic, GTG-1002 report — https://assets.anthropic.com/m/ec212e6566a0d47/original/Disrupting-the-first-reported-AI-orchestrated-cyber-espionage-campaign.pdf
   2025-11-13
9. Replit / SaaStr database deletion — https://incidentdatabase.ai/cite/1152/
   July 2025
10. Forrester, agent control planes and standards — https://www.forrester.com/blogs/agent-control-planes-still-need-a-robust-standards-stack/
    March 2026
11. Gartner, 40% of agentic AI projects canceled by 2027 — https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027
    2025-06-25

EVALS

1. Google ADK evaluation criteria — https://adk.dev/evaluate/criteria/ accessed 2026-09-28
2. LangChain agentevals — https://github.com/langchain-ai/agentevals 2026
3. Arize Phoenix LLM evals — https://arize.com/docs/phoenix/evaluation/llm-evals accessed 2026-09-28

EARLIER TALKS

1. *Your Agent Did WHAT? Forensic observability for systems that don't leave obvious footprints* (Adriana Villela, Kasper Borg Nissen) — https://github.com/kaspernissen/your-agent-did-what SRE Day London Q3 2026
2. *Taming Complexity: Building Observable Workflows with Dapr and OpenTelemetry* (Mauricio Salatino, Kasper Borg Nissen) KubeCon EU; upstream fixes cited: langchain4j #4616, a2a-java #707, dapr java-sdk #1693

NORTHSTAR REPOSITORY DOCUMENTS

1. `README.md` status per milestone, quick start, stages table
2. `docs/architecture.md` components, hops, protocols, span-by-span trace, "See it yourself", limitations
3. `docs/telemetry-contract.md` measured instrumentation, eight convention gaps, pin table, test counts
4. `docs/upstream-findings.md` Findings 1–16 and OQ-U1, none filed
5. `docs/demo-runbook.md` operator's document for the live demo
6. `docs/superpowers/specs/` (six milestone design specs, 2026-09-07 to 2026-09-15) design intent, annotated in place where reality diverged
7. `docs/superpowers/plans/` implementation plans per milestone
8. `.superpowers/sdd/*/progress.md` private build ledger: rulings with cost-if-wrong; git-ignored, not public